

C++ Tutorial

This C++ tutorial explains each topic with programs, such as Introduction, Installation, Variables, Data Types, Operators, Control Statement, Functions, Arrays, Classes and Objects, OOP, Constructors, Inheritance, Polymorphism, Encapsulation, Abstraction, Namespaces, Templates, String, Pointers, Signal Handling, Exceptions, Files & Streams, STL Functions, and many other functions.

What is C++?

C++ is a general-purpose programming language that was developed by Bjarne Stroustrup at Bell Labs in the early 1980s. The C++ language is an extension to the C language and provides several additional features, such as object-oriented programming, exception handling, and templates. It is mainly used to create system software, high-performance applications, embedded systems, game development, GUI, and many other applications.

Features of C++

There are several features of the C++ programming language. Some of them are as follows:

- It is an object-oriented, procedural, and middle-level programming language.
- C++ is used to create high-level applications.
- It is a highly preferred programming language that is used for cross-platform applications.
- It is compatible with multiple platforms.
- It supports both operator overloading and function overloading.
- It offers low-level memory manipulation with pointers.
- It provides several built-in functions and operators.

C++ Example

```
#include <iostream>
using namespace std; //using standard namespace
int main() { //main function
    cout << "Hello, Welcome to TpointTech!" << endl;
    return 0; //return function
}
```

Why Learn C++?

It includes several features of procedural, object-oriented, and generic programming languages that help to write efficient, reusable, and flexible code. These excellent features make it a better choice for developing system software, GUI, and several other applications. Here we have described some of the main advantages of learning C++:

- C++ is an easy-to-learn and implement.
- It offers Cross-Platform Support that enables you to run C++ programs on multiple operating systems.
- Using C++, developers can develop system software, game development applications and many others.
- It provides a large community and resources.

Object-Oriented Programming (OOP)

C++ supports object-oriented programming, and the four major pillars of object-oriented programming (OOP) used in C++ are:

1. Inheritance
2. Polymorphism
3. Encapsulation
4. Abstraction

C++ Standard Libraries

Standard C++ programming is divided into three important parts:

- The core library includes the data types, variables, literals, etc.

- The standard library includes a set of functions that manipulate strings, files, and many others.
- The STL includes a set of methods to manipulate a data structure.

Usage of C++

With the help of the C++ programming language, you can develop different types of secure and robust applications. Some of them are as follows:

- Window application
- Client-Server application
- Device drivers
- Embedded firmware etc

Difference between C and C++

Features	C	C++
Definition	C follows the procedural style programming.	C++ is multi-paradigm. It supports both procedural and object oriented.
Security	Data is less secured in C.	In C++, we can use modifiers for class members to make it inaccessible for outside users.
Approach	C follows the top-down approach.	C++ follows the bottom-up approach.
Function Overloading	C does not support function overloading.	C++ supports function overloading.
Functions Uses	In C, we can't use functions in structure.	In C++, we can use functions in structure.
Reference	C does not support reference variables.	C++ supports reference variables.
Input/Output	In C, scanf() and printf() are mainly used for input/output.	C++ mainly uses stream cin and cout to perform input and output operations.
Operator Overloading	Operator overloading is not possible in C.	Operator overloading is possible in C++.
Categories	C programs are divided into procedures and modules	C++ programs are divided into functions and classes.
Namespace	C does not provide the feature of namespace.	C++ supports the feature of namespace.
Exception Handling	Exception handling is not easy in C. It has to perform using other functions.	C++ provides exception handling using Try and Catch block.
Inheritance	C does not support the inheritance.	C++ supports inheritance.

C++ Keywords

C++ keywords play a crucial role in defining the syntax and functioning of the language. They include reserved words with functions, such as specifying data types, managing program flow, and activating additional features. Understanding these terms is essential for good C++ programming and enables programmers to build reliable and adaptable software.

A keyword is a reserved word that contains special meanings in C++ programming language. We cannot use it as a variable name, constant name, etc. These keywords are used to specify the components of the C++ language, including void, int, public, return, etc.

Keywords in C++

C++ programming language introduces several keywords with their updated version. Here is a table of 32 Keywords in C++ Language, which are also available in C language.

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

A list of 30 Keywords in C++ Language that are not available in C language is given below.

asm	dynamic_cast	namespace	reinterpret_cast	bool
explicit	new	static_cast	false	catch
operator	template	friend	private	class
this	inline	public	throw	const_cast
delete	mutable	protected	true	try
typeid	typename	using	virtual	wchar_t

Keywords Available in Both C and C++

Keyword Name	Description
auto	In C++, the auto keyword is mainly used for type inference. It enables the compiler to determine a variable's data type from its initializer expression.
break	By using the break keyword, the execution of a loop or switch statement is stopped.
case	It is used in a switch statement to provide several scenarios in which different code blocks should be run depending on the result of an expression.
char	The char keyword is used to declare character variables or data types.
const	Constant values or pointers that refer to constants are declared with the keyword const.
continue	This keyword is used to go on to the next iteration of a loop without running the last remaining statements.
default	When no other case matches, it is used in a switch statement to provide a default case.
do	This keyword initiates a loop that runs a series of statements as long as a certain condition is met.
double	Double-precision floating-point variables and data types are declared with the double keyword.
else	It works in conjunction with the if statement to specify an alternative code snippet that is run if the condition is false.
enum	An enumeration is defined by the enum keyword. A user-defined type is made up of a collection of named integer constants.
extern	It is used to declare variables or functions that are declared in another source file.
float	The float keyword is used to declare single-precision floating-point variables or data types.
for	It begins a loop by introducing initialization, condition, and update expressions.
goto	The goto keyword is used to give an unconditional jump from the goto to a labeled statement within the same function.

if	This operator is used to run a block of code only if a certain condition is met.
int	The int keyword is used to declare the integer variables or data types.
long	The long keyword is utilized to declare the integer variables with a wider range than int.
register	The compiler is advised to put the variable in a register for quicker access.
return	This keyword is used to end a function and, if desired, to provide the caller a value back.
short	The short keyword is used to declare an integer short variables.
signed	It is used to declare variables using signed integers.
sizeof	The sizeof() keyword is used to calculate the size (in bytes) of a data type or object.
static	The static keyword is used to declare static variables, which keep their values across function calls.
struct	A composite data type with a single name for all its variables is defined with the struct keyword.
switch	It is used to construct a switch statement, which compares an expression to its value and chooses a code block accordingly.
typedef	The typedef keyword is used to give a data type an alias (alternative name).
union	It is used to define a union, which resembles a struct but has one shared memory location for all its members.
unsigned	The unsigned keyword is used to define unsigned integer variables.
volatile	This keyword instructs the compiler that a variable's value may change at any point, even if it is not immediately apparent from the logic of the program.
void	This keyword is used to indicate a function that does not return a value. It also determine a pointer to a data with an unspecified type.
while	This keyword introduces a loop that repeatedly runs a set of statements if a certain condition is met.

Additional C++ Keywords Not Available in C

Keyword Name	Description
asm	The asm keyword is used to write programs in assembly language that can be inserted into C++ programs. It enables the writing of inline assembly code by programmers for certain hardware operations.
dynamic_cast	In C++, dynamic type casting uses this keyword. In object-oriented programming, it is mostly used for secure downcasting (from a base class to a derived class) when polymorphism is present.
namespace	The namespace keyword is used to construct a named scope, which aids in organizing and bringing together similar code pieces. It is very helpful in avoiding name disputes.
reinterpret_cast	In C++, this keyword is used to type casting at the lowest level. Without altering the actual data, it may be used to change the type of a pointer to a different type.
bool	The bool keyword is used to define the boolean data type that can only store true or false values. It is frequently employed in conditional statements and logical procedures.
explicit	In C++, the explicit keyword is used to specify that a constructor shouldn't be used automatically to convert types. It reduces the

	likelihood of accidental automated type conversions.
new	The heap memory is used to dynamically allocate memory for objects during runtime. It gives back a pointer to the RAM that was allotted.
static_cast	In C++, this keyword is utilized for fundamental type casting. It may be used to safely convert between kinds of related types.
false	The boolean value false is represented by the term false. One of C++'s two boolean literals, the other being true.
catch	The catch keyword in exception handling is used to capture and manage errors that the try block throws. It designates a section of code to run in response to a certain exception being thrown.
operator	In C++, standard operators can be overloaded to operate with user-defined types by using the operator keyword. It permits the development of unique operator behavior.
template	By parameterizing types and functions, templates are defined by the template keyword, which enables the generic programming. Templates provide for flexibility and reuse of code.
friend	A class or function is designated as a friend of another class using the friend keyword. Friends have access to their classmate's private information and protected information.
private	The access level of class members is specified using the private keyword. Only the class itself has access to private members.
class	In C++, a class is defined with the class keyword. A user-defined data type called a class that contains both data and functions.
this	It is a reference to the current instance of a class that is referred to by the term. Within its member functions, it is utilized to access members of the class.
inline	The inline keyword is used to instruct the compiler to expand a function inline. As a result, the performance can be increased by lowering function call overhead.
public	The access level of class members is specified with the public keyword. Access to public members is available during the whole program.
throw	The throw keyword is used to manually throw an exception in C++. It is a crucial component of the C++ exception-handling system.
const_cast	This keyword is used to remove a variable's const-ness. It mostly serves to change a variable's type qualifiers.
delete	The memory that was previously allocated using the new keyword is released with the delete keyword. Memory leaks must be avoided at all costs.
mutable	In C++, the mutable keyword is used to denote the ability to change a member of a const object. It is frequently employed when member variables within const member functions need to be updated.
protected	The protected keyword is used to specify the access level of class members. Accessible inside the class and classes descended from it are protected members.
truth	The word "true" stands for the boolean value "true". Along with false, it is one of the two boolean literals in C++.
try	In exception handling, the try keyword is used to surround a block of code that may throw exceptions. One or more catch blocks are

	placed after it to deal with certain exceptions.
typeid	The typeid keyword is used to learn more about an expression's type. It is frequently employed for runtime type identification.
typename	In templates, we may use the typename keyword to indicate that a dependent name is a type. When dealing with template metaprogramming, it is frequently employed.
using	Namespace aliasing, template specialization, and declarations all make use of the keyword. It facilitates namespace management and increases code readability.
virtual	The terms inheritance and polymorphism are used together to describe the concept of virtual. It designates a member function as virtual so that descendant classes may override it.
Wchar_t	A wide character data type that can store expanded character sets and support internationalization is defined by the wchar_t keyword.

C++ Operators

In C++, **operators** are special symbols that are used to perform operations on variables and values on operands. Operators are essential concepts of any programming language. There can be several operations, including arithmetic, value assignment, bitwise, and logical computations.

Types of Operators

There are following types of operators to perform different types of operations in C++ language.

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Ternary or Conditional Operators
- Misc Operators

1. Arithmetic Operators:

Operators	Name	Symbol	Description
Binary	Addition	+	It is utilized to add two values.
	Subtraction	-	It is utilized to subtract one value from another value.
	Multiplication	*	It is utilized to multiply two values.
	Division	/	It is utilized to divide one value from another.
	Modulus	%	It is utilized to find the remainder value after the division.
Unary	Increment	++	It is utilized to increase the value by 1.
	Decrement	--	It is utilized to decrease the value by 1.

2. Relational Operators:

Operator Name	Operator Symbol	Description
Is Equal To operator	==	It checks both operands are equal or not. If the condition matches, it returns true; otherwise, false. This operator is used to check the first operand value is greater than the second operand value. If the condition matches, it returns true; otherwise, false.
Greater Than	>	This operator is used to check the first operand value is greater than or equal to the second operand value. If the condition matches, it returns true; otherwise, false.
Greater Than or Equal To	>=	This operator is used to check the first operand value is greater than or equal to the second operand value. If the condition matches, it returns true; otherwise, false.

Less Than <

Less Than or Equal To <=

Not Equal To !=

3. Logical Operators

Operator Name	Operator Symbol
---------------	-----------------

Logical AND	&&
--------------------	----

Logical OR	
-------------------	--

Logical NOT	!
--------------------	---

4. Bitwise Operators

Operator Name	Operator Symbol
---------------	-----------------

Binary AND	&
-------------------	---

Bitwise OR	
-------------------	--

Bitwise XOR	^
--------------------	---

Binary One's Complement	~
--------------------------------	---

Right Shift	>>
--------------------	----

Left Shift	<<
-------------------	----

5. Assignment Operators

Operator Name	Operator Symbol
---------------	-----------------

Assignment	=
-------------------	---

Add and Assignment	+=
---------------------------	----

Subtract and Assignment	-=
--------------------------------	----

Multiply and Assignment	*=
--------------------------------	----

Divide and Assignment	/=
------------------------------	----

the condition matches, it returns true; otherwise, false.

This operator is used to check the first operand value is lesser than the second operand value. If the condition matches, it returns true; otherwise, false.

It checks that the first operand value is lesser than or equal to the second operand value. If the condition matches, it returns true; otherwise, false.

It checks both operand values are not equal. If the condition matches, it returns true; otherwise, false.

Description

If all operand's values are true or non-zero, it returns only the true value.

If either of the operand's values is true or non-zero, it returns true value.

If the operand value is false or zero, it returns a true value.

Description

If this operator is available in both operands, it copies a bit to the calculated result.

If this operator is available in any of the operands, it copies a bit to the calculated result.

If this operator is present in any of the operands but not both, it copies a bit to the calculated result.

This operator changes the binary values 1 to 0 and 0 to 1.

This operator shifts the value to the right by a number of bits specified by the right operands.

This operator shifts the value to the left by a number of bits specified by the right operands.

Description

It assigns the values from the right side operands to left side operand.

This operator first adds the right operand value to the left operand, and then the value is assign to the left operand.

This operator first subtracts the right operand value from the left operand, and then the value is assign to the left operand.

This operator first multiply the right operand with the left operand, and then the value is assign to the left operand.

This operator divides the left operand with the right operand, and then the value is assigns to the left operand.

Modulus and Assignment

`%=`

It takes remainders using two operands, and then the value is assign to the left operand.

6. Ternary or Conditional Operator

It has the following syntax:
(Condition a) ? Expression1 : Expression2;

7. Misc Operators

- Sizeof() Operators:
- Comma Operator (,):
- Arrow and dot Operator:
- Casting Operator:
- Addressof Operator (&) and Pointer Operator (*):

Operator Precedency and Associativity

Operator precedence and associativity are essential concepts in C++ programming. It defines the order, and part of the expression is calculated. Precedency shows that portion of the expression should be evaluated first. On the other hand, associativity shows which direction to calculate when precedence operators are in expression. Here, we will discuss both operators:

S.No	Category	Operator	Associativity
1.	Scope Resolution	::	Left to Right
2.	Postfix	() [] -> . ++ --	Left to right
3.	Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
4.	Multiplicative	* / %	Left to right
5.	Additive	+ -	Right to left
6.	Shift	<< >>	Left to right
7.	Relational	< <= > >=	Left to right
8.	Equality	== !=	Right to left
9.	Bitwise AND	&	Left to right
10.	Bitwise XOR	^	Left to right
11.	Bitwise OR		Right to left
12.	Logical AND	&&	Left to right
13.	Logical OR		Left to right
14.	Conditional	?:	Right to left
15.	Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
16.	Comma	,	Left to right

Differences between Identifiers and Keywords

Identifiers	Keywords
Identifiers are the names defined by the programmer to the basic elements of a program.	Keywords are the reserved words whose meaning is known by the compiler.
It is used to identify the name of the variable.	It is used to specify the type of entity.
It can consist of letters, digits, and underscore.	It contains only letters.
It can use both lowercase and uppercase letters.	It uses only lowercase letters.
No special character can be used except the underscore.	It cannot contain any special character.
The starting letter of identifiers can be lowercase, uppercase or underscore.	It can be started only with the lowercase letter.

It can be classified as internal and external identifiers.

Examples are test, result, sum, power, etc.

It cannot be further classified.

Examples are 'for', 'if', 'else', 'break', etc.

Difference between call by value and call by reference in C++

Features	Call by Value	Call by Reference
Value Passed	A copy of the value is passed to the function.	An address of value is passed to the function.
Changes	Changes made within the function are not reflected in other functions.	Changes made with the function are reflected outside the function also.
Memory Location	Actual and formal arguments will be created in different memory locations.	Actual and formal arguments will be created in the same memory location.
Data Types	It is suitable for basic and simple data types, including int, float, char, etc.	It is suitable for complex data types, such as arrays, structs, strings, etc.
Performance	It decreases the risk of unintentional side effects. It is faster than the call by reference because of the smaller data transfer.	It increases the risk of unintentional side effects. It can be slower because of the requirement to access data via references.

Difference between Inline and Macro

Features	Inline Functions	Macros
Definition	These are specified using the inline keyword.	These are specified using the #define pre-processor directives.
Expansion	These are expanded by the compiler.	These are expanded by the pre-processor.
Scope	These functions have scope and type checking such as regular functions.	These functions have no scope or type checking.
Debugging	These are easy to debug because they are executed during the compile time.	These are complex to debug because these are not executed during compile time.
Private Members	Inline functions can access private members of a class.	These can never access the private members of a class.
Evaluation	All the parameters of inline functions are evaluated for single time.	When the macro is used in the code, parameters are evaluated multiple times.
Error Checking	In inline functions, errors are checked by the compiler.	In Macros, errors are not detected properly.
Usage	It is suitable for small functions.	It is mainly used for constants.
Recursion	These functions can call themselves recursively.	These cannot be recursive.

OOPs (Object-Oriented Programming) Concepts in C++

The major purpose of C++ programming is to introduce the concept of object orientation to the C programming language. Object Oriented Programming is a paradigm that provides many concepts, such as [inheritance](#), data binding, [polymorphism](#), etc.

The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language. Smalltalk is considered the first truly object-oriented programming language.

OOPs (Object Oriented Programming System)

Object means a real world entity, such as a pen, chair, table, etc. Object-Oriented Programming is a methodology or paradigm to design a program using [classes](#) and objects.

It simplifies the software development and maintenance by providing some concepts:

- Class
- Object
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

1. Class

A **class** is a blueprint for creating objects. It defines **attributes** (data) and **methods** (functions) that describe what an object can do.

The collection of objects is called class. It is a logical entity. A Class in C++ is the foundational element that leads to Object-Oriented programming. A class instance must be created to access and use the user-defined data type's data members and member functions. An object's class acts as its blueprint.

Class Example:

In C++, a class is defined using the keyword class as shown in the following example.

```
#include <iostream>
using namespace std;
class Car {          // Class definition
public:              // Access specifier
    string brand;    // Attribute (data member)
    string color;
    int year;
    void displayInfo() { // Method (member function)
        cout << "Brand: " << brand << ", Color: " << color << ", Year: " << year << endl;
    }
};
int main()
{
    Car car1;        // Creating an object of class Car
    car1.brand = "Toyota";
    car1.color = "Red";
    car1.year = 2020;
    car1.displayInfo(); // Calling a method
    return 0;
}
```

Output : ./class1
Brand: Toyota, Color: Red, Year: 2020

Access Specifiers

Access specifiers control access to class members:

Specifier	Access Level
public	Accessible from anywhere
private	Accessible only inside the class
protected	Accessible in the class and derived classes

Differences between Classes and Objects in C++

Features	Classes	Objects
Definition	It is a blueprint for creating	It is an instance of a class with

	objects.	actual values.
Entity	In classes, conceptual entities describe the structure and behavior.	It is a real-world entity that is created from the class.
Syntax	class Class_Name;	class_Name obj;
Uses	It is mainly used for concepts and models.	It is mainly used for real-world entities, such as data and functionality.
Representation	It represents a general concept or type.	It represents a specific instance of a class.
Memory Allocation	In C++ classes, no memory is allocated until an object is created.	The object is created if an object is created in the program.

Scope Resolution Operator in C++

Scope resolution operator and its various uses in the C++ programming language. The scope resolution operator is used to reference the global variable or member function that is out of scope. Therefore, we use the scope resolution operator to access the hidden variable or function of a program. The operator is represented as the double colon (::) symbol.

Uses of the scope resolution Operator

1. It is used to access the hidden variables or member functions of a program.
2. It defines the member function outside of the class using the scope resolution.
3. It is used to access the static variable and static function of a class.
4. The scope resolution operator is used to override function in the Inheritance.

```
#include <iostream>
using namespace std;
class staticVariable {
    static int n1;
public:
    static int n2;
    void func(int n1)
    {
        cout << "Value of static integer n1:- " << staticVariable::n1 << endl;
        cout << "Value of local interger n1:- " << n1 << endl;
    }
};
int staticVariable::n1 = 10;
int staticVariable::n2 = 20;
int main(int argc, char** argv)
{
    staticVariable st;
    int n1 = atoi(argv[1]);
    st.func(n1);
    cout << "Value of base::n2:- " << staticVariable::n2 << endl;
    return 0;
}
```

part 1:

```
class staticVariable {
    static int n1;
public:
    static int n2;
    void func(int n1)
    {
        cout << "Value of static integer n1:- " << staticVariable::n1 << endl;
        cout << "Value of local interger n1:- " << n1 << endl;
    }
}
```

```

    }
};
static int n1;
    • This is a static data member
    • It belongs to the class itself, not to any object
    • There is only one copy of it for the entire program
    • It is private by default (because it's before public:)
static int n2;
    • Also a static data member
    • This one is public
void func(int n1)
    • This function takes a parameter named n1
    • This parameter hides the class variable n1

```

That's why this line is important:

```
staticVariable::n1
```

It explicitly means:

“Use the n1 that belongs to the class, not the function parameter.”

Access Specifiers	Same Class	Derived Class	Outside Class
Public Access Specifier	Yes	Yes	Yes
Private Access Specifier	Yes	No	No
Protected Access Specifier	Yes	Yes	No

Defining Member Function

• Member Function Inside the Class

In C++, we can define member functions inside a class. It can allow us to access data members and other member functions. It can be considered as an inline function. Member functions are belonging to a specific class or object and are used to operate on that data members of the class, allowing interaction with and manipulation of the object's internal state or with other objects of the same class.

If we want to define a member function inside a class, we can include the function's signature and implementation within the class definition.

Example:

```

#include <iostream>
using namespace std;
class Employee {
    private:
        int Em_id;
        string Em_name;
        double Em_salary;
    public:
        Employee(int id, string name, double salary)
        {
            Em_id = id;
            Em_name = name;
            Em_salary = salary;
        }
        void show()
        {
            cout << "Employee name:- " << Em_name << " its id:- " <<
Em_id << " & salary:- " << Em_salary << endl;
        }
};
int main()
{
    Employee em(1, "Samar", 38243);

```

```

    em.show();
    Employee em1(2, "Kailash", 38243);
    em1.show();
    return 0;
}

```

Output: ./member_function_inside_class

Employee name:- Samar its id:- 1 & salary:- 38243

Employee name:- Kailash its id:- 2 & salary:- 38243

- **Member Function Outside the Class**

In C++, we can define the member function outside the class using the scope resolution operator (::) that is placed before the function name. The function name needs to include the class name followed by the scope resolution operator. These member functions have access to all members of the class, such as public, private, and protected members.

Example:

```
#include <iostream>
```

```
using namespace std;
```

```
class Employee {
```

```
    private:
```

```
        int Em_id;
```

```
        string Em_name;
```

```
        double Em_salary;
```

```
    public:
```

```
        Employee(int id, string name, double salary);
```

```
        void show();
```

```
};
```

```
Employee::Employee(int id, string name, double salary)
```

```
{
```

```
    Em_id = id;
```

```
    Em_name = name;
```

```
    Em_salary = salary;
```

```
}
```

```
void Employee::show()
```

```
{
```

```
    cout << "Employee name:- " << Em_name << " its id:- " << Em_id << " &
salary:- " << Em_salary << endl;
```

```
}
```

```
int main()
```

```
{
```

```
    Employee em(1, "Samar", 38243);
```

```
    em.show();
```

```
    Employee em1(2, "Kailash", 38243);
```

```
    em1.show();
```

```
    return 0;
```

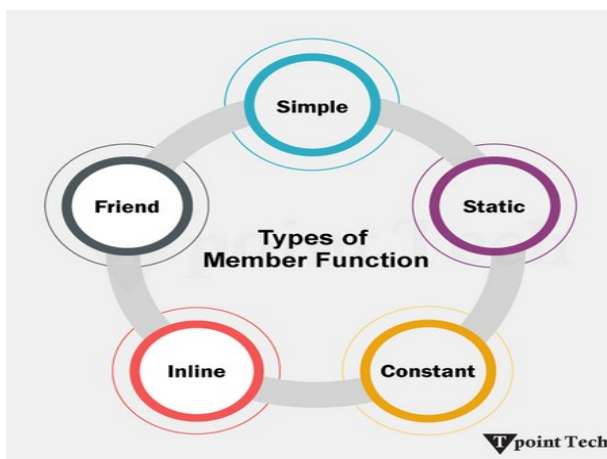
```
}
```

Output: ./member_function_outside_class

Employee name:- Samar its id:- 1 & salary:- 38243

Employee name:- Kailash
its id:- 2 & salary:- 38243

Types of Member Function



1) Simple Member Function

In C++, normal member functions are the commonly used type of member functions. We can define these functions within the class definition, and can have access to the data members and other class member functions.

2) Static Member Function

In C++, a function can be defined as static using a static keyword with the function name. If we declare a member function as static, only one copy of the function is created and utilized by all the class objects. These functions can be invoked without creating a class object.

```
#include <iostream>
using namespace std;
class Employee {
    public:
        static int total;
        Employee() {total++;}
};
int Employee::total = 0;
int main()
{
    Employee em1;
    cout << "Employee No.: " << em1.total << endl;
    Employee em2;
    cout << "Employee No.: " << em2.total << endl;
    Employee em3;
    cout << "Employee No.: " << em3.total << endl;
    return 0;
}
```

Output: ./static_member_function

Employee No.: 1

Employee No.: 2

Employee No.: 3

3) Constant Member Function

In C++, a constant member function is a member function that will not change any data members of the class. Using the [const keyword](#), we can declare the member function after the function declaration. This member function allows us to access the data members of an object but cannot modifying them.

```
#include <iostream>
using namespace std;
class Employee {
    private:
        int id;
        string name;
    public:
        Employee(int i, string n) : id(i), name(n) {}
        void show() const
        //void show()
        {
            //name = "Kailash"; //OR
            //name.operator=("Kailash");
            cout << "Employee name:- " << name << " & id is:- " << id << endl;
        }
};
int main()
{
    Employee em(1, "Samar");
    em.show();
    return 0;
}
```

```
}
```

Output: ./constant_member_function

Employee name:- Kailash & id is:- 1

4) Inline Member Function

In C++, an [inline function](#) is a function that is defined using the inline keyword. In the inline function, a compiler replaces the function call with the original code of the function during compile time. It can help to reduce the function call overhead and improve the performance, particularly for small and frequently used functions.

5) Friend Member Function

In C++, [friend functions](#) are defined outside the class definition and declared inside the class using the friend keyword. Friend functions are non-member functions that have access to the private and protected members of the class. If we want to declare a function as a friend function, a class can allow it special access privileges, which enables it to work on the class's internal data.

```
#include <iostream>
using namespace std;
class B;
class A {
    private:
        int x;
    public:
        A(int a){x = a;}
        friend void show(A, B);
};
class B {
    private:
        int y;
    public:
        B(int b){y = b;}
        friend void show(A, B);
};
void show(A obj1, B obj2)
{
    cout << "A: " << obj1.x << endl;
    cout << "B: " << obj2.y << endl;
}
int main()
{
    A obj1(5);
    B obj2(10);
    show(obj1, obj2);
    return 0;
}
```

Output: ./friend_member_function

A: 5

B: 10

Pure Virtual Function in C++

A pure virtual function in C++ is a virtual function declared in a base class that has no implementation and must be overridden by derived classes.

It is used to create abstract classes.

A class with at least one pure virtual function is called an abstract class.

You cannot create objects of an abstract class.

Derived classes must override all pure virtual functions, otherwise they also become abstract.

Used to achieve runtime polymorphism.

Why Use Pure Virtual Functions?

To define a common interface

To enforce derived classes to implement specific behavior

To support polymorphism

```
#include <iostream>
```

```
using namespace std;
```

```
class Shape {
```

```
public:
```

```
    virtual double getArea() = 0;
```

```
    virtual ~Shape() { cout << "Shape Distructor Called.\n";}
```

```
};
```

```
class Circle:public Shape {
```

```
private:
```

```
    double radius;
```

```
public:
```

```
    Circle(double r):radius(r){cout << "Circle Constructor Called.\n";}
```

```
    double getArea() override
```

```
    {
```

```
        return (22*radius*radius)/7;
```

```
    }
```

```
};
```

```
class Rectangle:public Shape {
```

```
private:
```

```
    double len;
```

```
    double bre;
```

```
public:
```

```
    Rectangle(double l, double b):len(l),bre(b){cout << "Rectangle Constructor Called.\n";}
```

```
    double getArea() override
```

```
    {
```

```
        return (len*bre);
```

```
    }
```

```
};
```

```
class Triangle:public Shape {
```

```
private:
```

```
    double base;
```

```
    double high;
```

```
public:
```

```
    Triangle(double ba, double hi): base(ba), high(hi) {cout << "Triangle Constructor Called.\n";}
```

```
    double getArea() override
```

```
    {
```

```
        return (base*high)/2;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Shape* s[3];
```

```
    s[0] = new Circle(7);
```

```
    s[1] = new Rectangle(7, 9);
```

```
    s[2] = new Triangle(9, 15);
```

```
    cout << "The area of the Circle is: " << s[0]->getArea() << endl;
```

```
    cout << "The area of the Rectangle is: " << s[1]->getArea() << endl;
```

```
    cout << "The area of the Triangle is: " << s[2]->getArea() << endl;
```

```
    for (int q = 0; q < 3; ++q)
```

```
    {
```

```
        delete s[q];
```

```
    }
```



```

        return 0;
    }
Output: ./pure_virtual_funx
Circle Constructor Called.
Rectangle Constructor Called.
Triangle Constructor Called.
The area of the Circle is: 154
The area of the Rectangle is: 63
The area of the Triangle is: 67.5
Shape Distructor Called.
Shape Distructor Called.
Shape Distructor Called.

```

Characteristics of Pure Virtual Functions

Pure virtual functions have several characteristics in C++. Some main characteristics are as follows:

- **No implementation in the base class:** A pure virtual function in the base class is stated as = 0, which indicates that no implementation is applied. Its main purpose is to serve as a placeholder, which indicates that derived classes must be implemented.
- **Defines an abstract class:** If a class contain only one pure virtual function, it becomes abstract.
- **Ensures a common interface:** Pure virtual functions provide a standardized interface. They require all subclasses to implement particular functions, which helps to maintain consistency between types derived from the same base.
- **Runtime Polymorphism:** Pure virtual functions use dynamic (late) binding, which means the appropriate function is called based on the actual object type at runtime, particularly when accessed through base class pointers or references.
- **Acts as a Contract:** A pure virtual function is effectively a contract that needs any class that inherits from the base to implement the stated functions. It helps to prevent missing or inconsistent method definitions during development.
- **Checked at compile time:** The compiler applied the implementation rules. If a subclass fails to override a needed function, an error is thrown at build time, which helps to reduce the runtime surprises and increases dependability.

1) Removal is not permitted in derived classes

In C++, a pure virtual function that is declared in a base class cannot be removed or left undefined in a derived class. Every subclass must provide an actual implementation; otherwise, it is abstract.

```

#include <iostream>
class Parent
{
    public:
        virtual void show() = 0; // Pure virtual function
};
class Child : public Parent
{
    // Missing override leads to a compilation error
    // void show() {} // Required override
};
int main() //main function
{
    return 0;
}

```

2) Cannot instantiate abstract classes

Any class that includes at least one pure virtual function is considered abstract and cannot be created directly.

3) Overriding Is Mandatory in Derived Classes

In C++, the derived class must have overrides for all pure virtual functions. If omitted, the derived class becomes abstract and cannot be instantiated either.

4) Base Class Can Have Constructor

In C++, if a base class contains pure virtual functions, it can still have a constructor. This constructor is executed when an object of a derived class is created.

Advantages of Pure Virtual Functions

Several advantages of pure virtual functions in C++ are as follows:

- **Enforce Interface Consistency:** All derived classes must implement particular methods and maintain a consistent structure.
- **Enable Polymorphism:** It enables Polymorphism for dynamic binding, which allows objects to behave differently dependent on their type at runtime.
- **Avoid Object Creation:** Abstract classes with pure virtual functions cannot be instantiated, which prevents misuse.
- **Modularity:** It promotes modularity by separating the interface from implementation, which allows for simpler extension and maintenance.
- **Compile-Time Safety:** It improves compile-time safety by detecting unimplemented functions early on, which reduces runtime errors.
- **Support Design Patterns:** It creates the basis for patterns like the Factory Method and Template Method, which enables scalable designs.

Disadvantages of Pure Virtual Functions

Several disadvantages of pure virtual functions in C++ are as follows:

- **Adds Complexity:** Overriding class hierarchies can complicate code maintenance.
- **Rigid Interface:** Changing or removing pure virtual functions can cause issues with existing subclasses.
- **No Shared Defaults:** Base classes cannot provide default logic, which leads to code duplication.
- **Minor Runtime cost:** Dynamic dispatch has a minimal impact on performance-critical applications.
- **Tight Coupling:** Any changes in the base class necessitate updates to all subclasses, which increases compile-time dependence.
- **Limited Scope:** It is suitable for polymorphic designs but not essential for simpler or non-hierarchical structures.

Overriding Member Function in C++

In C++, when a Derived class inherits a Member function from its Base class, it can redefine the behavior of that function in the Derived class. This process of redefining a Base class Member Function in a Derived class is called "**Overriding**" and the redefined function is referred to as an "**Overridden Member Function**".

In other words, when a Derived class defines a Member Function with the same name and signature as a Member Function in its Base class, the Derived class's function will Override the Base class function.

How to Override a Member Function in C++?

Define a Base Class with a Virtual Function

Define a Derived Class that Overrides the Virtual Function

Create Objects of the Derived Class and Call the Virtual Function

```
#include <iostream>
using namespace std;
class Draw {
public:
    virtual void show()
    {
```

```

        cout << "This is the base class of Draw.\n";
    }
};
class Square:public Draw {
    public:
        void show() override
        {
            cout << "This is the square.\n";
        }
};
class Circle:public Draw {
    public:
        void show() override
        {
            cout << "This is the Circle.\n";
        }
};
int main()
{
    Draw* d;
    Square s;
    d = &s;
    d->show();
    Circle c;
    d = &c;
    d->show();
    return 0;
}

```

Output: ./overriding_member_function

This is the square.

This is the Circle.

Advantages of Overriding Member Function

Polymorphism:

One of the major advantages of overriding is that it enables Polymorphism. Polymorphism allows a derived class to have multiple behaviors depending on the context in which it is used.

Code Reusability:

Overriding allows the reuse of code from the base class while allowing the derived class to modify the behavior of the base class's member functions.

Modularity:

Overriding makes it easier to maintain code because changes made in the derived class do not affect the base class.

Rules for Overriding Member Functions

- The Overridden Function in the derived class must have the same name and arguments as the base class function.
- The Overridden Function must have the same return type as the base class function or a covariant return type.
- The access level of the Overridden Function in the derived class cannot be more restrictive than the access level of the base class function.
- The **Virtual** keyword must be used in the base class function declaration.
- The function in the base class must be declared with the same access level or higher than the derived class.

Friend Function in C++

In C++, a friend function is a non-member function that contains access to the private and protected members of a class. The function is declared as a friend within the class using the friend keyword. If we want to declare a function as a friend function, a class

can allow it special access privileges, which enables it to work on the class's internal data.

In [C++](#), a friend may be a function, member function, class or class template, and function template case. In that case, the whole class and all the members of the class are friends.

```
#include <iostream>
using namespace std;
class Box {
    private:
        int len;
    public:
        Box():len(5) {cout << "Box constructor called.\n";}
        friend int getlen(Box);
};
int getlen(Box b)
{
    b.len += 6;
    return b.len;
}
int main()
{
    Box b;
    cout << "Length of box:- " << getlen(b) << endl;
    return 0;
}
Output: ./friend_function
Box constructor called.
Length of box:- 11
```

Friend Class

You can make an entire class a friend.

```
#include <iostream>
using namespace std;
class B;
class A {
    private:
        int x = 10;
        friend class B;
};
class B {
    public:
        void display(A obj)
        {
            cout << "A obj:- " << obj.x << endl;
        }
};
int main()
{
    A a;
    B b;
    b.display(a);
    return 0;
}
Output: ./class_as_friend
A obj:- 10
```

Accessing Private and Protected Members

In C++, the private and protected members cannot be accessed outside the class. If we want to access these members, we can utilize the friend function. The friend function enables us to access the private and protected members of the class directly.

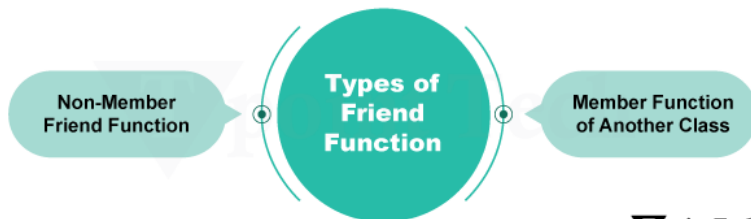
```
#include <iostream>
using namespace std;
class test {
    private:
        int x;
    protected:
        int y;
    public:
        test(int a, int b):x(a), y(b) {cout << "Test Constructor Called.\n";}
        friend int add(test obj);
};
int add(test obj)
{
    return obj.x+obj.y;
}
int main()
{
    test obj(12, 13);
    cout << "Sum value: " << add(obj) << endl;
    return 0;
}
```

Output: ./friend_get_private_protected

Test Constructor Called.

Sum value: 25

Types of Friend Functions



point Tech

Non-Member Friend Function

In C++, the non-member friend function is declared in the class, but these functions are not class members. It enables access to the private and protected members of the class. These non-member functions are commonly used to offer external functions that work on the private data members of the class.

Example: Same as **Accessing Private and Protected Members**

Member Function of another Class

In C++, member functions of another class can be declared as a friend function. It enables the friend function to access the private and protected members of another class.

It is very useful when two classes are required to work closely together, such as in [operator overloading](#) or utility functions that need direct access to internal data.

Example: Same as **Friend Class**

Features of the Friend function

Several features of the friend function in C++ are as follows:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object because it is not in the scope of that class.
- It can be called like a normal function without using the object.
- It can be declared either in the private or the public section of the class.

Advantages of Friend Function

- **Access to Private Members:** Friend functions conveniently make it possible to change the protected and private members of a class because they can access them, which provides more flexibility in design.

- **Operator Overloading:** They are also frequently utilized in operator overloading because of their ability to access the private data of the pertinent objects.
- **Separation of Concerns:** With the help of friend functions, the user interface can be separated from the actual implementation, which improves code organization.
- **Improved Performance:** Friend functions enable accessing the private parts of a class that reduces the use of getter/setter functions, which makes it more efficient.
- **Inter-Class Communication:** They allow different classes to communicate without needing a direct relationship, which is beneficial in complex designs.

Disadvantages of Friend Function

- **Encapsulation Break:** Friend functions can access private members, thus violating the encapsulation principle, which makes code maintenance a complex task.
- **Tight Coupling:** Friend functions make classes dependent on each other, which makes design modularity difficult.
- **Security Risks:** In C++, using the friend function and friend class to allow access to private data can discover the class to potential risk.

C++ this Pointer

In C++ programming, **this** is a keyword that refers to the current instance of the class. A class object can access itself using this pointer inside its member functions. Understanding this pointer serves as a key prerequisite when we need to operate within object-oriented structures for functions, such as self-reference calls, method chaining, and operator overloading.

In other words, this [pointer](#) operates as an implicit built-in pointer that operates inside all member functions of a class except static members. Through this pointer, the calling object obtains its memory address while enabling member accessibility.

C++ Simple this Pointer Example

```
#include <iostream>
using namespace std;
class test {
    public:
        void show()
        {
            cout << "This pointer holds:- " << this << endl;
        }
};
int main()
{
    test t;
    t.show();
    return 0;
}
```

Output: ./this_pointer_example
This pointer holds:- 0x7ffd705fa957

Another Example of this pointer

```
#include <iostream>
using namespace std;
class check {
    public:
        int id;
        string name;
        double salary;
        check(int id, string name, double salary)
        {
```

```

        this->id = id;
        this->name = name;
        this->salary = salary;
    }
    void show()
    {
        cout << "Employee name:- " << name << " & id - " << id << " salary - " <<
salary << endl;
    }
};
int main()
{
    check c(101, "Samar", 38243);
    check c1(102, "Kailash", 48243);
    c.show();
    c1.show();
    return 0;
}
Output: ./this_pointer_example1
Employee name:- Samar & id - 101 salary - 38243
Employee name:- Kailash & id - 102 salary - 48243

```

Characteristics of this Pointer

- This [Pointer](#) exists exclusively in member functions because static functions remain part of the class structure instead of an object structure.
- The system automatically creates this pointer during a member function execution.
- This pointer contains the address of the calling object yet remains accessible for modification.
- [Static](#) functions do not allow this keyword usage since static members operate on non-specific instances.
- Method chaining and operator overloading, along with naming conflict resolution, all depend on using this pointer.

Uses of this pointer in C++

- It can be used to pass a current object as a parameter to another method.
- It can be used to refer current class instance variable.
- It can be used to declare indexers.
- Function parameters and member variables can receive resolution through this pointer.
- It provides an approach to writing cleaner code that operates more efficiently with method chaining.
- The use of this enables operator overloading when returning an object reference from the current instance.

Different Ways to Use this Pointer

1. Accessing Data Members

In C++, this pointer is used to resolve naming conflicts when a function contains parameters that share their name with object member variables.

```

#include <iostream>
using namespace std;
class getX {
private:
    int x;
public:
    void setX(int x)
    {
        this->x = x;
    }
}

```

```

        void show()
        {
            cout << "Value of x:- " << x << endl;
        }
    };
    int main()
    {
        int n;
        cout << "Enter X value to set: ";
        cin >> n;
        getX g;
        g.setX(n);
        g.show();
        return 0;
    }

```

Output: ./this_pointer_data_access

Enter X value to set: 11

Value of x:- 11

2. Returning the Object (Method Chaining)

In C++, the method chaining technique becomes possible because returning `*this` (this pointer) enables multiple function calls in one statement.

```

#include <iostream>
using namespace std;
class update {
private:
    string str;
public:
    update& setVal(string str)
    {
        this->str = str;
        return *this;
    }
    void show()
    {
        cout << "str data:- " << str << endl;
    }
};
int main()
{
    string s;
    cout << "Enter sentence: ";
    getline(cin, s);
    update u;
    u.setVal(s).show();
    return 0;
}

```

Output: ./this_pointer_chaining

Enter sentence: I am Sanatani

str data:- I am Sanatani

3. Passing the Current Object

In C++, this pointer solves the requirement to transmit the current object instance to any function or method that accepts objects.

```

#include <iostream>
using namespace std;
class setup {
public:
    void show()
    {

```



```

        cout << "This is the show func().\n";
    }
    void set(setup* obj)
    {
        obj->show();
    }
    void execute()
    {
        set(this);
    }
};
int main()
{
    setup s;
    s.execute();
    return 0;
}

```

Output: ./this_pointer_passing_current_obj

This is the show func().

4. Using "this" pointer in Operator Overloading

In [operator](#) overloading the "this" pointer provides a way to return a reference to the modified object.

```

#include <iostream>
using namespace std;
class get {
private:
    int num;
public:
    get(int n):num(n) {cout <<"get constructor called.\n";}
    get& add(int n)
    {
        this->num = this->num + n;
        return *this;
    }
    void display()
    {
        cout << "Updated Value: " << num << endl;
    }
};
int main()
{
    int a, n;
    cout << "Enter the 2 value: " ;
    cin >> a >> n;
    get g(a);
    g.add(n);
    g.display();
    return 0;
}

```

Output: ./this_pointer_operator_overloading

Enter the 2 value: 2 5

get constructor called.

Updated Value: 7

Using this Pointer in a Constructor in C++

Inside a class constructor, this pointer serves as a reference to the active object instance. In C++, this pointer provides multiple benefits, which include object initialization alongside name resolution services for method chaining functionality.

1. Resolving Name Conflict in Constructor

When constructor parameters share identical names with the class members, this pointer functions as a distinction method.

- The [Constructor](#) returns this pointer to enable Method Chaining
- A method chain becomes possible through this pointer because it returns the current instance of the object.

```
#include <iostream>
using namespace std;
class test {
    private:
        int x;
    public:
        test(int x)
        {
            this->x = x;
            cout << "Constructor Called. Value Initialized: " << this->x << endl;
        }
        test* setNum (int val)
        {
            this->x = val;
            return this;
        }
        void display()
        {
            cout << "Value of x: " << x << endl;
        }
};
int main()
{
    test t(11);
    t.setNum(16)->display();
    return 0;
}
```

Output: ./this_pointer_resolving_name_conflict_in_constructor
Constructor Called. Value Initialized: 11
Value of x: 16

2. Removing "this" pointer in C ++

In C++, "this" pointer is an implicit address that exists for each class member function during their execution time to access the current object. Deleting the this pointer is generally avoided unless it is too necessary.

Member function can delete this pointer only when the following two conditions are found:

- A dynamic allocation of this object occurs through new.
- Deletion of the object prevents any new system access from starting.

```
#include <iostream>
using namespace std;
class remove1 {
    public:
        void destroy ()
        {
            cout << "Destroying the current object ..." << endl;
            delete this;
        }
};
int main()
{
    remove1* rm1 = new remove1();
```

```

    rm1->destroy();
    return 0;
}
Output: ./this_pointer_remove_this
Destroying the current object ...

```

Effects of this Pointer in C++

- Accessing Current Object: Programming access to current object members becomes possible through the use of this pointer.
- Method Chaining: Method Chaining is possible through this pointer because it provides a reference to the calling object.
- Avoiding Name Conflicts: The use of this resolves conflicts that occur between names used for local variables that match with data member names.
- Operator Overloading: Operator overloading functionality depends on this pointer, which returns the object reference from the calling location.

Limitations of this Pointer

- Can not be modified: A programmer cannot change the fundamental properties of this pointer as it remains default and irreversible.
- Not available in static functions: This pointer is inaccessible in static functions because they do not belong to any object example.
- Limited to class scope: Only non-static class members can appoint this indicator as an operation.

C++ static

In C++, static is a keyword or modifier that belongs to the type not instance. So instance is not required to access the static members. In C++, static can be field, method, constructor, class, properties, operator and event.

[C++](#) uses the static keyword for various functions that include extending variable lifetime duration and reducing scope accessibility while optimizing memory management. The static [keyword](#) enables access control for variables and functions as well as class members while managing their behavioural parameters.

C++ Static Field

A field that is declared as static is called a static field. Unlike the instance field, which gets memory each time whenever we create an object, there is only one copy of the static field produced in the memory. It is shared with all the objects.

It is used to refer to the common property of all objects such as rateOfInterest in case of Account, companyName, Employee, etc.

C++ Static Field Example

```

#include <iostream>
using namespace std;
class Bank {
public:
    int acc_no;
    string name;
    static float interestRate;
    Bank(int acc_no, string name)
    {
        this->acc_no = acc_no;
        this->name = name;
    }
    void show()
    {
        cout << "Account Holder Name:- " << name << " & its A/C:- " << acc_no
        << " & Rate of Interest(ROI): " << interestRate << endl;
    }
};
float Bank::interestRate = 7.25;
int main()

```

```

{
    Bank b1(655656565, "Samar");
    Bank b2(655656566, "Kailash");
    Bank b3(655656567, "Kailu");
    b1.show();
    b2.show();
    b3.show();
    return 0;
}

```

Output: ./static_static_field

Account Holder Name:- Samar & its A/C:- 655656565 & Rate of Interest(ROI): 7.25

Account Holder Name:- Kailash & its A/C:- 655656566 & Rate of Interest(ROI): 7.25

Account Holder Name:- Kailu & its A/C:- 655656567 & Rate of Interest(ROI): 7.25

C++ Counting Objects Examples Using Static Keyword

```

#include <iostream>
using namespace std;
class Testing {
public:
    int acc_no;
    string name;
    static int cnt;
    Testing(int acc_no, string name)
    {
        this->acc_no = acc_no;
        this->name = name;
        cnt++;
    }
    void display()
    {
        cout << "A/C Holder Name:- " << name << " & its A/C No.:- " << acc_no
<< endl;
    }
};
int Testing::cnt = 0;
int main()
{
    Testing t1(10101, "Ram");
    Testing t2(10102, "SitaRam");
    Testing t3(10103, "Shiv");
    Testing t4(10104, "ShaktiShiv");
    Testing t5(10105, "Hanuman Ji");
    t1.display();
    t2.display();
    t3.display();
    t4.display();
    t5.display();
    cout << "Total Objects are: " << Testing::cnt << endl;
    return 0;
}

```

Output: ./static_static_counting_objects

A/C Holder Name:- Ram & its A/C No.:- 10101

A/C Holder Name:- SitaRam & its A/C No.:- 10102

A/C Holder Name:- Shiv & its A/C No.:- 10103

A/C Holder Name:- ShaktiShiv & its A/C No.:- 10104

A/C Holder Name:- Hanuman Ji & its A/C No.:- 10105

1. Static Variables in Functions

In C++, static variable contains determined values throughout separate function executions while avoiding continuous initializations. Static variables in function are initialized only once, and then these variables hold values even through [function](#) calls.

C++ Static Variables in Functions Example

```
#include <iostream>
using namespace std;
void counter()
{
    static int cnt = 0;
    cnt++;
    cout << "Counter are: " << cnt << endl;
}
int main()
{
    counter();
    counter();
    counter();
    return 0;
}
```

Output: ./static_static_variable_in_function
Counter are: 1
Counter are: 2
Counter are: 3

2. Static Data Members in Classes

In C++, the static data members are class members that are defined using the static keyword. It is shared among all objects of the class and belongs to the class, not separate objects. There may not be multiple copies of the same static variable for several objects.

C++ Static Data Members in Classes Example

```
#include <iostream>
using namespace std;
class Test {
public:
    static int cnt;
    Test() { cnt++; }
};
int Test::cnt = 0;
int main()
{
    Test t1, t2, t3, t4, t5;
    cout << "Total Count of Objects are: " << Test::cnt << endl;
    return 0;
}
```

Output: ./static_static_member_in_class
Total Count of Objects are: 5

3. Static Member Functions in a Class

In C++, static member functions operate independently from objects of any class, and they can only access static class elements.

C++ Static Member Function in a Class Example

```
#include <iostream>
using namespace std;
class Test {
private:
    static int cnt;
public:
    static void incremental ()
```

```

        {
            cnt++;
            cout << "cnt Value:- " << cnt << endl;
        }
    };
    int Test::cnt = 0;
    int main()
    {
        Test::incremental();
        Test::incremental();
        Test::incremental();
        return 0;
    }
    Output: ./static_static_member_function_in_class
    cnt Value:- 1
    cnt Value:- 2
    cnt Value:- 3

```

4. Global Static Variables

A static global variable retains an internal connection that confines its usage to the particular file where statements declare it. The internal linkage of static variables protects them from conflicts that occur between different files.

C++ Global Static Variables Example

```

#include <iostream>
using namespace std;
static int globalCnt = 0;
void counter ()
{
    globalCnt++;
    cout << "Global Counter Value:- " << globalCnt << endl;
}
int main()
{
    counter();
    counter();
    counter();
    return 0;
}
    Output: ./static_global_static_variable
    Global Counter Value:- 1
    Global Counter Value:- 2
    Global Counter Value:- 3

```

5. Static Variables in Namespaces

A static variable within a namespace works similarly to a static global variable while maintaining its usage scope to the current file.

C++ Static Variables in Namespace Example

```

#include <iostream>
using namespace std;
namespace spaceFunc
{
    static int cnt = 0;
    void show()
    {
        cnt++;
        cout << "Namespace Counter Value: " << cnt << endl;
    }
}
int main()
{

```

```

        spaceFunc::show();
        spaceFunc::show();
        spaceFunc::show();
        return 0;
    }
    Output: ./static_variable_in_namespace
    Namespace Counter Value: 1
    Namespace Counter Value: 2
    Namespace Counter Value: 3

```

6. Static Local Variables in Loops

In C++, a static variable inside a loop retains its value across iterations.

C++ Static Local Variables in Loops Example

```

#include <iostream>
using namespace std;
void CounterLoop ()
{
    static int cnt = 0;
    for(int i=0;i<2;i++)
    {
        cnt++;
        cout << "Value of the Counter: " << cnt << endl;
    }
}
int main()
{
    CounterLoop();
    CounterLoop();
    return 0;
}

```

```

Output: ./static_static_local_variable_in_loop
Value of the Counter: 1
Value of the Counter: 2
Value of the Counter: 3
Value of the Counter: 4

```

7. Static in Multithreading

In C++, static variables in multithreaded programs trigger data races because threads share their values.

C++ Static in Multithreading Example

```

#include <iostream>
#include <thread>
using namespace std;
void counter()
{
    static int cnt = 0;
    cnt++;
    cout << "Thread Counter Value: " << cnt << endl;
}
int main()
{
    thread t1(counter);
    thread t2(counter);
    t1.join();
    t2.join();
    return 0;
}

```

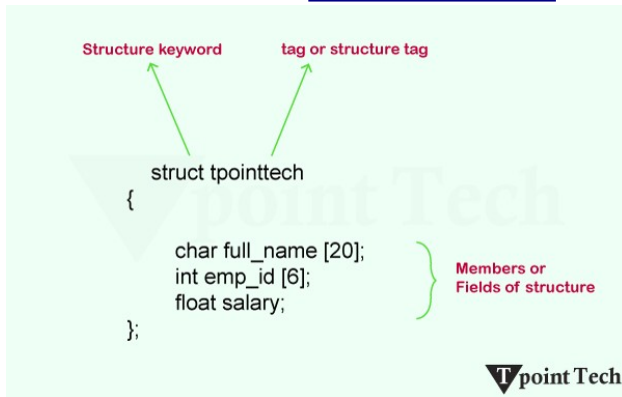
```

Output: ./static_static_in_multithreading
Thread Counter Value: 1
Thread Counter Value: 2

```

C++ Structs

In C++, structs are user-defined types that are used to create structured objects. They allow us to collect variables of different data types in a single name. Structs are used for lightweight objects, such as rectangles, colors, Points, etc. In [C++](#), structs can contain data members and [member functions](#).



Creating Structure Variable

Once the structure has been declared, a structure variable must be defined to allocate memory and store data. The [data type](#) for a structure variable is the structure name, which is followed by the variable name.

Syntax :

```
Struct Employee {
    string name;
    int age;
};
// Creating structure variable
Employee emp1;
```

Accessing Structure Members

Any members of a structure variable can be directly accessed by using the dot (.) operator to access structure members. The dot operator separates the name of the structure variable and the member name. The values stored in a structure can be retrieved or modified using this method.

C++ Struct Example

```
#include <iostream>
using namespace std;
struct SamarData {
    string name;
    int age;
    double weight;
    double height;
    string profession;
    double salary;
};
int main()
{
    SamarData sd;
    sd.name = "Samaranjan Pankajkumar Manjhi";
    sd.age = 25;
    sd.weight = 56;
    sd.height = 1.75;
    sd.profession = "Software Developer";
    sd.salary = 38243;
    cout << "Name: " << sd.name << endl;
    cout << " age      : " << sd.age << " years" << endl;
    cout << " weight   : " << sd.weight << " kgs" << endl;
    cout << " height   : " << sd.height << " mtr" << endl;
    cout << " profession : " << sd.profession << endl;
    cout << " salary    : " << "Rs. " << sd.salary << endl;
```



```

        return 0;
    }
Output: ./struct_example
Name: Samaranjan Pankajkumar Manjhi
age      : 25 years
weight   : 56 kgs
height   : 1.75 mtr
profession : Software Developer
salary   : Rs. 38243

```

Member Functions in C++ Structures

In C++, a structure can also have functions that enable direct operations on the data elements of the structure. These member functions are similar to regular functions, but these functions are defined in the scope of a structure.

Structures become more adaptable by combining data and behavior into a single unit, much like classes. C++ structure enables processes with several class components, including [access specifier](#), [constructor](#), [destructor](#), and many more.

C++ Member function in Structure Example

```

#include <iostream>
using namespace std;
struct Multi {
    int a, b;
    int product ()
    {
        return a*b;
    }
};
int main()
{
    int x, y;
    cout << "Enter 2 interger Value: ";
    cin >> x >> y;
    Multi m;
    m.a = x;
    m.b = y;
    int res = m.product();
    cout << "Result of Product:- " << res << endl;
    return 0;
}

```

Output: ./struct_struct_member_function

Enter 2 interger Value: 3 7

Result of Product:- 21

Array of Structures in C++

Multiple structure variables can be sequentially stored together in an array of structures. The dot (.) operator and the array index are used to access individual members, where each element in the array represents an instance of the structure.

C++ Array of Structure Example

```

#include <iostream>
using namespace std;
struct Cord {
    int x, y;
};
int main()
{
    Cord points[2] = {{1, 2}, {3, 4}};
    cout << points[0].x << " " << points[0].y << endl;
    cout << points[1].x << " " << points[1].y << endl;
    return 0;
}

```

```

}
Output: ./struct_struct_array
1 2
3 4

```

Structure Pointer in C++

A pointer that stores a structure variable's memory address is called a structure pointer. The arrow (->) operator is used to access structure members when working with structure pointers rather than the dot (.) operator.

C++ Structure Pointer Example

```

#include <iostream>
using namespace std;
struct Rectangle {
    int len, wid;
};
int main()
{
    int n, m;
    cout << "Enter 2 numbers: ";
    cin >> n >> m;
    Rectangle r1 = {n, m};
    Rectangle *ptr = &r1;
    cout << "Length of Rectangle: " << ptr->len << endl;
    cout << "Width of Rectangle : " << ptr->wid << endl;
    return 0;
}

```

```

Output: ./struct_struct_pointer
Enter 2 numbers: 1 2
Length of Rectangle: 1
Width of Rectangle : 2

```

typedef with Structures in C++

In C++, the typedef keyword enables us to make an alias for an existing variable. It makes an alias for the actual name of the structure. When it is used with the structure, it makes the code more readable and concise by eliminating the requirement to use the struct keyword every time we declare a structure variable.

C++ typedef with Structures Example

```

#include <iostream>
using namespace std;
typedef struct {
    int st_id;
    string st_name;
    char st_grade;
}Student;
int main()
{
    Student st1 = {101, "Samar", 'A'};
    Student st2 = {102, "Kailash", 'B'};
    cout << st1.st_id << " " << st1.st_name << " " << st1.st_grade << endl;
    cout << st2.st_id << " " << st2.st_name << " " << st2.st_grade << endl;
    return 0;
}

```

```

Output: ./struct_typedef_with_struct
101 Samar A
102 Kailash B

```

Difference between Structure and Class in C++

Structure	Class
If the access specifier is not declared explicitly, by default access specifier will	If the access specifier is not declared explicitly, by default access specifier will

be public.

Syntax of Structure:

```
struct structure_name  
{ // structure body }
```

The instance of the structure is known as "Structure variable".

be private.

Syntax of Class:

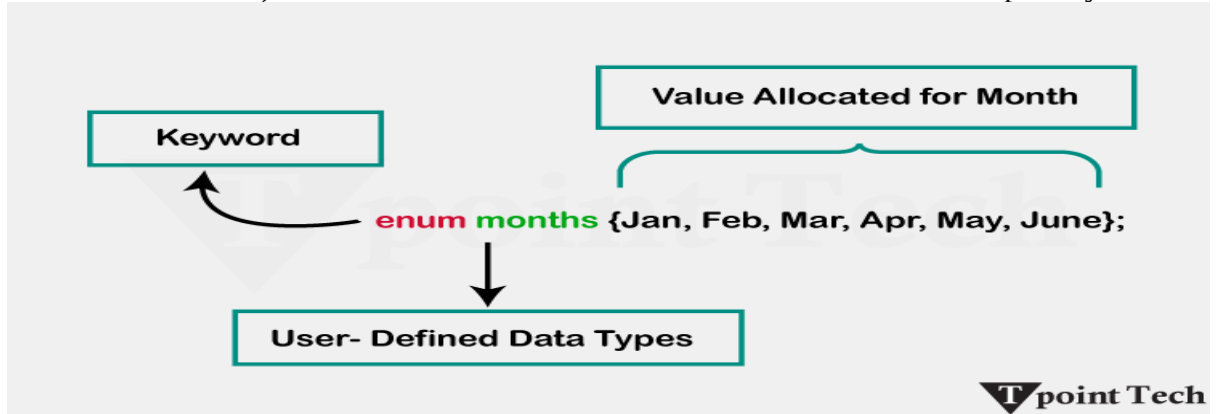
```
class class_name  
{ // class body }
```

The instance of the class is known as "Object of the class".

C++ Enumeration

In C++, Enumeration (Enum) is a user-defined data type that contains a fixed set of named integer constants. We can define the enumeration using the enum keyword. It helps to make the code more readable and maintainable.

It can be used for days (Sun, Mon, Tue, Wed, Thu, Fri, and Sat), months (Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec), directions (NORTH, SOUTH, EAST, and WEST), etc. The C++ enum constants are static and final implicitly.



Syntax

```
enum enum_name {  
    val1, val2, val3, ...  
};
```

In this syntax,

- **enum:** It is used to declare the enumeration type.
- **enum_name:** It represents the enumeration name.
- **val1, val2, val3...:** It represents the integral constants.

C++ Enumeration Example

```
#include <iostream>  
#include <string.h>  
using namespace std;  
enum month {jan, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec};  
int main()  
{  
    month mon_name;  
    mon_name = mar;  
    cout << "Month:- " << mon_name+1 << endl;  
    return 0;  
}
```

Output: ./enumeration_example
Month:- 3

Constructors in C++

In C++, the constructor is a special method that is invoked automatically at the time of object creation. It is used to initialize the data members of the new object generally. In [C++](#), the constructor has the same name as the [class](#) or structure.

Syntax for Constructor within the Class

```
class ClassName {  
    public:  
        ClassName(); //Constructor declaration
```

```
};
```

Syntax for constructor outside the class

```
ClassName::ClassName() {  
    // Constructor definition outside the class  
    // ClassName:: indicates the scope of the constructor  
}
```

Simple C++ Constructor Example

```
#include <iostream>  
using namespace std;  
class Car {  
    public:  
        string model;  
        string brand;  
        int speed;  
        int year;  
        Car(string m, string b, int s, int y): model(m), brand(b), speed(s), year(y) {}  
        void display()  
        {  
            cout << "Brand:- " << brand << " | model:- " << model << " | speed:- "  
"<<speed << " km/hr" << " | year:- " <<year << endl;  
        }  
};  
int main()  
{  
    Car c1("X1", "Audi", 80, 2001);  
    Car c2("X2", "Audi", 85, 2003);  
    c1.display();  
    c2.display();  
    return 0;  
}
```

Output:

```
./constructor_simple_example
```

```
Brand:- Audi | model:- X1 | speed:- 80 km/hr | year:- 2001
```

```
Brand:- Audi | model:- X2 | speed:- 85 km/hr | year:- 2003
```

Types of Constructors in C++

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor
4. Move Constructor

C++ Default Constructor

A constructor which does not argument is known as a default constructor. It is invoked at the time of creating an object. It allows us to initialize the class member variables to their default values. It is also known as a zero-argument constructor.

Syntax

```
class Car {  
    public:  
        Car();  
};
```

C++ Default Constructor Example

```
#include <iostream>  
using namespace std;  
class Employee {  
    public:  
        Employee()  
        {  
            cout << "This is the default Constructor Example.\n";  
        }  
}
```

```
};
int main()
{
    Employee e1;
    Employee e2;
    return 0;
}
```

Output: ./constructor_example_default

This is the default Constructor Example.

This is the default Constructor Example.

C++ Parameterized Constructor

In C++, a constructor that has parameters is called a parameterized constructor. It is used to provide different values to different objects. It is useful to initialize the member data. When we want to define the body of the constructor, we can use the parameters to initialize the objects.

Syntax

```
class Class_name {
public:
    Class_name(type1 param1, type2 param2, ...); //parameterized constructor
};
```

C++ Parameterized Constructor Example

```
#include <iostream>
using namespace std;
class Car {
public:
    string model;
    string brand;
    int speed;
    int year;
    Car(string m, string b, int s, int y): model(m), brand(b), speed(s), year(y) {}
    void display()
    {
        cout << "Brand:- " << brand << " | model:- " << model << " | speed:- "
        << speed << " km/hr" << " | year:- " << year << endl;
    }
};
int main()
{
    Car c1("X1", "Audi", 80, 2001);
    Car c2("X2", "Audi", 85, 2003);
    c1.display();
    c2.display();
    return 0;
}
```

Output:

./constructor_simple_example

Brand:- Audi | model:- X1 | speed:- 80 km/hr | year:- 2001

Brand:- Audi | model:- X2 | speed:- 85 km/hr | year:- 2003

Copy Constructor

A member function known as a copy constructor that helps to initialize an item using another object from the same class. The copy constructor requires a reference to an object belonging to the same class.

C++ Copy Constructor Example

```
#include <iostream>
using namespace std; //Using Standard Namespace
class Box {
private:
    int length;
```

```

public:
    // Parameterized constructor
    Box(int l) : length(l) {}
    // Copy constructor
    Box(const Box &b) {
        length = b.length;
    }
    // Function to display the length
    void display() {
        cout << "Length: " << length << endl;
    }
};

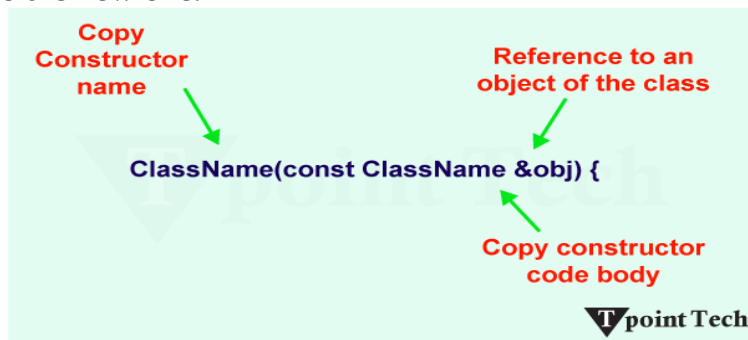
int main() { //Main Function
    // Creating an object using the parameterized constructor
    Box box1(10);
    // Creating a copy of box1 using the copy constructor
    Box box2 = box1;
    // Displaying the length of both boxes
    box1.display();
    box2.display();
    return 0;
}

```

Output: ./constructor_example_copy_sample
Length: 10
Length: 10

For extra information about Copy Constructor

In C++, a copy constructor is a type of constructor that creates a new object by copying the contents of an already existing object of the same class. It is called whenever an object is initialized with another object of the same class. This process, known as copy initialization, involves assigning the values of each data member of the original object to the new one.



In C++, a copy constructor performs member-wise copy, which means that it copies each data member of an object individually to create a new object. A copy constructor, like all other constructors in C++, has no return type, including void.

Syntax

```
Class_Name (const Class_Name &object);
```

- **Class_Name:** It represents the name of the class. It is the constructor of that class.
- **const Class_Name &object:** It represents a constant reference to an object of the same class, which is passed as a parameter.

Simple C++ Copy Constructor Example

```

#include <iostream>
using namespace std;
class Copy_construct
{
public:
    int d;

```

```

Copy_construct()
{
    d = 0;
}
Copy_construct(const Copy_construct &s)
{
    d = s.d;
}
};
int main() //Main
{
    Copy_construct obj_1;
    obj_1.d = 35;
    cout << "obj_1 data is: " << obj_1.d << endl;
    Copy_construct obj_2(obj_1);
    cout << "obj_2 data is: " << obj_2.d << endl;
    return 0;
}
Output: ./constructor_example_copy_sample_1
obj_1 data is: 35
obj_2 data is: 35

```

Characteristics of Copy Constructor in C++

- A copy constructor initializes a new object by copying data from an existing instance of the same class.
- It accepts a reference to an object of the same class as an argument.
- Copy initialization refers to the process of utilizing a copy constructor to initialize members of another object.
- The copy constructor does a member-wise copy, which means it transfers each member variable from the source object to the new object.
- If no copy constructor is explicitly defined, the compiler will build a default copy constructor.
- Programmers can create their own copy constructor to handle special copy behaviours like deep copying or resource management.
- Using a reference in the parameter avoids unnecessary copying and ensures better performance.

Types of Copy Constructor

1. Default Copy Constructor
2. User-Defined Copy Constructors

Default Copy Constructor

In C++, the compiler defines the default copy constructor. If the user defines no copy constructor, the compiler supplies its constructor. If a copy constructor is not explicitly stated, C++ will build a default copy constructor. This default constructor copies the values from the original object to the new object without the programmer providing any specific instructions.

C++ Default Copy Constructor Example

```

#include <iostream>
using namespace std; //using standard namespace
class Rect
{
    private:
        int width, height;
    public: //Access Modifier
        Rect(int w, int h)
        {
            width = w;
            height = h;
        }
}

```

```

        int getWidth()
        {
            return width;
        }
        int getHeight()
        {
            return height;
        }
    };
int main() //Main Function
{
    Rect r1(15, 58);
    Rect r2 = r1;
    // Default copy constructor is called
    cout << "Width is: " << r2.getWidth() << "\nHeight is: " << r2.getHeight() <<
endl;
    return 0;
}
Output: ./constructor_example_copy_default
Width is: 15
Height is: 58

```

Usage of Default Copy Constructor

- It is useful for classes that do not use pointers or dynamic memory and where a shallow copy of members is enough.
- In cases when no specific copy behavior is required, the compiler's default copy constructor reduces development effort and keeps code simple.

User-Defined Copy Constructors in C++

The programmer defines the user-defined constructor. When a programmer creates a copy constructor to customize object copying, it is known as a user-defined or explicit copy constructor.

A user-defined copy constructor allows the programmer to determine how data is copied from one object to another. It is useful when the default copy behaviour is insufficient, such as when using dynamic memory or custom logic.

The programmer ensures that the new object precisely copies the original by carefully defining the copy process, which gives them complete control over how members are initialized.

C++ User-Defined Copy Constructor Example

```

#include <iostream>
using namespace std; //using standard namespace
class Product
{
    public: //Access Modifier
        int price;
        Product(int pro)
        {
            price = pro;
        }
        Product(const Product &item)
        {
            price = item.price;
        }
};
int main() //Main Function
{
    Product p1(500);
    Product p2(p1);
    cout << "Price of copied product: " << p2.price << endl; //Displays the output
}

```



```

        return 0;
    }
    Ouput: ./constructor_example_copy_user-define
    Price of copied product: 500

```

Usage of User-Defined Copy Constructor

1. It is mainly used to monitor or log when an object is copied.
2. When shallow copying is appropriate, but additional behaviour (such as counting and validation) is required throughout the copy operation.

Behavior of Copy Constructor

In C++, a copy constructor is a special constructor that is used to create a new object as a copy of an existing object. There are two main types of copying behavior associated with copy constructors. These are as follows:

- Shallow Copy
- Deep Copy

-> **Shallow copy copies pointer address.**

-> **Deep copy copies actual data into new memory.**

-> **Shallow copy causes double free and dangling pointer issues.**

1. Shallow Copy Constructor in C++

In C++, a shallow copy constructor enables a programmer to specify how the values of one object are copied into another, including custom functionality such as logging or tracking. Only non-pointer members are duplicated directly during shallow copying, thus the underlying resources of both objects may be shared.

A shallow copy can only be produced by the default constructor. It is very useful and effective when it does not need any dynamic memory allocation.

A shallow copy copies the values of members as-is.

If a member is a pointer → it copies the pointer address, NOT the actual data it points to.

C++ Shallow Copy Example

```

#include <iostream>
using namespace std; //using standard namespace
class shallow_copy //class
{
    public: //Access Modifier
        int cod;
        shallow_copy(int c) : cod(c) {}
        shallow_copy (const shallow_copy &i) : cod(i.cod)
        {
            cout << "Copied Item with code: " << cod << endl; //prints the output
        }
};
int main() //Main Function
{
    shallow_copy s1(200);
    shallow_copy s2 = s1; //shallow copy
    return 0;
}

```

Ouput: ./constructor_example_copy_shallow
Copied Item with code: 200

Another Example:

```

#include <iostream>
#include <string.h>
using namespace std;
class Name {
    private:
        char* nm;
    public:
        Name(const char* s)

```

```

    {
        nm = new char[strlen(s)+1];
        strcpy(nm, s);
    }
    Name(const Name& other)
    {
        nm = other.nm;
        cout << "Shallow copy constructor called\n";
    }
    ~Name()
    {
        delete[] nm;
        cout << "Destructor Called\n";
    }
    void show()
    {
        cout << "Name:- " << nm << endl;
    }
};
int main(int argc, char** argv)
{
    char* str = argv[1];
    Name n(str);
    Name n1(n);
    n1.show();
    return 0;
}
Output: ./constructor_example_copy_shallow_private ShivRam
Shallow copy constructor called
Name:- ShivRam
Destructor Called
free(): double free detected in tcache 2
Aborted (core dumped)

```

2. Deep Copy Constructor in C++

Deep copy dynamically allocates the memory for the copy and then copies the actual value. Both the source and copy have distinct memory locations. In this way, both the source and copy are distinct and will not share the same memory location. Deep copy requires us to write the user-defined constructor. Deep copy constructors give control over resource duplication, providing object independence and safe memory handling.

```

#include <iostream>
#include <string.h>
using namespace std;
class Car {
private:
    int* speed;
public:
    Car(int s)
    {
        speed = new int(s);
    }
    Car(const Car& s)
    {
        speed = new int(*s.speed);
        cout << "Deep Copy Constructor Called.\n";
    }
    ~Car()
    {

```

```

        delete speed;
        cout << "Destructor Called.\n";
    }
    void show()
    {
        cout << "Speed is " << *speed << " km/hr.\n";
    }
};
int main()
{
    Car a(75);
    Car b = a;
    a.show();
    b.show();
    return 0;
}

```

Output: ./constructor_example_copy_deep_private
 Deep Copy Constructor Called.
 Speed is 75 km/hr.
 Speed is 75 km/hr.
 Destructor Called.
 Destructor Called.

Another Example

```

#include <iostream>
#include <string.h>
using namespace std;
class data {
    char* orig;
public:
    data(const char* input)
    {
        cout << "Constructor Called.\n";
        orig = new char[strlen(input)+1];
        strcpy(orig, input);
    }
    data(const data& s)
    {
        orig = new char[strlen(s.orig)+1];
        strcpy(orig, s.orig);
    }
    ~data()
    {
        delete[] orig;
        cout << "Destructor Called.\n";
    }
    void show()
    {
        cout << "data:- " << orig << endl;
    }
};
int main(int argc, char** argv)
{
    char* str = argv[1];
    data d(str);
    data d1 = d;
    d.show();
    d1.show();
    return 0;
}

```

```
}
```

```
Output: ./constructor_example_copy_deep_private_1 ShivRam
```

```
Constructor Called.
```

```
data:- ShivRam
```

```
data:- ShivRam
```

```
Destructor Called.
```

```
Destructor Called.
```

Move Constructor

In C++, the move constructor is a special type of constructor that allows the resources of one object to be moved into another instead of copying the object in the memory. It works with r-value references and is based on the move semantics. It enables us to transfer the ownership of resources from one object to another without performing a deep copy, which helps to improve the performance of the program.

Syntax

```
Class Class_name{  
public:  
    Class_name(Class_name&& obj); //declaration of move constructor  
};
```

C++ Move Constructor Example

```
#include <iostream>  
#include <string.h>  
using namespace std;  
class Employee {  
    char* name;  
public:  
    // Constructor  
    Employee(const char* n)  
    {  
        name = new char[strlen(n)+1];  
        strcpy(name, n);  
        cout << "Constructor Called.\n";  
    }  
    // Copy Constructor  
    Employee(const Employee& obj)  
    {  
        name = new char[strlen(obj.name)+1];  
        strcpy(name, obj.name);  
        cout << "Copy Constructor Called.\n";  
    }  
    // Move Constructor  
    Employee(Employee&& obj)  
    {  
        name = obj.name;  
        obj.name = NULL;  
        cout << "Move Constructor Called.\n";  
    }  
    ~Employee()  
    {  
        delete[] name;  
        cout << "Destructor Called.\n";  
    }  
    void show()  
    {  
        if(name)  
            cout << "Name:- " << name << endl;  
        else  
            cout << "Null String.\n";  
    }  
};
```

```

    }
};
int main(int argc, char** argv)
{
    char* str = argv[1];
    Employee e(str);
    Employee e1 = move(e); // Move constructor called
    e.show();
    e1.show();
    return 0;
}

```

Output: ./move_constructor_example ShivRam

Constructor Called.

Move Constructor Called.

Null String.

Name:- ShivRam

Destructor Called.

Destructor Called.

Another Example of Move Constructors

```

#include <iostream>
using namespace std;
class value {
    int* data;
public:
    value(int v) : data(new int (v))
    {
        cout << "Constructor Called.\n";
    }
    value(value&& obj) : data(obj.data)
    {
        obj.data = NULL;
        cout << "Transferring ownership.\n";
    }
    ~value()
    {
        if(data != NULL)
        {
            cout << "Destructor Called.\n";
            delete data;
        }
    }
    int getdata() const
    {
        return data ? *data : 0;
    }
    void setdata(int v)
    {
        if(data)
        {
            *data = v;
        }
    }
};
int main(int argc, char** argv)
{
    int n = atoi(argv[1]);
    value v(n);
    value v1 = move(v);
}

```

```

    cout << "v Value:- " << v.getdata() << endl;
    cout << "v1 Value:- " << v1.getdata() << endl;
    return 0;
}
Output: ./move_constructor_example_1 50
Constructor Called.
Transferring ownership.
v Value:- 0
v1 Value:- 50
Destructor Called.

```

Characteristics of Constructors in C++

- The constructor has the same name as the class it belongs to.
- Constructors are typically declared in the class's public member section.
- When we create a class object, the constructor is immediately invoked.
- Constructors do not return values in C++, so they don't have any return type.
- Declaring a constructor virtual is not permitted.
- Multiple constructors can not be declared within a single class.
- When allocating memory, the constructor makes implicit calls to the new and delete operators.

Constructors with Default Parameters

A constructor may have some default parameter values, which basically means that if no value is passed in, the default value will be used.

C++ Constructor With Default Parameters Example

```

#include <iostream>
using namespace std;
class Rectangle {
private:
    int len, wid;
public:
    Rectangle(int l = 10, int w = 15) : len(l), wid(w) {}
    void Area()
    {
        cout << "Area of Rectangle:- " << len*wid << endl;
    }
};
int main()
{
    Rectangle r1;
    r1.Area();
    Rectangle r2(7, 5);
    r2.Area();
    return 0;
}

```

Output: ./constructor_with_default_para
Area of Rectangle:- 150
Area of Rectangle:- 35

Parameterized Constructors in C++

C++ STL (Standard Template Library)

In C++, a standard template library (STL) is a powerful set of template classes and functions that provide general data structures and algorithms. It allows developers to use well-optimized implementation of general programming components, such as [dynamic arrays](#), [linked lists](#), [stacks](#), [queues](#), sets, [maps](#), and various algorithms.

Features of STL

- **Reusability:** The code reduces repetition through the reusable components.
- **Efficiency:** It includes adapted, pre-tested algorithms, and containers.
- **Generic programming:** It uses templates to work with any data type.
- **Consistency:** It provides uniform structure in various components, such as containers, algorithms, and recurrences.

Types of STL

- Containers
- Algorithms
- Iterators
- Functors (Function Objects)

STL Containers

Containers can be described as the objects that hold the data of the same type. Containers are used to implement different data structures, such as arrays, lists, trees, etc.

Container	Description	Header file	iterator
vector	vector is a class that creates a dynamic array allowing insertions and deletions at the back.	<vector>	Random access
list	list is the sequence containers that allow the insertions and deletions from anywhere.	<list>	Bidirectional
deque	deque is the double ended queue that allows the insertion and deletion from both the ends.	<deque>	Random access
set	set is an associate container for storing unique sets.	<set>	Bidirectional
multiset	Multiset is an associate container for storing non-unique sets.	<set>	Bidirectional
map	Map is an associate container for storing unique key-value pairs, i.e. each key is associated with only one value(one to one mapping).	<map>	Bidirectional
multimap	multimap is an associate container for storing key- value pair, and each key can be associated with more than one value.	<map>	Bidirectional
stack	It follows last in first out(LIFO).	<stack>	No iterator
queue	It follows first in first out(FIFO).	<queue>	No iterator
Priority-queue	First element out is always the highest priority element.	<queue>	No iterator

Sequence Containers:

- **Vector:** A dynamic array that automatically resizes.
- **Deque:** A double-ended queue allows rapid insertion and deletion at both ends.
- **List:** A doubly-linked list that supports rapid insertion and deletion in any situation.

Associative Containers:

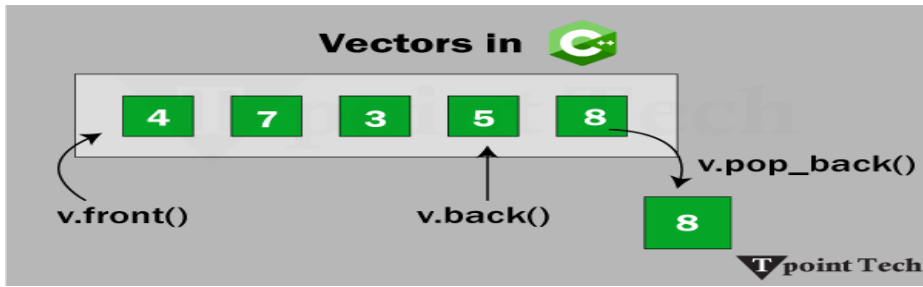
- **Set:** It stores unique elements in sorted order.
- **Map:** It stores the Key value pairs that are unique and sorted.
- **Multiset:** It is similar to the set but allows duplicate elements.
- **Multimap:** It is similar to MAP, but duplicate keys are allowed.

Unordered Associative Containers

- **Uncontrolled Set:** It stores unique elements in an unordered fashion.
- **Unordered Map:** It stores key-value pairs with unique keys in no particular order.
- **Unordered Multiset:** It is similar to unordered_set but enables duplicate elements.
- **Unordered Multimap:** It is similar to unordered_map but enables multiple elements with the same key.

C++ Vectors

In C++, vectors are a part of the standard template library (STL). They are dynamic arrays that can automatically resize themselves when the elements are added or removed.



Unlike [arrays](#), which have a certain size, vectors offer a wide range of flexibility and underlying functions, which makes them one of the most useful data structures in C++. The vectors provide many useful member functions, such as `push_back()`, `insert()`, `size()`, `clear()`, `front()`, `pop_back()`, etc.

A vector is a sequence container class that implements a dynamic array, which means size automatically changes when appending elements. A vector stores the elements in contiguous memory locations and allocates the memory as needed at run time. When more space is required, the vectors allocate new memory, copy existing elements into new memory, and deal with old memory.

Syntax

```
vector<T> v;
```

T represent type of the elements & v represent the name of the vector

Creating a Vector

If we need to create a vector, we have to include the `<vector>` header.:

```
std::vector<datatype> vector_name; // Empty vector
```

C++ Vector Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> nums;
    vector<int> size (5);
    vector<int> val(3, 10);
    vector<int> primes = {2, 3, 5, 7, 11};
    cout << "Vector created." << endl;
    for(int x : val)
    {
        cout << x << " ";
    }
    cout << endl;
    return 0;
}
```

Output: `./vector_create`

Vector created.

10 10 10

Inserting Elements

In C++, an element may be inserted into a vector via the `insert()` method, which takes a linear time. If we want to insert the number at the end, the `push_back()` method can be used. It works faster and takes only constant time.

Syntax

```
vector_name.push_back(value);
```

```
vector_name.insert(vector_name.begin() + index, value);
```


C++ Inserting Element in C++

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v = {2, 3, 5, 11, 17};
    v.push_back(19);
    v.insert(v.begin()+3, 7);
    cout << "Elements of vector are: ";
    for(int i=0;i<v.size();i++)
    {
        cout << v[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output: ./vector_insert

Elements of vector are: 2 3 5 7 11 17 19

Accessing and Updating Elements

We can access the elements using the [] operator or at() method. The [] method is fast but unsafe because it does not check whether the given index exists in the vector. We can also use the .at() method to access elements. The elements can be updated by assigning a new value using their index.

Syntax

```
vector_name[index];
vector_name.at(index);
vector_name[index] = new_value;
```

Accessing and Updating Elements Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v = {2, 3, 5, 7, 11, 13, 17, 19, 23};
    cout << "Element at 3rd position:- " << v[2] << endl;
    cout << "Element at 5th position:- " << v.at(4) << endl;
    v[2] = 29;
    v.at(4) = 31;
    cout << "Element at 3rd position:- " << v[2] << endl;
    cout << "Element at 5th position:- " << v.at(4) << endl;
    return 0;
}
```

Output: ./vector_accessing Updating

Element at 3rd position:- 5

Element at 5th position:- 11

Element at 3rd position:- 29

Element at 5th position:- 31

Find Vector Size

One of the main common problems of arrays is the need to maintain a separate variable to track their size. Vector can solve this problem by using the size() function, which helps to return the current number of elements in the vector.

Syntax

```
vector_name.size();
```

Example to Find Vector Size

```
#include <iostream>
#include <vector>
```

```
using namespace std;
int main()
{
    vector<int> v = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
    cout << "Size of the vector(v):- " << v.size() << endl;
    return 0;
}
```

Output: ./vector_size

Size of the vector(v):- 10

Traversing a Vector

We can traverse a vector using traditional for loops, range-based for loops, and iterators. We can utilize the loop and determine the size of the vector via the vector size() method to iterate via this range.

Example to Traverse a Vector

```
#include <iostream>
#include <vector>
#define v int
using namespace std;
int main()
{
    vector<v> v1 = {1 ,3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25};
    cout << "Elements of vector are:- ";
    for(int i=0;i<v1.size();i++)
    {
        cout << v1[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output: ./vector_traverse

Elements of vector are:- 1 3 5 7 9 11 13 15 17 19 21 23 25

Deleting Elements

The vectors allow the removal of the ultimate element or remove the elements from any condition using erase() and pop_back() functions. If we know the value of the element in the vector, we can use the find() function to find the location of this element. After that, we can use the vector pop_back() function to delete the element that takes only constant time.

Syntax

```
vector_name.pop_back();
vector_name.erase(vector_name.begin() + index);
```

Example of Delete Elements in a Vector

```
#include <iostream>
#include <vector>
#include <algorithm> // <-- REQUIRED for std::find
using namespace std;
int main()
{
    vector<int> v = {1 , 3, 5, 7, 9, 11, 13, 15, 17 ,19, 21};
    v.pop_back();
    auto it = find(v.begin(), v.end(), 11);
    if(it != v.end())
    {
        v.erase(it);
    }
    cout << "Elements of the vector(v) are:- ";
    for(int i=0;i<v.size();i++)
    {
```

```

        cout << v[i] << " ";
    }
    cout << endl;
    return 0;
}

```

Output: ./vector_delete

Elements of the vector(v) are:- 1 3 5 7 9 13 15 17 19

Time Complexity of Vector Operations

Operation	Time Complexity
Access (index)	O(1)
Insertion (end)	O(1)
Deletion (end)	O(1)
Insertion (mid)	O(n)
Deletion (mid)	O(n)

C++ Vector at()

It gives a reference to an element.

Syntax

```

vector<object_type> v;
v.at(k) ;

```

Parameter

k: k defines the position of element which is to be returned by at() function.

Return value

It returns the element of specified position.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v = {1, 3, 5, 7, 9};
    cout << "v.at(2):- " << v.at(2) << endl;
    return 0;
}

```

Output: ./vector_at

v.at(2):- 5

C++ Vector back()

It gives a reference to the last element.

Syntax

```

v.back();

```

Parameter

This function does not contain any parameter.

Return value

This function returns the last element of vector

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<string> name{"Shiv", "Ram", "SitaRam", "ShivShakti", "Hanuman"};
    cout << "vector back():- " << name.back() << endl;
    return 0;
}

```

Output: ./vector_back

vector back():- Hanuman

C++ Vector front()

It gives a reference to the first element of vector.

Syntax

```
v.front();
```

Parameter

This function does not contain any parameter.

Return value

This function returns the first element of the vector.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<char> dev{'H', 'A', 'N', 'U', 'M', 'A', 'N'};
    cout << "vector front():- " << dev.front() << endl;
    return 0;
}
Output: ./vector_front
vector front():- H
```

C++ Vector swap()

This function is used to exchange the elements specified in two vectors.

Syntax

```
v1.swap(v2);
```

Parameter

v2: v2 is a vector, whose value is to be swapped with another vector.

Return value

It does not return any value.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> odd{1, 3, 5, 7, 9};
    vector<int> even{0, 2, 4, 6, 8};
    cout << "Before, Elements of vector odd are: ";
    for(int x : odd)
    {
        cout << x << " ";
    }
    cout << endl;
    cout << "Before, Elements of vector even are: ";
    for(int x : even)
    {
        cout << x << " ";
    }
    cout << endl;
    odd.swap(even);
    cout << "After, Elements of vector odd are: ";
    for(int x : odd)
    {
        cout << x << " ";
    }
    cout << endl;
    cout << "After, Elements of vector even are: ";
    for(int x : even)
```

```

{
    cout << x << " ";
}
cout << endl;
return 0;
}

```

Output: ./vector_swap

Before, Elements of vector odd are: 1 3 5 7 9

Before, Elements of vector even are: 0 2 4 6 8

After, Elements of vector odd are: 0 2 4 6 8

After, Elements of vector even are: 1 3 5 7 9

C++ Vector **push_back()**

This function adds a new element at the end of the vector.

Syntax

```
v.push_back(k);
```

Parameter

k: k is the value which is to be inserted at the end of the vector.

Return value

This function does not return any value.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    for(int i=0;i<v.size();i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}

```

Output: ./vector_push_back

1 2 3

C++ Vector **pop_back()**

It deletes the last element and reduces the size of the vector by one.

Syntax

```
v.pop_back();
```

Parameter

It does not contain any parameter.

Return value

It does not return any value.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<char> ch{'H', 'A', 'N', 'U', 'M', 'A', 'N', 'J', 'T'};
    for(int i=0;i<ch.size();i++)
        cout << ch[i] ;
    cout << endl;
    ch.pop_back();
    ch.pop_back();
    for(int i=0;i<ch.size();i++)

```

```

        cout << ch[i] ;
    cout << endl;
    return 0;
}
Output: ./vector_pop_back
HANUMANJI
HANUMAN

```

C++ Vector empty()

This function determines whether the vector is empty or not.

Syntax

```
bool empty();
```

Parameter

It does not contain any parameter.

Return value

It returns a boolean value either 0 or 1.

```

Example
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v;
    if(v.empty())
        cout << "Vector v is Empty.\n";
    else
        cout << "Vector v is not Empty.\n";
    return 0;
}
Output: ./vector_empty
Vector v is Empty.

```

C++ Vector insert()

It is used to insert new element at specified position.

Syntax

```

insert(iterator,val);
insert(iterator,n,val);
insert(iterator,InputIterator first,InputIterator last);

```

Parameter

- **iterator**: An iterator defines the position, where the new elements are to be inserted.
- **val**: The val specifies the value which is to be inserted.
- **n**: Number of times the value is to be occurred.
- **(first,last)**: It defines the range of elements which is to be inserted.

Return value

It returns an iterator pointing to the newly inserted element.

Example 1

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    vector<int> c{100, 101, 102};
    v.insert(v.begin()+2, 6);//single element
    //v.insert(v.begin()+2, 3, 6);//multiple element
    //v.insert(v.begin()+1, c.begin(), c.end()); // from another vector
    //v.insert(v.begin()+4, {10001, 10002, 10003});
    for(int i=0;i<v.size();i++)

```

```

        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./vector_insert_example
1 2 6 3 4 5

```

Example 2

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    v.insert(v.begin()+2, 3, 6); //multiple element
    for(int i=0; i<v.size(); i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./vector_insert_example
1 2 6 6 3 4 5

```

Example 3

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    vector<int> c{100, 101, 102};
    v.insert(v.begin()+1, c.begin(), c.end()); // from another vector
    for(int i=0; i<v.size(); i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./vector_insert_example
1 100 101 102 2 3 4 5

```

Example 4

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    v.insert(v.begin()+4, {10001, 10002, 10003});
    for(int i=0; i<v.size(); i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./vector_insert_example
1 2 3 4 10001 10002 10003 5

```

C++ Vector erase()

It deletes the specified elements pointed by the iterator

Syntax

```

v.erase(pos);
v.erase(start_iterator, end_iterator);

```

Parameter

pos: It defines the position of the element which is to be removed from the vector.
(start_iterator,end_iterator) : It defines the range of the elements to be removed from the vector.

Return value

It does not return any value.

Example 1

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    v.erase(v.begin()+2);
    for(int x : v)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./vector_erase

1 2 4 5

Example 2

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    v.erase(v.begin()+2, v.begin()+5);
    for(int x : v)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./vector_erase

1 2 6 7 8 9 10

C++ Vector resize()

It modifies the size of the vector to the specified value

Syntax

```
v.resize(n,val);
```

Parameter

n : It is the new vector size.

val : If n is greater than the current vector size, then value(val) is inserted in the added space.

Return value

It does not return any value.

Example 1

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5, 6, 7, 8, 9};
    cout << "Elements of vector are before resize():- ";
    for(int x : v)
        cout << x << " ";
    cout << endl;
}
```



```

v.resize(5);
cout << "Elements of vector are after resize():- ";
for(int x : v)
    cout << x << " ";
cout << endl;
return 0;
}

```

Output: ./vector_resize

Elements of vector are before resize():- 1 2 3 4 5 6 7 8 9

Elements of vector are after resize():- 1 2 3 4 5

Example 2

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5, 6, 7, 8, 9};
    cout << "Elements of vector are before resize():- ";
    for(int x : v)
        cout << x << " ";
    cout << endl;
    v.resize(13, 108);
    cout << "Elements of vector are after resize():- ";
    for(int x : v)
        cout << x << " ";
    cout << endl;
    return 0;
}

```

Output: ./vector_resize

Elements of vector are before resize():- 1 2 3 4 5 6 7 8 9

Elements of vector are after resize():- 1 2 3 4 5 6 7 8 9 108 108 108 108

C++ Vector clear()

This function removes all the elements from the vector.

Syntax

```
v.clear();
```

Parameter

It does not contain any parameter.

Return value

It does not return any value.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    cout << "Elements of vector v before clear are: ";
    for(int x : v)
        cout << x << " ";
    cout << endl;
    v.clear();
    cout << "Elements of vector v after clear are: ";
    for(int x : v)
        cout << x << " ";
    cout << endl;
    return 0;
}

```

Output: ./vector_clear

Elements of vector v before clear are: 1 2 3 4 5

Elements of vector v after clear are:

C++ Vector size()

It determines the number of elements in the vector.

Syntax

```
int n=v.size();
```

Parameter

It does not contain any parameter.

Return value

It returns the number of elements in the vector.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
    cout << "Size of the vector(v):- " << v.size() << endl;
    return 0;
}
```

Output: ./vector_size

Size of the vector(v):- 10

C++ Vector capacity()

This function determines the current capacity of the vector.

Note: Capacity of the vector can be equal or greater than the size of the vector and if it is greater than the size of the vector, means allowing extra space to accommodate further operations.

Syntax

```
int c=v.capacity();
```

Parameter

It does not contain any parameter.

Return value

It returns the current allocated capacity of the vector.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v;
    for(int i=0;i<=10;i++)
    {
        v.push_back(i);
    }
    cout << "Size of vector v:- " << v.size() << endl;
    cout << "Capacity of vector v:- " << v.capacity() << endl;
    return 0;
}
```

Output: ./vector_capacity

Size of vector v:- 11

Capacity of vector v:- 16

C++ Vector assign()

This function assigns new values to the vector and replacing the old ones.

Syntax

```
v.assign(first,last);
```

```
v.assign(n,val);
```

Parameter

(first,last) :It defines the range.first is an input iterator pointing to the first element and last is an input iterator referring to the past the last element.

n : Number of times the value to be occurred.

val : It defines the value which is to be assigned.

Return value

It does not return any value.

Example 1

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    vector<int> v1;
    v1.assign(v.begin(), v.end()-2);
    for(int x : v1)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./vector_assign

1 2 3

Example 2

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v1;
    v1.assign(5, 7);
    for(int x : v1)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./vector_assign

7 7 7 7 7

C++ Vector operator=()

This function assigns new values to the vector container and replacing old ones.

Syntax

```
v.operator=(v1);
```

Parameter

v1:v1 is a vector object.

Return value

It returns *this.

Example 1

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    vector<int> v1;
    v1.operator=(v);
    for(int x : v1)
        cout << x << " ";
}
```

```

        cout << endl;
        return 0;
    }
    Output: ./vector_operator
    1 2 3 4 5

```

Example 2

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v1{1, 2, 3, 4, 5};
    vector<int> v = {6, 7, 8, 9};
    v1.operator=(v);
    for(int x : v1)
        cout << x << " ";
    cout << endl;
    return 0;
}
    Output: ./vector_operator
    6 7 8 9

```

C++ Vector operator[]()

This function is used to access a specified element.

Syntax

```
v.operator[ ](pos);
```

Parameter

pos: It defines the position of the element.

Return value

It returns the element of specified location.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<string> dev{"Radhe Radhe", "Sita", "Ram", "Shakti", "Shiv"};
    for(int i=0;i<dev.size();i++)
        cout << dev.operator[](i) << " ";
    cout << endl;
    return 0;
}
    Output: ./vector_operator1
    Radhe Radhe Sita Ram Shakti Shiv

```

C++ Vector end()

This function returns an iterator referring to the past-last-element in the vector container.

Syntax

```
Iterator it=v.end()
```

Parameter

It does not contain any parameter.

Return value

It returns an iterator following the last element.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()

```

```

{
    vector<int> v{1, 2, 3, 4 ,5};
    vector<int> ::iterator it;
    for(it=v.begin();it!=v.end();it++)
        cout << *it << " ";
    cout << endl;
    return 0;
}

```

Output: ./vector_end

1 2 3 4 5

C++ Vector **emplace()**

This function inserts a new element just before the position pos and size of the vector container increases.

Syntax

Iterator it=v.emplace(pos,args);

Parameter

pos: It defines the position before which the new element is to be inserted.

args: Arguments forwarded to construct the new element.

Return value

It returns an iterator to the newly inserted element.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    cout << "Elements of vector v are:- ";
    for(int x : v)
        cout << x << " ";
    cout << endl;
    cout<<"After adding two elements, elements are :";
    vector<int>::iterator it=v.emplace(v.begin()+2,8);
    v.emplace(it, 9);
    for(int i=0;i<v.size();i++)
        cout<<v[i]<<" ";
    cout << endl;
    return 0;
}

```

Output: ./vector_emplace

Elements of vector v are:- 1 2 3 4 5

After adding two elements, elements are :1 2 9 8 3 4 5

C++ **emplace_back()**

This function is used to insert new element at the end of the vector and increases the size of the vector container.

Syntax

v.emplace_back(args);

Parameter

args: Arguments forwarded to construct the new element.

Return value

It does not return any value.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{

```

```

        vector<char> v{'C', '+'};
        v.emplace_back('+');
        for(char x : v)
            cout << x ;
        cout << endl;
        return 0;
    }
    Output: ./vector_emplace_back
    C++

```

C++ Vector rend()

The rend() function stands for 'reverse end' and is used to point the element preceding the first element of the vector.

Syntax

```
reverse_iterator ritr=v.rend();
```

Parameter

It does not contain any parameter.

Return value

It returns a reverse iterator pointing to the reverse end of the vector container.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    vector<int> ::reverse_iterator ritr;
    for(ritr=v.rbegin();ritr!=v.rend();ritr++)
        cout << *ritr << " ";
    cout << endl;
    return 0;
}
    Output: ./vector_rend
    5 4 3 2 1

```

C++ Vector rbegin()

The rbegin() stands for 'reverse beginning' and is used to point the last element of the vector.

Syntax

```
reverse_iterator ritr=v.rbegin();
```

Parameter

It does not contain any parameter.

Return value

It returns a reverse iterator pointing to the last element of the vector.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<char> v{'a', 'v', 'i', 'h', 'S'};
    vector<char> ::reverse_iterator rtr;
    for(rtr=v.rbegin();rtr!=v.rend();rtr++)
        cout << *rtr ;
    cout << endl;
    return 0;
}
    Output: ./vector_rbegin
    Shiva

```

C++ Vector begin()

This function is used to point the first element of the vector.

Syntax

```
iterator it =v.begin();
```

Parameter

It does not contain any parameter.

Return value

It returns an iterator pointing to the first element of the vector.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    vector<int> ::iterator itr;
    itr = v.begin();
    cout << *itr << endl;
    return 0;
}
```

Output: ./vector_begin

1

C++ Vector max_size()

This function is used to provide the maximum size that vector can hold.

Syntax

```
int m=v.max_size();
```

Parameter

It does not contain any parameter.

Return value

It returns the maximum number of elements that vector can hold.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 3, 5, 7, 9};
    cout << "vector v max_size():- " << v.max_size() << endl;
    return 0;
}
```

Output: ./vector_max_size

vector v max_size():- 2305843009213693951

C++ Vector cend()

This function is used to point to the past-the-last element (element after the last element) in the vector.

Syntax

```
const_iterator itr=v.cend();
```

Parameter

It does not contain any parameter.

Return value

It returns a constant iterator pointing to the past-the-last element in the vector.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
```

```

{
    vector<string> v{"Jai", "Jai", "Jai", "Shree", "Ram"};
    vector<string> ::const_iterator citr;
    for(citr=v.cbegin();citr!=v.cend();citr++)
        cout << *citr << " ";
    cout << endl;
    return 0;
}

```

Output: ./vector_cend

Jai Jai Jai Shree Ram

C++ Vector cbegin()

This function is used to point the first element of the vector container.

Syntax

```
const_iterator itr=v.cbegin();
```

Parameter

It does not contain any parameter.

Return value

It returns the constant iterator pointing to the first element of the vector.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<char> v{'S', 'i', 't', 'a', 'R', 'a', 'm'};
    vector<char> ::const_iterator ctr;
    ctr = v.cbegin()+2;
    cout << "v.cbegin()+2:- " << *ctr << endl;
    return 0;
}

```

Output: ./vector_cbegin

v.cbegin()+2:- t

C++ Vector crbegin()

The crbegin() function stands for reverse beginning. It is used to point the last character of the vector container.

Syntax

```
const_reverse_iterator itr=v.crbegin();
```

Parameter

It does not contain any parameter.

Return value

It returns constant reverse iterator pointing to the reverse beginning of the container.

Example

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    vector<int> ::const_reverse_iterator crt = v.crbegin()+2;
    cout << *crt << endl;
    return 0;
}

```

Output: ./vector_crbegin

3

C++ Vector **crend()**

This function is used to point the element preceding the first element in the vector container. It refers as reverse end.

Syntax

```
const_reverse_iterator itr=v.crend();
```

Parameter

It does not contain any parameter.

Return value

It returns the constant reverse iterator pointing to the reverse end in the sequence.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<char> ch{'S', 'i', 't', 'a', 'R', 'a', 'm'};
    vector<char> ::const_reverse_iterator cetr = ch.crend() - 1;
    cout << "ch.crend() - 1:- " << *cetr << endl;
    return 0;
}
Output: ./vector_crend
ch.crend() - 1:- S
```

C++ Vector **data()**

This function returns a pointer to an array internally used by a vector to store its elements.

Syntax

```
data_type *p=v.data();
```

Parameter

It does not contain any parameter.

Return value

It returns a pointer to an array.

Example

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    int *x = v.data();
    for(int i=0;i<v.size();i++)
        cout << *x++ << " ";
    cout << endl;
    return 0;
}
Output: ./vector_data
1 2 3 4 5
```

C++ Vector **shrink_to_fit()**

This function is used to change the capacity of the vector and makes it equal to the size of the vector.

Syntax

```
v.shrink_to_fit();
```

Parameter

It does not contain any parameter.

Return value

It does not return any value.

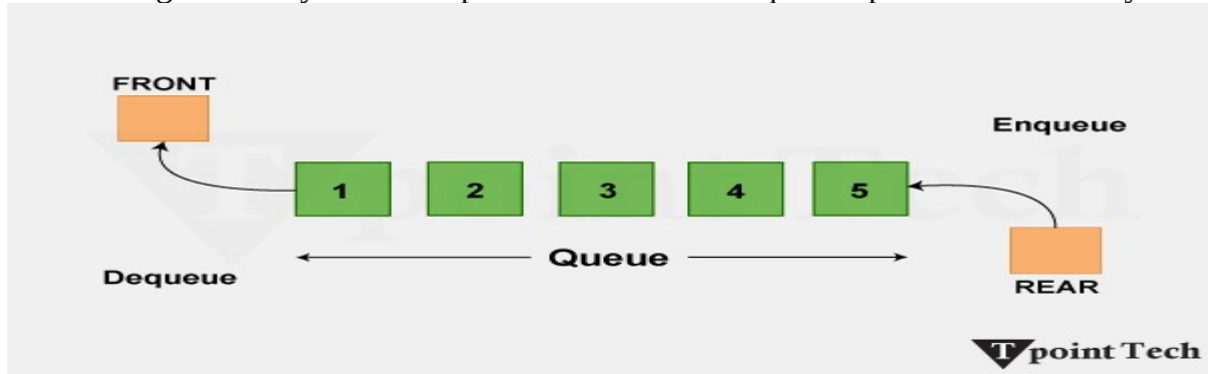
C++ Vector Functions

Function	Description
at()	It provides a reference to an element.
back()	It gives a reference to the last element.
front()	It gives a reference to the first element.
swap()	It exchanges the elements between two vectors.
push_back()	It adds a new element at the end.
pop_back()	It removes a last element from the vector.
empty()	It determines whether the vector is empty or not.
insert()	It inserts new element at the specified position.
erase()	It deletes the specified element.
resize()	It modifies the size of the vector.
clear()	It removes all the elements from the vector.
size()	It determines a number of elements in the vector.
capacity()	It determines the current capacity of the vector.
assign()	It assigns new values to the vector.
operator=()	It assigns new values to the vector container.
operator[]()	It access a specified element.
end()	It refers to the past-lats-element in the vector.
emplace()	It inserts a new element just before the position pos.
emplace_back()	It inserts a new element at the end.
rend()	It points the element preceding the first element of the vector.
rbegin()	It points the last element of the vector.
begin()	It points the first element of the vector.
max_size()	It returns the maximum number of elements that the vector can hold.
cend()	It refers to the past-last-element in the vector.
cbegin()	It refers to the first element of the vector.
crbegin()	It refers to the last character of the vector.
crend()	It refers to the element preceding the first element of the vector.
data()	It writes the data of the vector into an array.
shrink_to_fit()	It reduces the capacity and makes it equal to the size of the vector.

Note:- the vectors in C++ are highly efficient, easy to use, and a strong option for raw arrays. They provide a rich set of tasks for dynamic resizing, insertion, deletion, and traversal and integrate well with the rest of STL. Mastering vectors will not only simplify the code but will also make it more efficient and cleaner. Whether we are handling complex structures or simple data such as matrices, vectors are our go-to tool in C++.

C++ Queue

In C++, Queues represent fundamental data structures that operate according to first-in-first-out (FIFO) logic. The Standard Template Library (STL) in C++ offers a queue class through its readymade components that enhance queue operations efficiency.



Queue is the linear data structure that process elements through a queue mechanism that inserts items at the back and removes them from the front end. Queues work similarly to a queue at the ticket counter because customers receive service based on their order of arrival.

Queues establish their essential role in programming because developers need them to execute task-scheduling functionality, simulate line-based services, and perform level-order traversals in trees.

Operations of the Queue

1. **Enqueue:** It is used to insert an element at the end of the queue.
2. **Dequeue:** it is used to remove an element from the front end of the queue.
3. **Front:** It returns the first element of the queue.
4. **Rear:** It returns the last element of the queue.

Syntax

```
template<class Obj, class Container = deque<Obj> > class queue;
```

- **Obj:** It represents the type of the element in the queue.
- **Container:** It represents the type of container object.

Example

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    queue<int> q;
    q.push(10);
    q.push(20);
    q.push(30);
    cout << "Front element of queue:- " << q.front() << endl;
    cout << "Rear element of queue:- " << q.back() << endl;
    q.pop();
    cout << "Front element of queue:- " << q.front() << endl;
    return 0;
}
```

Output: ./queue_example
Front element of queue:- 10
Rear element of queue:- 30
Front element of queue:- 20

Key Characteristics of Queues:

- **FIFO Order:** Queues under the FIFO order system process their elements exactly according to their arrival sequence.
- **Dynamic Structure:** The queue structure enables both addition and removal operations to happen at a constant time.

Such systems actively participate in scheduling jobs while managing customer service operations and improving data buffering efficiency.

Declaration and Initialization of Queues in C++

Queue elements are initialized by using the push() function to insert elements one by one after declaration. In C++, we can declare and initialize the queue in several ways.

Syntax

```
#include <queue>
```

```
std::queue<datatype> queue_name; // Declaration
```

- **datatype:** The type of elements the queue will store (e.g., int, string, float).
- **queue_name:** It represents the identifier for the queue.

Example to Declare and Initialize the Queue

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    queue<char> q;
```

```
    q.push('R');
```

```
    q.push('A');
```

```
    q.push('M');
```

```
    cout << "Front Element:- " << q.front() << endl;
```

```
    cout << "Rear Element:- " << q.back() << endl;
```

```
    return 0;
```

```
}
```

```
Output: ./queue_initial
```

```
Front Element:- R
```

```
Rear Element:- M
```

Member Types

Member Types	Description
value_type	Element type is specified.
container_type	Underlying container type is specified.
size_type	It specifies the size range of the elements.
reference	It is a reference type of a container.
const_reference	It is a reference type of a constant container.

1) Insertion (push())

In C++, the insertion operation places the new element behind the current last entry in a queue system. The insertion process does not modify elements situated in the queue's front position. The process of inserting elements in a queue is also known as Enqueue. Several elements can be added to the queue sequentially.

Syntax

```
queue_name.push(value);
```

C++ Example for Inserting Element in Queue

```
#include <iostream>
```

```
#include <queue>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    queue<string> q;
```

```
    q.push("Jai");
```

```
    q.push("Jai");
```

```
    q.push("Shree");
```

```
    q.push("Ram");
```

```
    cout << q.front() << " " << q.back() << endl;
```

```
    return 0;
```

```
}
```

Output: ./queue_insert
Jai Ram

2) Deletion (pop())

The queue front element becomes the first element that will be removed through this operation. It follows the FIFO rule. The system processes and removes one element through each operation using the pop() function.

Popping an element does not provide the removed value because it requires users to access the front element first through the front() function. The removal process of elements from a queue is known as dequeuer.

Syntax

```
queue_name.pop();
```

Example for Deleting the Elements

```
#include <iostream>
#include <queue>
using namespace std;
int main(int argc, char** argv)
{
    queue<string> q;
    string str = "";
    for(int i=1;i<argc;i++)
    {
        q.push(argv[i]);
    }
    cout << "Before pop() - Front:- " << q.front() << " Rear:- " << q.back() << endl;
    q.pop();
    cout << "After pop() - Front:- " << q.front() << " Rear:- " << q.back() << endl;
    return 0;
}
```

Output: ./queue_delete Ravan SitaRam ShivSakti

Before pop() - Front:- Ravan Rear:- ShivSakti

After pop() - Front:- SitaRam Rear:- ShivSakti

3) Access Front Element (front())

This function provides access to the front queue element that will be removed from the queue next.

Syntax

```
queue_name.front();
```

C++ Example to Access Front Element

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    queue<int> q;
    q.push(10);
    q.push(30);
    cout << "Front:- " << q.front() << endl;
    return 0;
}
```

Output: ./queue_front

Front:- 10

4) Access Rear Element (back())

The method provides access to a reference of the queue's latest added element.

Syntax

```
queue_name.back();
```

Example

```
#include <iostream>
```

```
#include <queue>
using namespace std;
int main()
{
    queue<int> q;
    q.push(10);
    q.push(20);
    cout << "Back:- " << q.back() << endl;
    return 0;
}
```

Output: ./queue_back

Back:- 20

5) Checking Size (size())

The function returns the size of the queue container, which is a measure of the number of elements stored in the queue.

Syntax

```
queue_name.size();
```

C++ Example to Check the Size

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    queue<int> q;
    q.push(10);
    q.push(20);
    q.push(30);
    q.push(40);
    q.push(50);
    cout << "Queue q size():- " << q.size() << endl;
    return 0;
}
```

Output: ./queue_size

Queue q size():- 5

6) Empty() a Queue

The function is used to test for the emptiness of a queue. A function returns true only when the queue contains no elements, but it returns false for all other cases.

Syntax

```
queue_name.empty();
```

C++ Example to Empty() function in a Queue

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    queue<int> q;
    if(q.empty())
        cout << "Queue is Empty.\n";
    q.push(10);
    if(!q.empty())
        cout << "Queue is not Empty.\n";
    return 0;
}
```

Output: ./queue_empty

Queue is Empty.

Queue is not Empty.

7) Traversing a Queue

Queues require a traversal process to display all elements starting from the first until the last item. We traverse through and print queue elements by using a loop along with `front()` and `pop()` functions because `std::queue` does not support iterators.

The queue becomes empty after traversal because all elements have been removed.

We can preserve the original queue by making a copy into a different temporary queue while using this new queue for traversal.

C++ Example to Traverse a Queue

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    queue<char> q;
    q.push('S');
    q.push('h');
    q.push('r');
    q.push('e');
    q.push('e');
    q.push(' ');
    q.push('R');
    q.push('a');
    q.push('m');
    cout << "Queue(q) elements are:- ";
    while(!q.empty())
    {
        cout << q.front();
        q.pop();
    }
    cout << endl;
    return 0;
}
```

Output: `./queue_traverse`

Queue(q) elements are:- Shree Ram

Time Complexity

Operations	Time Complexity
Insert Element	O(1)
Delete Element	O(1)
Access Front Element	O(1)
Access Rear Element	O(1)
Traverse Queue	O(1)

Note:- Queues in C++ operate as essential data structures that implement FIFO logic to process instructions according to their arrival sequence. Through the queue container system of the Standard Template Library (STL) in C++, developers obtain efficient access to enqueue, dequeue and peek operations.

C++ Priority Queue

In C++, the priority queue is a derived container in [STL](#) that considers only the highest priority element. The queue follows the FIFO policy while the priority queue pops the elements based on the priority, i.e., the highest priority element is popped first.

- Each element in a priority queue has an associated priority, unlike a regular queue where all elements are treated equally.
- The element with the highest priority in a priority queue will be removed first, while the queue follows the FIFO(First-In-First-Out) policy means the element that is inserted first will be deleted first.
- If more than one element exists with the same priority, then the order of the elements in a queue will be taken into consideration.

Syntax

```
Priority_queue<int>variable_name;
```

- **int:** It is the type of the element.
- **variable_name:** It is the name of the variable.

Priority queue container offers the built-in implementation of binary heap data structure. These can be two types of heaps.

1) Max Heap Priority Queue (by default)

In a Max Heap priority queue, the element with the highest value has the highest priority and is accessed first. It is the default behaviour of `std::priority_queue` in C++.

Syntax

```
std::priority_queue<data_type> maxHeap;
```

- **data_type:** It represents the elements that are stored in the priority queue.

Max Heap Priority Queue Example

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    priority_queue<int> maxHeap;
    maxHeap.push(5);
    maxHeap.push(15);
    maxHeap.push(25);
    maxHeap.push(10);
    cout << "Elements in maxHeap of priority_queue:- ";
    while(!maxHeap.empty())
    {
        cout << maxHeap.top() << " ";
        maxHeap.pop();
    }
    cout << endl;
    return 0;
}
```

Output: ./priority_queue_maxHeap

Elements in maxHeap of priority_queue:- 25 15 10 5

Note:- The `priority_queue<int>` uses `std::less<int>` internally. The largest element (30) is given the highest priority. Elements are printed in decreasing order of value.

2) Min Heap Priority Queue (Custom Comparator)

In a Min Heap priority queue, the element with the lowest value has the highest priority and is accessed first. We can achieve this using a custom comparator like `std::greater`.

Syntax

```
std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;
```

- **Vector<int>:** It represents the STL Container.
- **greater<int>:** It represents the comparator class.

Min Heap Priority Queue Example

```
#include <iostream>
#include <vector>
#include <queue>
#include <functional>
using namespace std;
int main()
{
    priority_queue<int, vector<int>, greater<int>> minHeap;
    minHeap.push(25);
    minHeap.push(5);
    minHeap.push(15);
    minHeap.push(35);
    cout << "Elements in minHeap of priority_queue:- ";
    while(!minHeap.empty())
    {
        cout << minHeap.top() << " ";
        minHeap.pop();
    }
    cout << endl;
    return 0;
}
```

Output: ./priority_queue_minHeap

Elements in minHeap of priority_queue:- 5 15 25 35

Note:- The third template argument, `std::greater<int>`, changes the behaviour to a min-heap. The smallest element (5) gets the highest priority. Elements are printed in increasing order of value.

Basic Operations of Priority Queues

1) Inserting Elements

In priority queues, the `push()` method is used to insert elements in the queue. The heap property stays intact during each insertion because it automatically positions the most significant aspect (greatest by default) toward the top.

2) Accessing Top Element

The `top()` function extracts the maximum element from the priority queue based on its default largest value priority. The priority queue allows users to obtain the maximum element using constant time ($O(1)$) operations without deleting it. The `top` function will enable us to view a maximum value before executing actions such as the `pop()` function.

Inserting and Accessing Top Element Example

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    priority_queue<int> pq;
    pq.push(10);
    pq.push(30);
    pq.push(20);
    cout << "Maximum Value:- " << pq.top() << endl;
    return 0;
}
```

Output: ./priority_queue_insert_access

Maximum Value:- 30

3) Deleting Top Element

The `pop()` function is used to delete the element at the front position (top element) of a priority queue based on its priority level. A user should use the `top()` function first to retrieve the removed element because `pop()` does not return any object. The queue

system processes the next largest element, which automatically assumes the position of the top selection.

Deletion of Top Element in Priority Queue Example

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    priority_queue<int> pq;
    pq.push(10);
    pq.push(30);
    pq.push(20);
    cout << "Before pop(), Maximum Value:- " << pq.top() << endl;
    pq.pop();
    cout << "After pop(), Maximum Value:- " << pq.top() << endl;
    return 0;
}
```

Output: ./priority_queue_delete
Before pop(), Maximum Value:- 30
After pop(), Maximum Value:- 20

4) Traversing (Pseudo Traversal)

A priority queue does not provide direct access for traversing the way vectors or lists do. The process for accessing all elements begins with top() followed by an immediate pop() operation. A print sequence of elements occurs in descending order since the priority queue operates as a default max-heap structure.

C++ Traversing Example in Priority Queue

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    priority_queue<int> pq;
    pq.push(10);
    pq.push(30);
    pq.push(20);
    cout << "Elements of priority_queue(pq) are:- ";
    while(!pq.empty())
    {
        cout << pq.top() << " ";
        pq.pop();
    }
    cout << endl;
    return 0;
}
```

Output: ./priority_queue_traversal
Elements of priority_queue(pq) are:- 30 20 10

5) Changing Priority Queue Order

C++ developers can alter priority queue order in their applications through the definition of a custom comparator. The default operation of a C++ priority queue establishes max-heap behaviour that keeps higher priority elements higher in the data structure. A custom comparator enables us to transform a priority queue from its default max-heap type to either a min-heap or any custom prioritization scheme.

Example of Changing the Priority Queue Order

```
#include <iostream>
#include <queue>
using namespace std;
int main()
```

```

{
    priority_queue<int, vector<int>, greater<int>> minHeap;
    minHeap.push(10);
    minHeap.push(40);
    minHeap.push(30);
    minHeap.push(20);
    cout << "minHeap size:- " << minHeap.size() << endl;
    cout << "Top Element (min-heap):- " << minHeap.top() << endl;
    return 0;
}

```

Output: ./priority_queue_changing_order

minHeap size:- 4

Top Element (min-heap):- 10

Time Complexity

Operation	Time Complexity	Explanation
pop()	$O(\log n)$	Removing the top element requires reheapifying the structure.
push()	$O(\log n)$	Inserting an element requires maintaining the heap property (logarithmic).
top()	$O(1)$	Accessing the maximum (or minimum) element is done in constant time.
empty() / size()	$O(1)$	Checking if the queue is empty or getting its size is constant time.
Traversal via popping	$O(n \log n)$	Popping all n elements takes $\log n$ time each, totalling to $n \log n$.

Member Function of Priority Queue

Function	Description
push()	It inserts a new element in a priority queue.
pop()	It removes the top element from the queue, which has the highest priority.
top()	This function is used to address the topmost element of a priority queue.
size()	It determines the size of a priority queue.
empty()	It verifies whether the queue is empty or not. Based on the verification, it returns the status.
swap()	It swaps the elements of a priority queue with another queue having the same type and size.
emplace()	It inserts a new element at the top of the priority queue.

6) swap() in std::priority_queue

Swaps contents of two priority queues in $O(1)$ time.

Syntax

```

pq1.swap(pq2);
swap(pq1, pq2);

```

Example of swap in priority queue

```

#include <iostream>
#include <queue>
using namespace std;
int main()
{
    priority_queue<int> pq1;
    priority_queue<int> pq2;
    pq1.push(1);
    pq1.push(3);
    pq1.push(5);
    pq2.push(2);
}

```

```

    pq2.push(4);
    pq2.push(6);
    cout << "Before swap(), top element of pq1:- " << pq1.top() << endl;
    pq1.swap(pq2);
    cout << "After swap(), top element of pq1:- " << pq1.top() << endl;
    return 0;
}

```

Output: ./priority_queue_swap

Before swap(), top element of pq1:- 5

After swap(), top element of pq1:- 6

7) **emplace() in std::priority_queue**

emplace() constructs a new element in-place at the correct position in the heap (without creating a temporary object).

Syntax

```

pq.emplace(args);

```

Example

```

#include <iostream>
#include <queue>
using namespace std;
int main()
{
    priority_queue<int> pq;
    pq.emplace(15);
    pq.emplace(5);
    pq.emplace(25);
    cout << "Maximum Value: " << pq.top() << endl;
    return 0;
}

```

Output: ./priority_queue_emplace

Maximum Value: 25

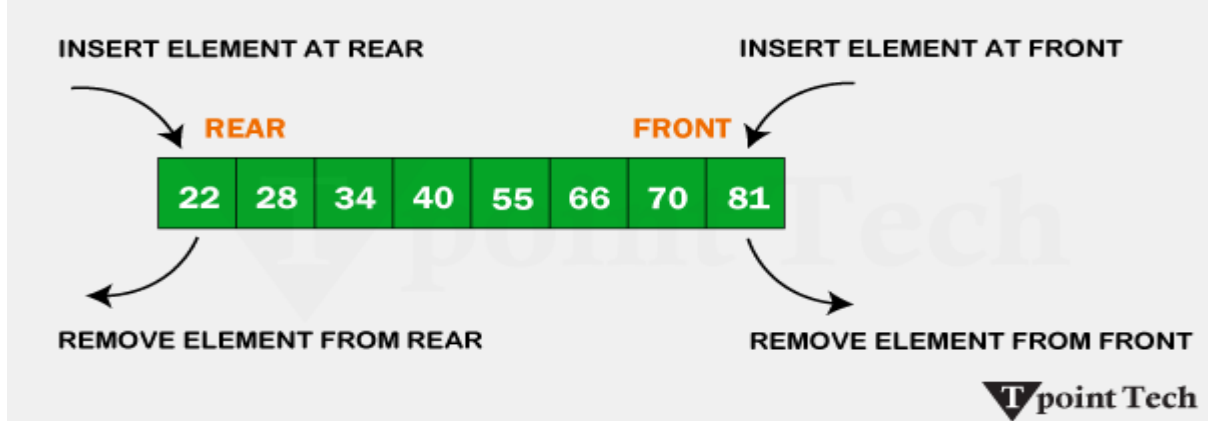
Purpose of the Priority Queues

- **Efficient Element Access:** Priority queues enable constant-time ($O(1)$) element lookup of the most prioritized item and logarithmic ($O(\log n)$) operations on insert and delete commands. Therefore, they excel at high-performance applications.
- **Order Management:** The system performs automatic element realignment through reordering processes to keep priority intact after adding or removing entries.
- **Support for Custom Priorities:** The priority queue system allows users to create their element comparison methods through custom comparator tools.
- **Useful in Real-time Processing:** Real-time processing systems receive maximum benefit from using priority queues because they enable quick responses to high-priority tasks.

Note:- C++ priority queues provide data structures that allow developers to retrieve either maximum or minimum priority elements efficiently. Through the usage of an STL priority queue, developers can build heaps for scheduling applications pat, finding systems, and resource allocation processes. The time complexity for inserting and deleting data in priority queues is $O(\log n)$, which makes them an ideal option for performance-sensitive applications.

C++ Deque

In C++, the deque (Double-Ended Queue) is a queue in which elements can be added and removed from the front and back. It supports both FIFO and LIFO operations, which offers greater flexibility than queues. In C++, the STL deque is a sequential container that allows for efficient insertion and deletion of elements from both the front and back.



Unlike a regular [queue](#), which only allows for insertion at the back and removal from the front, a deque provides more flexibility by enabling operations at both ends. It also allows element access using index positions, which makes it suitable for scenarios where random access and dynamic resizing are necessary. If we want to work with a deque container, we have to include a <deque> header file.

Syntax

`deque<object_type> deque_name;`

- **object_type:** The `object_type` parameter determines the type of components that the deque will store, such as `int`, `string`, and custom class.
- **deque_name:** It represents the name of the identifier and variable that is assigned to deque.
- **;;** It ends the statement according to C++ syntax instructions.

Simple C++ Deque Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 2, 3, 5, 7};
    cout << "Elements of deque(dq) are:- ";
    for(auto a:dq)
        cout << a << " ";
    cout << endl;
    return 0;
}
```

Output: `./deque_example`

Elements of deque(dq) are:- 1 2 3 5 7

Basic operations on Deque

1) Creating and Inserting Elements in a Deque:

In order to declare a deque, use the deque keyword followed by the data type wrapped in angle brackets (< >), and the variable name. The Standard Template Library (STL) provides the deque as a container that enables fast insertions and deletions at both ends.

In C++ deque, we can perform the insertion and deletion operation at either end using the `push_back()` and `push_front()` functions. If we want to insert an element at a specific position, we can use the `insert()` method.

Example of Creating and Inserting Element on Deque

```
#include <iostream>
```

```
#include <deque>
using namespace std;
int main()
{
    deque<int> dq;
    dq.push_back(1);
    dq.push_back(3);
    dq.push_front(2);
    dq.push_front(4);
    dq.emplace_back(5);
    dq.emplace_back(7);
    dq.emplace_front(6);
    dq.emplace_front(8);
    cout << "Deque Elements are:- ";
    for(auto& x : dq)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_create_insert

Deque Elements are:- 8 6 4 2 1 3 5 7

2) Accessing the Elements in a Deque

The elements of a deque can be accessed directly with index notation, which contains square [] brackets. Deques are 0-indexed, which means that the first element is at index 0, the second is at index 1, and so on. It allows for quick and easy retrieval of elements from particular positions within the container. It also offers the front() and back() methods to easily access the first and last element from the list.

Example of Accessing the Elements in a Deque

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<string> dq{"Shree", "Narayan", "Man", "Hari", "Narayan", "Hari"};
    cout << "Use front():- " << dq.front() << endl; //Shree
    cout << "Use index 2:- " << dq[2] << endl; //Man
    cout << "Use at(1) :- " << dq.at(1) << endl; //Narayan
    cout << "Use index 4:- " << dq[4] << endl; //Narayan
    cout << "Use at(3) :- " << dq.at(3) << endl; //Hari
    cout << "Use back() :- " << dq.back() << endl; //Hari
    return 0;
}
```

Output: ./deque_accessing

Use front():- Shree

Use index 2:- Man

Use at(1) :- Narayan

Use index 4:- Narayan

Use at(3) :- Hari

Use back() :- Hari

3) Modifying the Deque Element

In order to change the value of a specific element in a deque, use its index wrapped in square brackets [] and the at() method. These functions offer direct access to an element's location and value replacement. While [] operator allows for speedier access, but the at() function is more secure because it performs bounds checks and throws an exception if the index is outside of range.

Example of Modifying the Deque Element

```
#include <iostream>
```

```

#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    dq[0] = 11;
    dq.at(2) = 17;
    dq.push_front(23);
    dq.push_back(29);
    dq.pop_front();
    dq.pop_back();
    cout << "Remaining Data in dq:- ";
    for(auto& x : dq)
        cout << x << " ";
    cout << endl;
    return 0;
}

```

Output: ./deque_modifying

Remaining Data in dq:- 11 3 17 7 9

4) Removing Elements from a Deque

In C++, the deque container allows efficient removal operations from both the front and back of the queue. The `pop_front()` function is used to remove the element at the front, while the `pop_back()` function is used to remove the element at the rear. These functions permanently delete the respective elements without returning them.

Example of Removing Elements from a Deque

```

#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<string> dq{"Ravan", "SitaRam", "Ram", "Radhe Radhe", "Shiv",
    "ShivShakti", "Jalandhar"};
    cout << "Before delete, Element of deque are:- ";
    for(const string& str: dq)
        cout << str << " ";
    cout << endl;
    dq.pop_front();
    dq.pop_back();
    cout << "After delete, Element of deque are:- ";
    for(const string& str: dq)
        cout << str << " ";
    cout << endl;
    return 0;
}

```

Output: ./deque_deleting

Before delete, Element of deque are:- Ravan SitaRam Ram Radhe Radhe Shiv ShivShakti Jalandhar

After delete, Element of deque are:- SitaRam Ram Radhe Radhe Shiv ShivShakti

5) Deque Size

In C++, the `size()` method returns the number of elements currently contained in a deque container. It is a constant-time operation that helps in queue size monitoring, which is useful for logic control, iteration, and status reporting in real-world applications.

Example of Deque Size

```

#include <iostream>
#include <deque>
using namespace std;

```

```
int main()
{
    deque<int> dq{1, 2, 3, 4, 5};
    cout << "deque (dq) size():- " << dq.size() << endl;
    return 0;
}
```

Output: ./deque_size
 deque (dq) size():- 5

6) Check if a Deque is Empty

In C++, the empty() method determines whether or not a deque container contains any elements. It returns true (1) if the deque is empty and false (0) if it contains at least one element. It is useful in real-time scenarios where operations, such as task processing, user input response, and queue management, should only occur if the container contains data.

C++ Example to Check if a Deque is Empty

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq;
    if(dq.empty())
        cout << "deque (dq) is Empty.\n";
    else
        cout << "deque (dq) is not Empty.\n";
    return 0;
}
```

Output: ./deque_empty
 deque (dq) is Empty.

7) Looping Through a Deque

A loop is used to retrieve and process each element in a deque. The index-based loop allows for position-based access and is appropriate when element positions are critical. The range-based for-each loop (added in C++11) is more readable and useful when the attention is only on the elements themselves.

Example of looping through a Deque

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9, 11};
    cout << "Using index size loop based:- ";
    for(int i=0;i<dq.size();i++)
        cout << dq[i] << " ";
    cout << endl;
    cout << "Using for each loop based:- ";
    for(auto& x : dq)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_looping
 Using index size loop based:- 1 3 5 7 9 11
 Using for each loop based:- 1 3 5 7 9 11

C++ Deque Functions

C++ Deque assign()

C++ Deque assign() function assigns new contents to the deque container and the size of the container is modified accordingly.

Syntax

```
void assign(InputIterator first, InputIterator last);  
void assign(int n,value_type val);
```

Parameter

(first,last): It defines the range between which the new elements to be inserted.

n: It defines the new size of the deque container.

val: New value to be inserted.

Return value

It does not return any value.

Example

```
#include <iostream>  
#include <deque>  
using namespace std;  
int main()  
{  
    deque<int> dq1{1, 2, 3, 4, 5};  
    deque<int> dq2;  
    deque<int> ::iterator itr=dq2.begin();  
    dq2.assign(dq1.begin(), dq1.end());  
    cout << "Elements of the dq2 are:- ";  
    for(itr=dq2.begin();itr!=dq2.end();itr++)  
        cout << *itr << " ";  
    cout << endl;  
    return 0;  
}
```

Output: ./deque_assign

Elements of the dq2 are:- 1 2 3 4 5

C++ Deque emplace()

C++ Deque emplace() function inserts a new element just before the specified position and the size of the container is increased by one.

Syntax

```
iterator emplace(const_iterator position,value_type val);
```

Parameter

position: It defines the position before which the new element is to be inserted.

val: New value which is to be inserted.

Return value

It returns an iterator to the newly constructed element.

Example

```
#include <iostream>  
#include <deque>  
using namespace std;  
int main()  
{  
    deque<int> dq{1, 3, 5, 7, 9, 11};  
    dq.emplace(dq.begin(), 23);  
    cout << "Elements of dq:- ";  
    for(auto& x : dq)  
        cout << x << " ";  
    cout << endl;  
    return 0;  
}
```

Output: ./deque_emplace

Elements of dq:- 23 1 3 5 7 9 11

C++ **emplace_back()**

C++ Deque `emplace_back()` function adds a new element at the end of the deque and the size of the container is increased by one.

Syntax

```
void emplace_back(value_type val);
```

Parameter

val: New value to be inserted at the end of the deque.

Return value

It does not return any value.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 2, 3, 4, 5};
    dq.emplace_back(13);
    for(auto& x : dq)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_emplace_back

1 2 3 4 5 13

C++ **Deque emplace_front()**

C++ Deque `emplace_front()` function adds a new element to the beginning of the deque container and the size of the container is increased by one.

Syntax

```
void emplace_front(value_type val);
```

Parameter

val: New value is to be inserted at the beginning of the deque.

Return value

It does not return any value.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    dq.emplace_front(13);
    for(auto& a : dq)
        cout << a << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_emplace_front

13 1 3 5 7 9

C++ **Deque insert()**

C++ Deque `insert()` function inserts new element just before the specified position `pos` and the size of the container increases by the number of elements are inserted.

Insertion of an element can be done either from front or from the back.

Syntax

```
iterator insert(iterator pos, value_type val);
void insert(iterator pos, int n, value_type val);
```

```
void insert(iterator pos, InputIterator first, InputIterator last);
```

Parameter

pos: Position before which the new element is to be inserted.

val: New value to be inserted.

n: Number of times the value to be inserted.

(first,last): It defines the range of the elements which are to be inserted.

Return value

It returns an iterator to the newly constructed element.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7};
    deque<int> ::iterator it=dq.begin();
    dq.insert(it, 9);
    it = it+3;
    dq.insert(it, 2, 11);
    for(auto& x: dq)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_insert

9 1 3 5 11 11 7

C++ Deque push_back()

C++ Deque push_back() function adds a new element at the end of the deque container and the size of the container is increased by one.

Syntax

```
void push_back(value_type val);
```

Parameter

val: New value to be inserted at the end of the deque container.

Return value

It does not return any value.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 2, 3, 4, 5};
    dq.push_back(23);
    for(auto& x:dq)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_push_back

1 2 3 4 5 23

C++ Deque push_front() Function

In the C++ programming language, the deque (double-ended queue) is one of the most important features across various Standard Template Library (STL) containers. It allows the programmer to work with elements at either end of the container. The deque has the benefit of fast insertions and deletions at both ends of the container. The

most common STL function used to perform an insertion at the front of the deque is the `push_front()` member function.

In C++, the Deque's `push_front()` function inserts a new element at the beginning of the deque container, and the container's size increases by one. The `push_front()` member function provides efficient addition of elements at the front of a **deque container**. It allows the coder to efficiently implement a deque in cases where queues, stacks, sliding windows, or some form of history tracking is to be simulated.

Syntax

```
void push_front(value_type val);
```

- **val:** New value to be added in the beginning.
- **Return value:** It does not return any value.

C++ Deque `push_front()` Simple Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{3, 5, 7, 9, 11};
    dq.push_front(1);
    for(auto& x: dq)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: `./deque_push_front`

`1 3 5 7 9 11`

When to use the `push_front()` function in C++?

- We need the addition of newly created elements to the beginning of a sequence.
- We create data structures like double-ended queues, deques, or stacks.
- We need a way to prioritize new elements by adding them to the beginning of the sequence.
- We want to insert in reverse chronological order.
- We want to create sliding window algorithms or buffer data.

Real-World Examples

- It can be used to persist the most recent user actions at the beginning.
- If we want to visit a new page, it is added to the front.
- It is very useful when we need to insert and remove elements from both ends.

Conclusion

In conclusion, the C++ `deque::push_front()` function is commonly utilized to insert an element at the beginning of the deque. It effectively includes new elements to the front of the container without affecting existing elements, which makes it useful when frequent front insertions are required.

C++ `deque pop_back()` Function

In the C++ programming language, the `deque::pop_back()` method is a function shown by the deque (double-ended queue) class. The `pop_back()` method is commonly utilized to remove the value of the object from the last element of the deque. It allows the insertion and removal of objects from either end of the deque in constant time, $O(1)$. It enables us to manipulate the nested elements of a data structure effectively.

In C++, the deque's `pop_back()` function is commonly used to reduce the size of the object by one, but it does not return the type of the last deleted member. It can be used to delete the last element of a certain **deque** container. When we invoke the `pop_front()` function on an empty deque, it may produce undefined behavior. Therefore, it is also suitable to use the `empty()` function to ensure that the deque is complete.

Syntax:

```
deque_name.pop_back();
```

Parameters:

The pop_back() function does not take any parameters.

Return Value:

This function does not return any value (void return type).

C++ Simple deque pop_back() function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9, 11};
    cout << "Initial dq elements are:- ";
    for(auto& x : dq)
        cout << x << " ";
    cout << endl;
    dq.pop_back();
    cout << "After pop_back(), dq elements are:- ";
    for(auto& x : dq)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_pop_back

Initial dq elements are:- 1 3 5 7 9 11

After pop_back(), dq elements are:- 1 3 5 7 9

Features of the deque pop_back() function in C++

- In C++, the deque pop_back() function is commonly utilized to remove the elements that are present at the end of the deque container.
- When we call the deque pop_back() function in the deque container, it always reduces the size of the deque by deleting one element from the container.
- This function allows us to modify the container by removing the last element from the deque container.
- The deque pop_back() function does not throw any exceptions in a C++ program. However, if we call this function in the empty deque, it can lead to undefined behavior.
- When we perform any operation using the deque pop_back() function in C++, it can take only constant time, i.e., $O(1)$.

Conclusion:

In conclusion, the deque pop_back() function in C++ provides a simple and effective method to remove the last element of a double-ended deque. The deque provides a flexible management of elements that allows us to add or remove elements from both the front and the back of the deque.

If we want to perform any operation using the deque's pop_back() function in a C++ program, it can lead to undefined behavior. Overall, the pop_back() is an easy method to safely and quickly remove or eliminate an element from the back of a C++ deque.

C++ Deque pop_front()

C++ Deque pop_front() function removes the first element from the deque and the size of the container is reduced by one.

Syntax

```
void pop_front();
```

Parameter

It does not contain any parameter.

Return value

It does not return any value.

Example

```
#include <iostream>
#include <string.h>
#include <deque>
using namespace std;
int main()
{
    deque<string> dq{"Rakshas", "Shiv Ram", "Raja Ram", "Dev", "Shree",
    "Mahadev", "Dhara"};
    cout << "Initial dq elements are:- ";
    for(auto& x : dq)
        cout << x << " ";
    cout << endl;
    dq.pop_front();
    cout << "After pop_front() dq elements are:- ";
    for(auto& x : dq)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_pop_front

Initial dq elements are:- Rakshas Shiv Ram Raja Ram Dev Shree Mahadev Dhara

After pop_front() dq elements are:- Shiv Ram Raja Ram Dev Shree Mahadev Dhara

C++ Deque swap() Function

In the C++ programming language, the double-ended queues (deque) are a sequence container that allows us to insert and delete elements at both the beginning and the end. Unlike vectors, deques don't ensure contiguous storage allocation, but they enable fast access to the elements via random access iterators.

In C++, the deque swap() function is a built-in member function that is present in the **Standard Template Library (STL)**. It is commonly utilized to exchange the content of one deque with another deque that is passed in the same size. After the completion of the swapping operation, the two queues exchange their elements.

Syntax

```
void swap(deque& second);
```

Or

```
<Deque1>.swap(<deque2>)
```

- **second:** It is another deque container whose content is to be swapped with the given deque.
- **Deque1 & Deque2:** It is the deque names that need to be swapped with deque2.

Return value: It does not return any value.

C++ Simple deque swap() function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> odd{1, 3, 5, 7, 9, 11};
    deque<int> even{0, 2, 4, 6, 8, 10};
    cout << "Initial odd deque elements are:- ";
    for(auto& x : odd)
        cout << x << " ";
    cout << endl;
    cout << "Initial even deque elements are:- ";
    for(auto& x : even)
        cout << x << " ";
    cout << endl;
```

```

odd.swap(even);
cout << "After odd.swap(even) odd deque elements are:- ";
for(auto& x : odd)
    cout << x << " ";
cout << endl;
cout << "After odd.swap(even) even deque elements are:- ";
for(auto& x : even)
    cout << x << " ";
cout << endl;

swap(odd, even);
cout << "After swap(odd, even) odd deque elements are:- ";
for(auto& x : odd)
    cout << x << " ";
cout << endl;
cout << "After swap(odd, even) even deque elements are:- ";
for(auto& x : even)
    cout << x << " ";
cout << endl;

return 0;
}

```

Output: ./deque_swap

Initial odd deque elements are:- 1 3 5 7 9 11

Initial even deque elements are:- 0 2 4 6 8 10

After odd.swap(even) odd deque elements are:- 0 2 4 6 8 10

After odd.swap(even) even deque elements are:- 1 3 5 7 9 11

After swap(odd, even) odd deque elements are:- 1 3 5 7 9 11

After swap(odd, even) even deque elements are:- 0 2 4 6 8 10

Key Features of the Deque swap() function in C++

- The deque swap() function performs its operation in constant time ($O(1)$) because it only exchanges the internal pointers but doesn't exchange separate elements.
- In C++, we can also utilize the swap() function that is found in the <utility> and <algorithm> headers. It internally calls the member version.
- It is a part of the <deque> header file in C++.

Time Complexity Analysis of the Deque swap() Function

The time complexity of the deque swap() function has an $O(1)$ complexity, which is a constant time. The reason behind the constant complexity is that a deque uses an internal array, which is a map of fixed-size blocks. Swapping of the deques only exchanges the map pointers, metadata, and allocator state, which has a fixed number of operations independent of the number of elements.

Conclusion

In conclusion, the C++ deque::swap() function is commonly utilized to efficiently exchange the contents of two deques of the same type. It executes in constant time by swapping internal pointers rather than individual elements, which helps to make it a fast and resource-friendly operation. It is very useful when we want to quickly exchange data between two deques without manually copying the deque elements.

C++ deque clear() Function

In the C++ programming language, the deque clear() function is a predefined member function of the deque, i.e., double-ended queue. It can help to remove all the elements from the deque. Once we call the clear() function, the deque becomes empty and size has the value zero, and its capacity may remain unchanged. It is helpful to efficiently reset the queue without removing the complete object, and thus it can be reused.

In C++, the function certainly destroys all elements properly and frees the memory occupied by them, but the deque may have new elements placed in it without having to reallocate memory at once. It is clearly used whenever we need to reuse the same deque in a program to keep fresh data in it without constructing a new one.

Syntax:

```
deque_name.clear();
```

Parameters:

- The clear() function does not take any parameters.
- It simply removes all the elements present in the deque.

Return type:

- It does not return any value (return type void).
- After execution, the deque becomes empty. (size becomes 0).

C++ Simple deque clear() function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};

    cout << "Initial dq size:- " << dq.size() << " & elements are:- ";
    for(auto& a:dq)
        cout << a << " ";
    cout << endl;
    dq.clear();
    cout << "dq size after clear() is:- " << dq.size() << endl;
    return 0;
}
```

Output: ./deque_clear

Initial dq size:- 5 & elements are:- 1 3 5 7 9

dq size after clear() is:- 0

Features of the deque clear() function in C++

- In the C++ programming language, the deque clear() function is commonly utilized to remove all the elements from the deque until it becomes zero or empty.
- This function is very helpful if we want to reset or reinitialize the deque without reallocating the memory in the deque.
- This function doesn't return any value in C++. The return type of the deque clear() function is void.
- It takes an average time complexity ($O(n)$) because it needs to delete each and every element separately from the deque.
- It doesn't destroy the deque object itself, which enables it to be reused after clearing the element.

Conclusion

In conclusion, the C++ deque clear() function is a member function of the deque container that is a part of STL. It gives a simple way to clear all of the elements of a deque container. This function resets the deque size to zero, which allows us to perform new insertion operation. However, the capacity limit of the deque may remain unchanged. It gives a simple method to reuse the current deque without making a new deque. It is very helpful in those cases where we need to refresh or reset the data frequently.

C++ Deque empty()

C++ Deque empty() function checks whether the container is empty or not. If the container is empty, it returns 1 otherwise 0.

Syntax

```
void empty()
```

Parameter

It does not contain any parameter.

Return value

It does not return any value.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq;
    if(dq.empty())
        cout << "deque (dq) is Empty.\n";
    else
        cout << "deque (dq) is not Empty.\n";
    return 0;
}
Output: ./deque_empty
deque (dq) is Empty.
```

C++ Deque erase()

C++ Deque erase() function removes the element from the specified position or range and this effectively reduces the size of the deque by the number of elements removed.

Syntax

```
iterator erase(iterator pos);
iterator erase(iterator first,iterator last);
```

Parameter

pos: It defines the position at which the element to be removed from the deque.

(first,last): It defines the range within deque between which the elements to be removed.

Return value

It returns an iterator pointing to the element that followed the last element removed by the function.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 2, 3, 4, 5, 6, 7, 8, 9};
    cout << "Initial dq elements are:- ";
    for(auto& a: dq)
        cout << a << " ";
    cout << endl;
    dq.erase(dq.begin()+2);
    cout << "After position remove dq.begin()+2 dq elements are:- ";
    for(auto& a: dq)
        cout << a << " ";
    cout << endl;
    dq.erase(dq.begin()+2, dq.begin()+5);
    cout << "After range remove from dq.begin()+2 to dq.begin()+5 dq elements are:- ";
    for(auto& a: dq)
        cout << a << " ";
    cout << endl;
    return 0;
}
```

```
}
```

Output: ./deque_erase

Initial dq elements are:- 1 2 3 4 5 6 7 8 9

After position remove dq.begin()+2 dq elements are:- 1 2 4 5 6 7 8 9

After range remove from dq.begin()+2 to dq.begin()+5 dq elements are:- 1 2 7 8 9

C++ Deque max_size()

C++ Deque max_size() function is a built in function that returns the maximum number of elements that deque container can hold.

Syntax

```
return_type max_size();
```

here, return_type is a unsigned integral type.

Parameter

It does not contain any parameter.

Return value

It returns the maximum size of the deque.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    cout << "Maximum size of dq is:- " << dq.max_size() << endl;
    return 0;
}
```

Output: ./deque_max_size

Maximum size of dq is:- 2305843009213693951

C++ Deque resize() Function

The C++ Deque resize() function is an inbuilt function that is present in the Standard Template Library (STL) of C++. This function basically changes the size of the deque container to the size given in the argument based on a few conditions.

In the **C++ programming language**, the deque::resize() function is commonly utilized to resize the container to a specified number of elements (n). If the elements (n) is greater than the container size, the container size is extended to n elements by inserting new elements at the extended space. If the number of elements (n) is smaller than the container size, the container is reduced to n elements and removes all the elements beyond n.

Where n represents a new size of the container given in the argument.

Syntax

```
deque<T>.resize(size_type n); // Resize to n, default-initialized
deque<T>.resize(size_type n, const T& val); // Resize to n, filled with val
```

- **n**: It represents a new container size, and it is a mandatory parameter.
- **val**: It is represented as a new value to be added in the extended space.

Return value: It does not return any value.

Resizing a Deque to a Smaller Size Using resize() in C++

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5};
    dq.resize(5, 7);
    for(auto& a:dq)
        cout << a << " ";
    cout << endl;
    return 0;
}
```

```
}  
Output: ./deque_resize  
1 3 5 7 7
```

C++ Deque shrink_to_fit()

C++ Deque shrink_to_fit() function reduces the memory usage to fit the size of the container. This function does not modify the content of a deque.

Syntax

```
void shrink_to_fit();
```

Parameter

It does not contain any parameter.

Return value

It does not return any value.

Example

```
#include <iostream>  
#include <deque>  
using namespace std;  
int main()  
{  
    deque<int> dq{1, 3, 5, 7, 9};  
    cout << "Size of dq is:- " << dq.size() << endl;  
    dq.resize(13);  
    cout << "Size of dq after shrink_to_fit():- " << dq.size() << endl;  
    return 0;  
}  
Output: ./deque_shrink_to_fit  
Size of dq is:- 5  
Size of dq after shrink_to_fit():- 13
```

C++ Deque size()

C++ Deque size() function determines the number of elements present in the deque container.

For example:

```
deque<int> d={1,2,3,4,5};  
d.size()=5;
```

Syntax

```
return_type size();
```

where, return_type is an unsigned integral type.

Parameter

It does not contain any parameter.

Return value

It returns the number of elements in the deque container.

```
Example  
#include <iostream>  
#include <deque>  
using namespace std;  
int main()  
{  
    deque<int> dq{1, 2, 3, 4, 5};  
    cout << "deque (dq) size():- " << dq.size() << endl;  
    return 0;  
}  
Output: ./deque_size  
deque (dq) size():- 5
```

C++ Deque at() Function

In the C++ programming language, the deque (double-ended queue) is a container class that is a part of the Standard Template Library (STL). It allows fast insertion and deletion of elements at both the beginning and the end of the deque.

In C++, the Deque at() function is commonly utilized to access the element at a specified position. It allows us to perform both read and write operations. It is also helpful to access the elements from the **deque** with boundary checking.

Note: If the pos is greater than the size of the container, the function throws an exception, i.e, "out of range"

Syntax

reference at(size_type pos);

const_reference at(size_type pos) const; //using const

- **pos:** It is used to define the position of an element, which is to be returned.
- **Size_type:** It is used to represent an unsigned integral type.

Return value

It returns the reference of the specified element.

Simple C++ deque at() function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{10, 20, 30, 40, 50};
    dq.at(1) = 70;
    dq.at(4) += 100;
    for(auto& a:dq)
        cout << a << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_at
10 70 30 40 150

Features of the deque at() function in C++

- The deque at() function in C++ is commonly used to access the elements from the deque with boundary checking, which helps to avoid invalid index access.
- When we perform the operations to access the elements using the deque at() function, it only takes a constant time complexity of O(1).
- If we want to access the invalid index using this function, it throws an out_of_range exception instead of terminating the entire program.
- It enables both const and non-const deque, which allows us to use this function in read-only and modifiable contexts.
- This function is defined in the <deque> header file, which comes under the std namespace.

Conclusion

In conclusion, the deque::at() function is a member function of the deque container, which is part of the STL. It gives a simple and effective method to access the elements that are stored in the deque container. It allows us to perform boundary checking that makes sure the specified index is within the valid limits. It avoids unintentional access to invalid memory locations.

In C++, the deque at() function is commonly helpful in those cases where data safety and program stability are necessary. However, it is slower than the subscript operator ([]) because of the boundary checking. Overall, it helps to improve the code security and ensure secure access to the deque elements.

C++ Deque operator[]()

C++ Deque operator[] function is used to access the element at specified position pos. If position pos is greater than the size of container then it returns a value 0.

Difference between operator[]() & at()

When position pos is greater than the size of the container then operator[]() function returns a value 0 while at() function throws an exception i.e out of range.

Syntax

reference operator[] (int pos);

Parameter

pos: It defines the position of an element which is to be accessed.

Return value

It returns a reference to an element at position pos in the deque container.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<string> dq{"This", "is", "the", "code", "for", "testing", "."};
    for(int i=0;i<dq.size();i++)
        cout << dq.operator[](i) << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_operator

This is the code for testing .

C++ Deque operator=() Function

In the C++ programming language, the Standard Template Library (STL) offers a wide range of container classes, and within the set of container classes. The deque (double-ended queue) is one of the most flexible containers. Besides its flexibility, the deque's performance is efficient in the insertion and deletion of an element at both ends, which makes the deque a powerful alternative to the vector in some implementations.

In C++, the Deque operator=() function is one of the most essential functions in the deque container. It allows us to assign the content of one deque to another deque. It assigns new contents to the container and replaces the current content of the same type. The size of the deque can be modified accordingly. It is important for container assignment semantics, which enables the deep copying of efficient move semantics for deque objects.

Syntax:

```
deque& operator=(const deque& other); // Copy assignment
deque& operator=(deque&& other) noexcept; // Move assignment (C++11)
deque& operator=(initializer_list<T> ilist); // Initializer list assignment (C++11)
```

- **other:** It is a deque container whose content is to be copied into another deque object.
- **ilist:** It is used to define an initializer list that contains elements to assign to the deque.

Return value:

It returns a reference to the current deque *this.

Types of Assignment with deque::operator=()

There are mainly three important types of the deque assignment operator. Some of them are as follows:

1. Copy Assignment
2. Move Assignment
3. Initializer List Assignment

Copy Assignment

In C++, copy assignment is commonly utilized to copy the contents of one deque to another deque. The size and contents of the destination deque are replaced. If changes are made to one deque, it does not affect the other.

Simple C++ Deque Operator =() Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
```

```

{
    deque<int> dq{1, 3, 5, 7, 9};
    deque<int> x;
    cout << "Size of deque(x) :- " << x.size() << endl;
    x.operator = (dq);
    cout << "After Copy Assignment operator, Size of deque(x) :- " << x.size() <<
endl;
    return 0;
}

```

Output: ./deque_copy_assignment

Size of deque(x) :- 0

After Copy Assignment operator, Size of deque(x) :- 5

Move Assignment

Move assignment in C++ is used to transfer resources from one deque to another without performing a deep copy. The original deque will be moved from and will be empty (but still valid). It is typically more efficient for temporary objects.

C++ Move Assignment Operator with deque operator=() Example

```

#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    deque<int> x;
    cout << "Size of deque(dq) :- " << dq.size() << endl;
    cout << "Size of deque(x) :- " << x.size() << endl;
    x = move(dq);
    cout << "After Move Assignment, Size of deque(dq) :- " << dq.size() << endl;
    cout << "After Move Assignment, Size of deque(x) :- " << x.size() << endl;
    return 0;
}

```

Output: ./deque_move_assignment

Size of deque(dq) :- 5

Size of deque(x) :- 0

After Move Assignment, Size of deque(dq) :- 0

After Move Assignment, Size of deque(x) :- 5

Initializer List Assignment

In C++, an initializer list assignment is commonly used to directly assign values from brace-enclosed lists with an initializer list ({}). It is a very nice way to assign known values in one step.

C++ Initializer List Assignment using deque operator =() Example

```

#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    cout << "dq elements before initailizer Assignment:- ";
    for(auto& x: dq)
        cout << x << " ";
    cout << endl;
    dq = {11, 13, 15, 17, 19};
    cout << "dq elements after initailizer Assignment:- ";
    for(auto& x: dq)
        cout << x << " ";
    cout << endl;
    return 0;
}

```

```
Output: ./deque_initializer_assignment
dq elements before initailizer Assignment:- 1 3 5 7 9
dq elements after initailizer Assignment:- 11 13 15 17 19
```

Features of the Deque operator =() in C++

- In C++, it allows us to replace the content of one deque with another deque.
- This operator allows us to assign the values in various ways, such as: Copy Assignment, Move Assignment, and Initializer List assignment.
- It returns a reference to the current deque (*this), which enables chained assignments like `a = b = c;`.
- It provides better exception handling, which ensures that no data is missing if an exception occurs.
- When we use the deque operator=(), the available elements are automatically destroyed from the deque, and new elements are included as required.

Conclusion

In conclusion, the `deque::operator=()` function in C++ is a built-in member function in the double-ended queues container. It can be used to copy contents from a possible deque, move values without incurring the copy penalty, and quickly zero out values using an initializer list. It supports several methods to initialize a value in the deque: copy assignment, move assignment, and assignment using an initializer list.

C++ Deque back()

C++ Deque `back()` function is used to access the last element of the deque container.

Syntax

```
reference back();
```

Parameter

It does not contain any parameter.

Return value

It returns a reference to the last element of the deque container.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    cout << "Output by back():- " << dq.back() << endl;
    return 0;
}
```

```
Output: ./deque_back
```

```
Output by back():- 9
```

C++ Deque begin() Function

In the C++ programming language, the Standard Template Library (STL) has a number of containers that efficiently store and manipulate collections of data. The deque (double-ended queue) is one of these containers that can efficiently add and remove elements from both the front and the back of the container.

The `begin()` function is one of the simplest and most used member functions in any STL container, like deque. The `begin()` function will return an iterator referencing the 1st element in the container. In C++, the deque `begin()` function returns an iterator pointing to the first element of the deque container. If the container is empty, the returned iterator will be equal to the **end() function**. Therefore, it is one of the essential parts to be able to iterate, traverse, and manipulate the elements.

Syntax

```
iterator begin();
```

- **Parameter:** It does not contain any parameter.
- **Return value:** It returns an iterator pointing to the first element of the deque.

Time Complexity: The begin() function is O(1) in time complexity because it only has to return a pointer or iterator to the starting block of the deque.

C++ Simple Deque begin() Function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    cout << "dq begin element is:- " << *dq.begin() << endl;
    *dq.begin() = 11;
    cout << "After modify, dq begin element is:- " << *dq.begin() << endl;
    return 0;
}
```

Output: ./deque_begin

dq begin element is:- 1

After modify, dq begin element is:- 11

Features of the deque begin() function in C++

- It is commonly utilized to return an iterator that points to the first element of the deque container in C++.
- It is mainly executed in constant time, which means that it retrieves the iterator very effectively.
- The begin() function is also utilized with loops and STL algorithms that help to define the start position of the range in the deque.
- If we use the deque begin() function in C++, we can easily change the first element directly through dereferencing.
- If we use the empty deque(), it returns an iterator that is equal to the end() function, which helps to ensure proper safety during the iteration check.

Conclusion

In conclusion, the deque::begin() function is a simple but effective feature in C++. It will provide the user with an iterator to the first element of the deque, making it important for traversing, modifying, and passing to STL algorithms. Whether building efficient queue-based systems, implementing a sorting algorithm, or managing a buffer, understanding the effectiveness of begin() will help create safer and more efficient code.

C++ Deque cbegin()

C++ Deque cbegin() function returns a constant iterator pointing to the first element of the deque container. This iterator can be incremented or decremented just like the iterator returned by the begin() function. If the container is empty then the returned iterator will be equal to the cend().

Syntax

```
const_iterator cbegin();
```

Parameter

It does not contain any parameter.

Return value

It returns a constant iterator pointing to the beginning of the container.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    cout << "dq begin element is:- " << *dq.cbegin() << endl;
    *dq.begin() = 11;
    cout << "After modify, dq begin element is:- " << *dq.cbegin() << endl;
}
```



```

        return 0;
    }
    Output: ./deque_cbegin
    dq begin element is:- 1
    After modify, dq begin element is:- 11

```

C++ Deque end() Function

In the C++ programming language, the deque (double-ended queue) is a container class that is a part of the Standard Template Library (STL). It allows fast insertion and deletion of elements at both the beginning and the end of the deque.

In C++, the Deque end() function is commonly utilized to return an iterator that points to the "past-the-last" element of the deque. If the deque container is empty, the end() function returns the same as the begin() function. The "past-the-last" is the element that is used to follow the last element, and it marks the end boundary for forward iteration.

Syntax

```

    iterator end() noexcept;
    const_iterator end() const noexcept;

```

- It does not contain any parameters.

Return value: It returns an iterator referring to the past-the-last element. If the deque is constant, it also returns the const_iterator.

C++ Deque end() Simple Example

```

#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    auto it = dq.end();
    --it;
    cout << "Last element using end() :- " << *it << endl;
    cout << "Last element using end() :- " << *(dq.end()-1) << endl;
    return 0;
}

```

```

Output: ./deque_end
Last element using end() :- 9
Last element using end() :- 9

```

Features of the Deque end() Function in C++

- The deque end() function is commonly utilized with several iteration functions, such as a loop. It can also be utilized with the combination of the begin() function to traverse the whole deque.
- It does not point to a valid element in C++.
- It is very useful with several STL Algorithms, such as find(), sort, and many others, which need the range iterators.
- It helps to prevent out-of-bounds errors when we iterate through the deque container.
- It does not take any arguments, and it only returns the iterator position.
- It has a constant time complexity, i.e., $O(1)$.

Conclusion

In conclusion, the C++ deque end() function is a simple and effective method in the deque container. It is commonly utilized to return an iterator that is pointing to the "past-the-last" element of the deque. This function helps to define ranges for the iteration, modification, and algorithm operations. It is used in combination with the begin() function, and it can't be used alone in the program. The constant time complexity, exception safety, and support for const iterators of the end() function make it a reliable and effective function in C++.

C++ Deque cend() Function

The C++ Standard Template Library (STL) includes many containers, iterators, and algorithms to make programming easier. One of the most flexible containers is the deque (double-ended queue) because we can both insert and remove elements from both ends with reasonable complexity. The cend() function is an example of an iterator member function that is used when we want a constant iterator pointing to the end of a container. When using cend() with a deque, we can access the option to traverse elements in a read-only approach to the logical end.

In [C++](#), the deque cend() function returns a constant iterator pointing to the past-the-last element. An iterator can be incremented or decremented, but it cannot modify the content of a [deque](#). If the container is empty, the cend() function returns the same as the cbegin() function.

Syntax

```
const_iterator cend();
```

- **Parameter:** It does not contain any parameter.
- **Return value:** It returns a constant iterator referring to the past-the-last element in a deque.

C++ Simple deque cend() function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{1, 3, 5, 7, 9};
    auto it = dq.cend();
    --it;
    cout << "Last element using cend() :- " << *it << endl;
    cout << "Last element using cend() :- " << *(dq.cend()-1) << endl;
    return 0;
}
```

Output: ./deque_cend

Last element using cend() :- 9

Last element using cend() :- 9

Features of the deque cend() function in C++

- In the C++ programming language, the cend() function is commonly utilized to return a constant iterator that points to the end of the deque.
- It is also helpful with the mutable and const deques.
- It gives a better option to the end() function when alteration is not required.
- It provides const-correctness, which helps to avoid modification of elements in C++.
- It is commonly utilized with the cbegin() function for read-only traversal.

Conclusion

In conclusion, the C++ cend() function is a simple and powerful feature that provides safe and efficient traversal of a deque container without modifying its elements. It is especially helpful when we work with const deques or when we need to enforce read-only access.

If we combine the cbegin() and cend() functions, it provides a complete and const-correct traversal mechanism for C++ deques. The cend() function is often used with algorithms, such as find, count_if, all_of, etc., because it is safer and more expressive.

C++ Deque rbegin() Function

In the C++ programming language, the rbegin() function in C++ is a member function of the deque container. The rbegin() function returns a reverse iterator pointing to the last element of the [deque container](#).

In [C++](#), the rbegin() function allows us to traverse the [deque](#) backward, which starts from the end towards the beginning. It is especially useful when we want to process

elements in reverse order without actually reversing the container. It is declared in the `<deque>` header and is a part of the [C++ Standard Template Library \(STL\)](#).

Note: Reverse iterators are those that iterate from the back and move towards the beginning of the deque.

Syntax:

```
reverse_iterator rbegin();
```

```
const_reverse_iterator rbegin() const;
```

- **Parameter:** It does not contain any parameter.
- **Return value:** It returns a reverse iterator referring to the last element of a deque.

Simple C++ Deque rbegin() Function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<char> dq{'a', 'b', 'c', 'd', 'e', 'f'};
    cout << "Last element using rbegin():- " << *(dq.rbegin()) << endl;
    return 0;
}
```

Output: ./deque_rbegin

Last element using rbegin():- f

Features of the Deque rbegin() Function in C++

- The deque rbegin() function is commonly used to reverse an iterator that points to the last elements of the deque container.
- We can use this method to directly access the last element of the deque container without requiring the back() function.
- It is mainly utilized in several operations, such as reverse printing, backward searching, implementing the reverse-order logic, and many more.
- It enables iterating via the deque from the end to the beginning, which is very useful for the backward operations in C++.
- In C++, the reverse iterators support several operations, such as ++, --, and offset addition (rbegin() + n).
- The rbegin() function only gives access to deque elements. However, it doesn't change the deque itself.

Conclusion

In conclusion, the rbegin() function is a simple and essential method of the C++ deque container. We can use this function to access and traverse elements in reverse order. It returns a reverse iterator that points to the last element, which enables effective backward iteration without changing the container. The rbegin() function is commonly used for reverse printing and searching, which increases flexibility and improves code readability. If we include this function with the rend() function, it makes reverse traversal simple, safe, and effective.

C++ Deque crbegin()

C++ Deque crbegin() function returns a constant reverse iterator referring to the last element of the deque. An iterator can be incremented or decremented but it cannot modify the content of deque.

Where, crbegin() stands for constant reverse beginning.

Syntax

```
const_reverse_iterator crbegin();
```

Parameter

It does not contain any parameter.

Return value

It returns a constant reverse iterator pointing to the last element in the deque container.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{100, 200, 300, 400, 500};
    cout << "Last Element using crbegin():- " << *dq.crbegin() << endl;
    return 0;
}
```

Output: ./deque_crbegin

Last Element using crbegin():- 500

C++ Deque rend() Function

In the C++ programming language, the deque is a sequence container, which is a part of the Standard Template Library (STL). The deque enables us to insert or delete elements from both the front and back. The rend() function is the member function of the deque container, which provides access to elements held by that deque container. In [C++](#), the rend() function is commonly used to return a reverse iterator that points to the theoretical element that precedes the first element of the [deque](#). In this way, it can be used in conjunction with rbegin() to iterate or manipulate the deque in reverse order. It is mostly used in reverse iteration loops and algorithms involving reverse iterators, such as std::reverse, std::find, std::sort, and many other functions. The deque rend() function can be defined in the <deque> header file.

Syntax:

```
reverse_iterator rend();
const_reverse_iterator rend() const noexcept;
```

reverse_iterator rend();

- It is used to return a reverse iterator pointing to the element before the first element of the deque.
- It is commonly utilized with non-constant (modifiable) deques.

const_reverse_iterator rend() const noexcept;

- It is used to return a constant reverse iterator that refers to the same spot (before the first element).
- It is commonly utilized with constant deques where elements cannot be modified.

C++ Simple deque rend() Function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{11, 12, 23, 34, 45, 67, 78, 89};
    cout << "Elements of deque in reverse order using rend():- ";
    for(auto it=dq.rbegin();it!=dq.rend();it++)
        cout << *it << " ";
    cout << endl;
    return 0;
}
```

Output: ./deque_rend

Elements of deque in reverse order using rend():- 89 78 67 45 34 23 12 11

Features of the Deque rend() Function in C++

- In the C++ programming language, the deque rend() function is commonly utilized to return the reverse iterator pointing to the theoretical element before the first element of the deque.
- We can utilize the deque rend() function in C++ to iterate via a deque in the reverse order.

- It is commonly utilized in C++ to specify the end boundary of the reverse iteration range, which is similar to the `end()` function that does for forward iteration.
- It is a member function of the STL deque container, which is defined in the `<deque>` header file.
- We can use the `rend()` function with the [rbegin\(\) function](#) to iterate via a deque in the reverse order.
- If we want to perform any task, it takes only ($O(1)$) time complexity.

Conclusion

In conclusion, the C++ deque `rend()` is a member function of the deque container. The `rend()` function is commonly utilized to return a reverse iterator that points to the theoretical element just before the first element of the deque, for the purpose of reverse traversal of the deque container. It is often used along with `rbegin()` so that a programmer is able to iterate, search, or otherwise manipulate the elements of a deque from the back towards the front.

C++ Deque `crend()` Function

In the C++ programming language, the `deque::crend()` function is present in the deque container that is part of the Standard Template Library (STL). It is commonly used to return an iterator pointing to the element preceding the first element of the deque container. An iterator can be incremented or decremented, but it cannot modify the content of the deque. The iterator that the method returns is a constant reverse iterator.

Syntax

```
const_reverse_iterator crend();      Or
dequeName.crend();
```

- **Parameter:** It does not contain any parameter.
- **Return value:** It returns a constant reverse iterator referring to the element preceding the first element of the deque container.

C++ deque `crend()` Function Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<int> dq{10, 20, 30, 40, 50, 60, 70, 80, 90};
    cout << "Elements of deque in reverse order using crend():- ";
    for(auto it=dq.crbegin();it!=dq.crend();it++)
        cout << *it << " ";
    cout << endl;
    return 0;
}
```

Output: `./deque_crend`

Elements of deque in reverse order using `crend()`:- 90 80 70 60 50 40 30 20 10

Features of the Deque `crend()` function in C++

- The deque `crend()` function in C++ is commonly utilized to return a constant reverse iterator that points to the position before the first element of the deque container.
- In the C++ programming language, we cannot modify the elements if we access the elements using the deque `crend()` function. It helps to ensure the correctness and data safety of the program.
- We can traverse the elements using the deque `crend()` function because it returns a const iterator. However, we cannot change their values.
- The deque `crend()` function in C++ is generally utilized with the `crbegin()` function to perform the reverse traversal operation of the deque from end to beginning.

- If we want to perform any operation using the `deque::crend()` function in C++, it takes only constant time complexity, i.e., $O(1)$.
- In C++, the `deque::crend()` function doesn't take any argument. It just returns an iterator object.

Conclusion

In conclusion, the `deque::crend()` function in C++ is a simple and efficient feature of the Standard Template Library (STL) that enables read-only reverse traversal of a deque container. It is commonly utilized to return a constant reverse iterator that points to the position before the first element. It ensures that the elements cannot be modified while iterating.

In C++, it is generally utilized with the `crbegin()` function. It gives an efficient and secure method to access deque elements in reverse order, which helps to maintain the const-correctness and data integrity.

C++ Deque front()

The C++ **`deque::front()`** function returns a reference to the **first element** of the deque. You can both **read and modify** the element (if the deque is non-const).

If the deque is empty, calling `front()` results in **undefined behavior**, so it should be used only when the deque has elements.

Syntax

`T& front();`

`const T& front() const;`

- For **non-const deque**, it returns a reference (`T&`) that can be modified.
- For **const deque**, it returns a const reference (`const T&`) that cannot be modified.

Parameters

`front()` does **not take any parameters**.

Return value

- Returns a **reference** to the first element of the deque.
- If the deque is empty, accessing this reference is **undefined behavior**.

Example

```
#include <iostream>
#include <deque>
using namespace std;
int main()
{
    deque<string> dq{"Jai Mata Di", "Jai Shree Ram", "Har Har Mahadev", "Radhe Radhe"};
    cout << "First Element using front():- " << dq.front() << endl;
    return 0;
}
```

Output: `./deque_front`

First Element using `front()`:- Jai Mata Di

Notes

- `front()` always points to the **first element**, unlike `back()` which points to the **last element**.
- If you need a **read-only reference**, use a const deque.
- Use `front()` for quick access to the beginning without using iterators (`begin()` or `cbegin()`).

C++ List

In C++, the `std::list` is a sequence container that enables the non-contiguous storage of elements. It is implemented as a [doubly linked list](#), where every element contains the address of both the next and previous list elements. It allows us to store the data in non-contiguous memory, which provides fast insertion and deletion at any position if the position is known.

Unlike the [std::vector](#), which contains the elements in contiguous memory and enables fast random access, a list provides support for sequential access, which means the element should be accessed by traversing from the front and back.

The `std::list` supports bidirectional access and provides an efficient way for insertion and deletion operations. However, traversal is slow in the list because list elements are accessed sequentially, while a vector supports random access.

Syntax

```
list<type> list_name;
```

- **type:** It represents the type of elements in the list.
- **list_name:** It represents the name that is assigned to the list.

Initialization of List

```
list<int> new_list{1,2,3,4};          OR  
list<int> new_list = {1,2,3,4};
```

Basic Operations of C++ List

1) push_back() and push_front()

In C++, the `push_back()` function appends an element to the end, and the `push_front()` function inserts at the beginning. It is suitable for queue and deque-like behaviours.

C++ `push_back()` and `push_front()` Function Example

```
#include <iostream>  
#include <list>  
using namespace std;  
int main()  
{  
    list<int> l;  
    l.push_back(10);  
    l.push_back(30);  
    l.push_front(50);  
    auto it = l.begin();  
    advance(it, 1);  
    l.insert(it,15);  
    for(int x: l)  
        cout << x << " ";  
    cout << endl;  
    return 0;  
}
```

Output: ./list_create_insert
50 15 10 30

2) pop_front() and pop_back()

In C++, the `pop_front()` function is used to remove the first element, and the `pop_back()` function is used to remove the last element from the list. These functions are also used to shrink lists from either end.

C++ List Example using `pop_front()` and `pop_back()` function

```
#include <iostream>  
#include <list>  
using namespace std;  
void print(list<int> l)  
{  
    for(int x:l)  
        cout << x << " ";  
    cout << endl;
```

```

}
int main()
{
    list<int> l{1, 3, 5, 7, 9, 11, 13};
    print(l);
    l.pop_back();
    print(l);
    l.pop_front();
    print(l);
    return 0;
}

```

Output: ./list_pop_back-front

1 3 5 7 9 11 13

1 3 5 7 9 11

3 5 7 9 11

3) insert()

In C++, we can insert elements anywhere using iterators. It allows us to add new elements before a specific position.

C++ List Example using insert() function

```

#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{1, 3, 5, 9};
    auto it = l.begin();
    for(int i=0;i<3;i++)
        ++it;
    //advance(it, 2);
    l.insert(it, 7);
    for(auto x:l)
        cout << x << " ";
    cout << endl;
    return 0;
}

```

Output: ./list_insert

1 3 5 7 9

4) erase()

In C++, the erase() function is used to remove the element at a given position or range. After that, the iterator-based removal allows precise control.

C++ List Example using erase() function

```

#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{1, 2, 3, 4, 5, 6};
    auto it = l.begin();
    ++it;
    l.erase(it);
    auto it2 = l.end();
    --it2;
    l.erase(it2);
    for(auto x:l)
        cout << x << " ";
    cout << endl;
    return 0;
}

```


Output: ./list_erase

1 3 4 5

5) size() and empty()

In C++, the size() function is used to return the number of elements, and the empty() function checks if the list has none. It is very useful for condition checks and loops.

C++ List Example using size() and empty() Function

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{1, 3, 5};
    cout << "Size of the list(l) :- " << l.size() << endl;
    cout << "List is" << (l.empty() ? " empty." : " not empty.") << endl;
    return 0;
}
```

Output: ./list_size_empty

Size of the list(l) :- 3

List is not empty.

6) clear()

In C++, the clear() method is mainly used to remove all elements from the list, which makes the list empty. It is useful to reuse the list without reallocating.

C++ List Example using clear() function

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{1, 3, 5, 7, 9, 11};
    cout << "Before clear(), Size of list:- " << l.size() << endl;
    l.clear();
    cout << "After clear(), Size of list:- " << l.size() << endl;
    return 0;
}
```

Output: ./list_clear

Before clear(), Size of list:- 6

After clear(), Size of list:- 0

7) sort()

In C++, the sort() function is mainly used to sort the list in ascending order. After that, it works only for sortable data types.

C++ List Example using the sort() Function

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{2, 5, 1, 3, 6, 4, 9, 7};
    cout << "Before sort(), Element of list:- ";
    for(auto x:l)
        cout << x << " ";
    cout << endl;
    l.sort();
    cout << "After sort(), Element of list:- ";
    for(auto x:l)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

```
}
```

Output: ./list_sort

Before sort(), Element of list:- 2 5 1 3 6 4 9 7

After sort(), Element of list:- 1 2 3 4 5 6 7 9

9) reverse()

In C++, the reverse() function is used to reverse the order of elements in the list. It does not return anything but modifies in place.

C++ List Example using reverse() function

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{2, 5, 1, 3, 6, 4, 9, 7};
    cout << "Before reverse(), Element of list:- ";
    for(auto x:l)
        cout << x << " ";
    cout << endl;
    l.reverse();
    cout << "After reverse(), Element of list:- ";
    for(auto x:l)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./list_reverse

Before reverse(), Element of list:- 2 5 1 3 6 4 9 7

After reverse(), Element of list:- 7 9 4 6 3 1 5 2

9) remove(value)

In C++, the remove(value) function is used to remove all the elements that are equal to the specified value. It is an efficient way to remove duplicates or unwanted values.

C++ List Example using remove (Value) function

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{2, 5, 1, 3, 6, 4, 9, 7};
    cout << "Before remove(), Element of list:- ";
    for(auto x:l)
        cout << x << " ";
    cout << endl;
    l.remove(6);
    cout << "After remove(6), Element of list:- ";
    for(auto x:l)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./list_remove

Before remove(), Element of list:- 2 5 1 3 6 4 9 7

After remove(6), Element of list:- 2 5 1 3 4 9 7

10) max_size(), front() and back()

The **max_size()** function returns the maximum number of elements that a container can hold. It's important to note that this number is system-dependent, and while it may be large, it's not the absolute limit of the system's memory.

The **front()** function returns a reference to the first element in the container. It is useful for accessing or modifying the first element of the container.

The **back()** function returns a reference to the last element in the container. It can also be used to modify the last element.

Example

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{1, 2, 3};
    cout << "max_size() of list:- " << l.max_size() << endl;
    cout << "front element using front():- " << l.front() << endl;
    cout << "back element using back():- " << l.back() << endl;
    return 0;
}
```

Output: ./list_max-size_front_back
max_size() of list:- 384307168202282325
front element using front():- 1
back element using back():- 3

11) swap() and merge()

The **swap()** function swaps the contents of two containers of the same type. After calling **swap()**, both containers will have exchanged their elements.

The **merge()** function is used to merge two **sorted** containers of the same type, maintaining their sorted order. It merges elements from one container into another.

Example:

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l1{4, 5};
    list<int> l2{1, 2, 3};
    l1.swap(l2);
    cout << "List 1: ";
    for (auto x : l1) cout << x << " ";
    cout << endl;
    cout << "List 2: ";
    for (auto x : l2) cout << x << " ";
    cout << endl;
    l1.merge(l2);
    cout << "After merging:- ";
    for (auto x : l1) cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./list_swap_merge
List 1: 1 2 3
List 2: 4 5
After merging:- 1 2 3 4 5

12) splice(), unique(), resize()

The **splice()** function is used to transfer elements from one container to another. It can be used to insert elements at any position in a container or to move elements between containers. It's particularly useful for efficiently transferring entire lists.

The **unique()** function is used to remove consecutive duplicate elements in a container. It only removes adjacent duplicates, so if there are non-adjacent duplicates, they will remain in the container.

The **resize()** function changes the size of a container. If the new size is smaller, elements are removed from the end. If the new size is larger, default-constructed elements are added to the end (or you can specify a value to fill with).

Example:

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l1{1, 2, 2, 3, 3, 4, 5};
    list<int> l2{6, 7, 8, 9};
    l1.splice(l1.end(), l2);
    cout << "After splice():- ";
    for(auto x: l1) cout << x << " ";
    cout << endl;
    l1.unique();
    cout << "After unique():- ";
    for(auto x: l1) cout << x << " ";
    cout << endl;
    l1.resize(11, 10);
    cout << "After resize():- ";
    for(auto x: l1) cout << x << " ";
    cout << endl;
    return 0;
}
```

Output: ./list_splice_unique_resize

After splice():- 1 2 2 3 3 4 5 6 7 8 9

After unique():- 1 2 3 4 5 6 7 8 9

After resize():- 1 2 3 4 5 6 7 8 9 10 10

13) **assign()**, **emplace()**, **emplace_back()** and **emplace_front()**

The **assign()** function is used to assign new values to a container. It can either fill the container with a specific value or copy elements from another range (or container).

The **emplace()** function is used to **construct** an element in place at the specified position in the container. This is more efficient than **insert()** because it avoids copying or moving an element. Instead, the element is constructed directly in place with the provided arguments.

The **emplace_back()** function is used to construct a new element in place at the **end** of the container. It is similar to **push_back()**, but instead of copying or moving an element, it constructs the element directly at the end of the container.

The **emplace_front()** function is used to construct a new element in place at the **beginning** of the container. It works similarly to **emplace_back()**, but the element is inserted at the front of the container instead of the back.

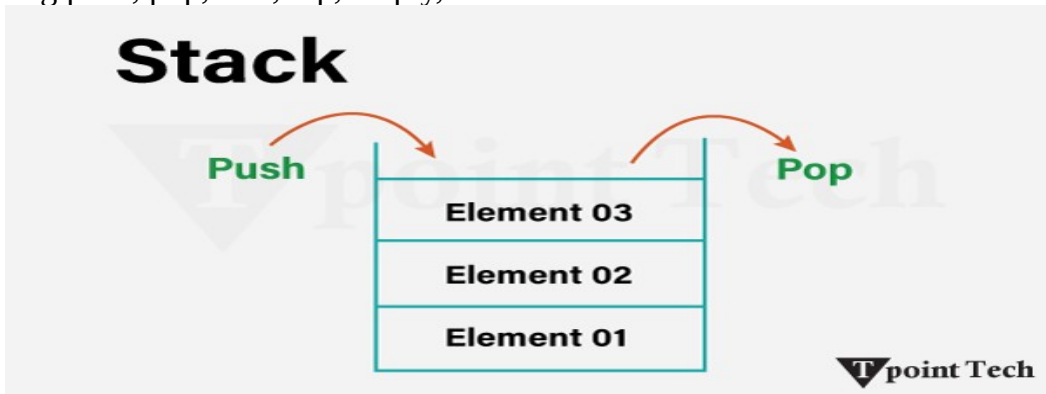
Example

```
#include <iostream>
#include <list>
using namespace std;
int main()
{
    list<int> l{1, 2, 3};
    auto it = l.begin();
    ++it;
    l.assign(5, 7);
    cout << "After assign(5, 7):- ";
    for(auto x:l) cout << x << " ";
    cout << endl;
    l.emplace(it, 15);
    cout << "After emplace(15):- ";
```

```
for(auto x:l) cout << x << " ";  
cout << endl;  
l.emplace_front(25);  
cout << "After emplace_front(25):- ";  
for(auto x:l) cout << x << " ";  
cout << endl;  
l.emplace_back(35);  
cout << "After emplace_back(35):- ";  
for(auto x:l) cout << x << " ";  
cout << endl;  
return 0;  
}  
Output: ./list_assign_emplace_emplace-back_emplace-front  
After assign(5, 7):- 7 7 7 7 7  
After emplace(15):- 7 15 7 7 7 7  
After emplace_front(25):- 25 7 15 7 7 7 7  
After emplace_back(35):- 25 7 15 7 7 7 7 35
```

C++ Stack

In C++, a **stack** is a linear data structure that performs its operations on the Last In First Out (LIFO) principle. It means that the element added last will be removed first and the first added element removed last. In C++, the STL offers a stack container adaptor to perform LIFO operations efficiently. It contains all the standard features, including push, pop, size, top, empty, etc.



Stack elements cannot be accessed by the index numbers. As the elements are added and removed from the top, we can only access the elements at the top of the stack.

Syntax

```
template<class T, class Container = deque<T> > class stack;
```

- **T:** The argument specifies the type of the element, which the container adaptor will be holding.
- **Container:** The argument specifies an internal object of the container where the elements of the stack are held.

C++ Stack Example

```
#include <iostream>
#include <stack>      //using the stack container from STL
using namespace std; // Using standard namespace
int main()           // Main function
{
    stack<int> st;    // Declare a integer stack
    // Push elements into the stack (LIFO order)
    st.push(15);
    st.push(20);
    st.push(25);
    // Display the top element of the stack
    cout << "Top element of the list: " << st.top() << endl;
    // Pop the top element from the stack
    st.pop();
    // Display the new top element
    cout << "After pop, top element: " << st.top() << endl;
    // Display the size of the stack
    cout << "Size of Stack: " << st.size() << endl;
    // Check if the stack is empty
    cout << "Is stack empty? " << (st.empty() ? "Yes" : "No") << endl;
    return 0;
}
```

Output: ./stack_example
Top element of the list: 25
After pop, top element: 20
Size of Stack: 2
Is stack empty? No

Basic Operations of C++ Stack

1) Inserting Elements

In a C++ stack, we can easily insert the new elements at the top of the stack using the push() method. Insertion of an element into any other place in the stack is not possible.

Syntax

```
st.push(data_type)
```

C++ Inserting Elements Example

```
#include <iostream>
#include <stack>
using namespace std;
int main()
{
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);
    while(!st.empty())
    {
        cout << st.top() << " ";
        st.pop();
    }
    cout << endl;
    return 0;
}
```

Output: ./stack_insert

30 20 10

2) Accessing Elements

In C++ Stack, we can only access the top element of the stack by using the top() method. We cannot access any middle element of the stack.

Syntax

```
int topElement = st.top();
```

C++ Accessing Elements Example

```
#include <iostream>
#include <stack>
using namespace std;
int main()
{
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);
    cout << "top element:- " << st.top() << endl;
    return 0;
}
```

Output: ./stack_accessing

top element:- 30

3) Deleting Elements

In a single operation, only the last element of the stack can be removed by using the pop() method. If we want to remove any other element, we need to first remove all the elements that have been added after that element.

Syntax

```
st.pop();
```

C++ Deleting Elements Example

```
#include <iostream>
#include <stack>
using namespace std;
```

```

int main()
{
    stack<int> st;
    st.push(1);
    st.push(2);
    st.push(3);
    st.push(4);
    st.push(5);
    cout << "Before pop(), top element :- " << st.top() << endl;
    st.pop();
    cout << "Before pop(), top element :- " << st.top() << endl;
    return 0;
}

```

Output: ./stack_delete

Before pop(), top element :- 5

Before pop(), top element :- 4

4) Pseudo Traversal

As any element apart from the top element cannot be accessible in the stack, it cannot be traversed. However, we can make a copy of it, access the top element and remove the top. Doing it up to the point when the copy stack is emptied, we can traverse effectively without changing the original stack.

Syntax

```

stack<int> temp = st;
while (!temp.empty()) {
    cout << temp.top() << " ";
    temp.pop();
}

```

C++ Pseudo Traversal Example

```

#include <iostream>
#include <stack>
using namespace std;
int main()
{
    stack<int> st;
    st.push(10);
    st.push(20);
    st.push(30);
    st.push(40);
    st.push(50);
    //stack<int> temp = st;
    stack<int> temp(st);
    while(!temp.empty())
    {
        cout << temp.top() << " ";
        temp.pop();
    }
    cout << endl;
    return 0;
}

```

Output: ./stack_pseudo_traversal

50 40 30 20 10

5) Check if the Stack Is Empty

In C++ Stack, we utilize the empty() method to test whether the stack is empty or not. The empty() method returns:

- 1 (true) - if the stack is empty
- 0 (false) - when the stack is not empty

Syntax

```
if (st.empty()) { /* stack is empty */ }
```

C++ Example to check if the Stack is Empty

```
#include <iostream>
#include <stack>
using namespace std;
int main()
{
    stack<int> st;
    cout << "Stack is " << (!st.empty() ? "not empty." : "empty.") << endl;
    st.push(10);
    cout << "After push(10), Stack is " << (!st.empty() ? "not empty." : "empty.") <<
endl;
    return 0;
}
```

Output: ./stack_empty

Stack is empty.

After push(10), Stack is not empty.

6) swap() and emplace()

Example

```
#include <iostream>
#include <stack>
using namespace std;
int main()
{
    stack<int> st1, st2;
    st1.push(1);
    st1.push(2);
    st1.push(3);
    st2.emplace(4);
    st2.emplace(5);
    st2.emplace(6);
    st1.swap(st2);
    cout << "Stack 1 (after swap): ";
    while (!st1.empty()) {
        cout << st1.top() << " ";
        st1.pop();
    }
    cout << endl;
    cout << "Stack 2 (after swap): ";
    while (!st2.empty()) {
        cout << st2.top() << " ";
        st2.pop();
    }
    cout << endl;
    return 0;
}
```

Output: ./stack_swap_emplace

Stack 1 (after swap): 6 5 4

Stack 2 (after swap): 3 2 1

Time Complexity

Operation	Time Complexity
Insert an element (push)	O(1)
Delete an element (pop)	O(1)
Access top element (peek)	O(1)
Traverse the stack	O(n)

Implement queue using stack in C++

Example

```
#include <iostream>
#include <stack>
using namespace std;
stack<int> st1, st2;
void enqueue(int x)
{
    st1.push(x);
}
int dequeue()
{
    if(st1.empty() && st2.empty())
    {
        cout << "Queue is empty." << endl;
        return -1;
    }
    if(st2.empty())
    {
        while(!st1.empty())
        {
            st2.push(st1.top());
            st1.pop();
        }
    }
    int val = st2.top();
    st2.pop();
    return val;
}
int front()
{
    if(st1.empty() && st2.empty())
    {
        cout << "Queue is empty." << endl;
        return -1;
    }
    if(st2.empty())
    {
        while(!st1.empty())
        {
            st2.push(st1.top());
            st1.pop();
        }
    }
    return st2.top();
}
int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
    cout << "Front() value of queue:- " << front() << endl;
    cout << "dequeue() value of queue:- " << dequeue() << endl;
    return 0;
}
```

Output: ./stack_to_queue1

Front() value of queue:- 10

dequeue() value of queue:- 10

C++ STL Set

In C++, a set represents the fundamental data structure that arranges the unique elements in some sorting order and stores them. The sets are known as the associative containers, and the unique sorted elements are known as the sorted keys. We can insert or delete these keys, but we cannot modify them.

By default, sets follow ascending order to sort the keys. Sets are part of the Standard Template Library (STL) available in C++, where in the STL, the set class offers useful components that upgrade the efficiency of the set operation. As the sets use the default sorting order for keys and do not use an indexing mechanism, we cannot access these keys with their index numbers.

Syntax

```
template < class T,    // set::key_type/value_type
          class Compare = less<T>,    // set::key_compare/value_compare
          class Alloc = allocator<T>    // set::allocator_type
          > class set;
```

Parameter:

- **T:** T represents the type of element stored in the container set.
- **Compare:** A comparison class that takes two arguments of the same type bool and returns a value. This argument is optional, and the binary predicate less<T> is the default value.
- **Alloc:** Type of the allocator object which is used to define the storage allocation model.

Declaration and Initialization of Set

In C++, we can declare the set using <set> header file and initialized with the std::set container.

Syntax

```
#include <set> //Required for using set
std::set<data_type> set_name;
```

C++ declaration and initialization of Set Example

```
#include <iostream>
#include <set>
using namespace std;
int main()
{
    set<int> s{2, 4, 1, 6, 3, 7, 5};
    for(auto& i:s)
        cout << i << " ";
    cout << endl;
    return 0;
}
Output: ./set_example
1 2 3 4 5 6 7
```

Operations of the Set

1) Insert Elements

In C++, the insert() function is used to insert elements in the set. The insert() or emplace() operations are used to insert the elements and if we try to insert an already existing element in the set, it will not get inserted. Moreover, we are not allowed to specify the insertion position as it is automatically taken care of at the time of sorting order.

C++ Insertion Elements Example

```
#include <iostream>
#include <set>
using namespace std;
int main()
{
```

```

    set<int> s{1, 5, 2, 0, 4};
    s.insert(10);
    s.emplace(8);
    s.insert(19);
    s.emplace(17);
    for(auto& i:s)
        cout << i << " ";
    cout << endl;
    return 0;
}

```

Output: ./set_insert

0 1 2 4 5 8 10 17 19

2) Accessing Elements

In the C++ set, we cannot directly access the elements using the index. The begin() and end() iterators are used to access the elements by their position. Along with these, the next() and advance() methods can also be used to access the set elements.

C++ Accessing Elements Example

```

#include <iostream>
#include <set>
using namespace std;
int main()
{
    set<int> s{1, 4, 6, 2, 7, 4, 9, 10};
    auto v1 = s.begin();
    auto v2 = next(s.begin(), 3);
    cout << *v1 << " " << *v2 << endl;
    return 0;
}

```

Output: ./set_accessing

1 6

3) Finding Elements

In C++, the find() function is used to find an element from the set. The find() function enables fast search by value operation, and if the searched element is present in the set, it will return the value, else it returns the end() iterator.

C++ Finding Elements Example

```

#include <iostream>
#include <set>
using namespace std;
int main()
{
    set<int> s{3, 1, 6, 4, 2, 8, 4};
    auto it = s.find(6);
    if(it != s.end())
        cout << *it << endl;
    else
        cout << "Element not found in the set.\n";
    return 0;
}

```

Output: ./set_find

6

4) Traversing Elements

In C++, the begin() and end() iterators are used to traverse or iterate the set with the help of for loops, which are based on range.

C++ Traversing Elements Example

```

#include <iostream>
#include <set>
using namespace std;

```

```

int main()
{
    set<int> s{1, 5, 2, 4, 9, 6, 3, 8, 7};
    for(auto i=s.begin();i!=s.end();i++)
        cout << *i << " ";
    cout << endl;
    return 0;
}

```

Output: ./set_traversing

1 2 3 4 5 6 7 8 9

5) Deleting Elements

In C++, the erase() method is used to remove elements from a set either by value or by position.

C++ Deleting Elements Example

```

#include <iostream>
#include <set>
using namespace std;
int main()
{
    set<int> s{10, 0, 9, 1, 8, 2, 5, 3, 8};
    s.erase(10);
    s.erase(s.begin());
    for(auto& x:s)
        cout << x << " ";
    cout << endl;
    return 0;
}

```

Output: ./set_deleting

1 2 3 5 8 9

Time Complexity for the Basic Operations

- **Insert:** The time complexity for inserting an element is $O(\log n)$.
- **Delete:** The time complexity for deleting an element is $O(\log n)$.
- **Find:** The time complexity for searching an element is $O(1)$.
- **Find by Value:** The time complexity for searching an element by value is $O(\log n)$.
- **Traverse:** The time complexity for traversing elements in a set is $O(n)$.

Types of Set

1) Ordered Set

In C++, the set that obeys the sorting order when storing elements in a set, these sets are known as the Ordered Sets.

2) Unordered Set

In C++, the set that stores the elements in an arbitrary order; such type of set is known as an Unordered set.

Difference between Ordered Set and Unordered Set

- An ordered set stores the elements in the set by following the sorting mechanism, which is ascending by default. On the other hand, an unordered set stores the elements in the set by following an arbitrary order.
- In an ordered set, the operations such as insert, delete, and find operations take the logarithmic $O(\log n)$ time complexity to perform the assigned tasks. On the other hand, the unordered set uses the hashing technique when performing any operations such as insert, delete, or accessing the elements, and so it takes $O(1)$ time complexity to perform the assigned tasks.

C++ Bitset

In C++, a bitset is a container that holds a fixed-size sequence of bits (0s and 1s). It enables us to manage and manipulate a fixed-size sequence of bits efficiently. It is useful for low-level operations, memory-efficient storage, and bitwise logic. It is implemented as a class template and defined in the <bitset> header file within the std namespace.

Syntax

```
std::bitset<N> b;
```

In this syntax, N is the number of bits that we want to store, and b is the name that is assigned.

Initialization of Bitset

In C++, we can initialize the bitset function in several ways based on their use cases. A bitset is a fixed-size sequence of N bits (such as 0's and 1's), where we can manipulate each bit individually. When we create a bitset, all bits are unset (0) by default. However, we may initialize it with definite values during its declaration. Several initializations of bitset in C++ are as follows:

1) Default Initialization (All Bits = 0)

```
std::bitset<7> b1;
```

Here, the bitset is of size 7 bits and

2) Initialization with an Unsigned long

We can also initialize a bitset with an unsigned long value. In the unsigned long, the binary representation of the number is stored from right to left.

```
std::bitset<8> b2(42);
```

3) Initialization using a Binary String

The string must have the same or fewer characters than the size of the bitset. Extra bits are truncated from the left.

```
std::bitset<8> b3("11001100");
```

C++ Bitset Example

```
#include <iostream>
#include <bitset>
using namespace std;

int main() {
    // CREATE
    bitset<8> bs; // 8-bit bitset initialized to 00000000
    cout << "Initial bitset: " << bs << endl;

    // Create / Insert (set bit)
    bs.set(2);
    bs.set(5);
    cout << "After setting bits 2 and 5: " << bs << endl;

    // READ
    cout << "Bit at position 2: " << bs.test(2) << endl;
    cout << "Bit at position 4: " << bs.test(4) << endl;
    cout << "Total set bits: " << bs.count() << endl;

    // UPDATE
    bs.set(4); // change 0 → 1
    bs.flip(5); // toggle bit
    cout << "After update: " << bs << endl;

    // DELETE
    bs.reset(2); // set bit to 0
```

```

    cout << "After deleting bit 2: " << bs << endl;

    // CLEAR ALL
    bs.reset();
    cout << "After clearing all bits: " << bs << endl;

    return 0;
}
Output: ./bitset_example
Initial bitset: 00000000
After setting bits 2 and 5: 00100100
Bit at position 2: 1
Bit at position 4: 0
Total set bits: 2
After update: 00010100
After deleting bit 2: 00010000
After clearing all bits: 00000000

```

Common Bitset Operations

Operation	Function
Create	bitset<N> bs
Set bit	bs.set(pos)
Read bit	bs.test(pos)
Update bit	bs.set(pos) / bs.flip(pos)
Delete bit	bs.reset(pos)
Clear all	bs.reset()
Count 1s	bs.count()

```

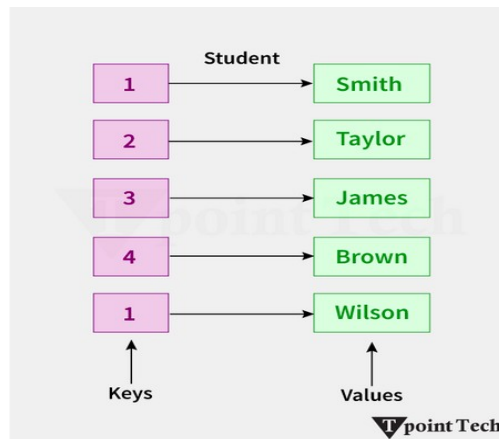
bs.any(); // true if any bit is 1
bs.none(); // true if all bits are 0
bs.all(); // true if all bits are 1
bs.flip(); // flip all bits

```

Function	Description
set(pos)	make bit 1
reset(pos)	make bit 0
flip(pos)	toggle bit
test(pos)	read bit
count()	count 1 bits
any()	true if any bit =1
none()	true if all bits =0
all()	true if all bits =1

C++ Map

In C++, maps are part of the STL (Standard Template Library). Maps are the associative containers that store sorted key-value pair, in which each key is unique and it can be inserted or deleted but cannot be altered. Values associated with keys can be changed.



Syntax

```
map<key_type, value_type, comp> t;
```

Parameter

- **Key_type:** The key data type to be stored in the map.
- **Value_type:** The data type of value to be stored in the map.
- **compare:** A comparison class that takes two arguments of the same type bool and returns a value. This argument is optional, and the binary predicate `less<"key">` is the default value.
- **t:** It represents the name assigned to the map.

C++ Algorithm Example

```
#include <iostream>
#include <map>
#include <string>
using namespace std;
int main()
{
    map<int, string> Employees;
    Employees[100] = "Sam";
    Employees[105] = "Samar";
    Employees.insert(pair<int, string>(102, "Bob"));
    Employees.insert(make_pair(103, "Eve"));
    Employees.emplace(104, "Charlie");
    Employees.insert_or_assign(101, "Maha"); //C++17 or Above
    map<int, string> newEmployees{{106, "Rass"}, {108, "Kali"}};
    Employees.insert(newEmployees.begin(), newEmployees.end());
    for(const auto& val : Employees)
        cout << "ID: " << val.first << " " << "Name: " << val.second << endl;
    return 0;
}
```

Output: ./map_example

```
ID: 100 Name: Sam
ID: 101 Name: Maha
ID: 102 Name: Bob
ID: 103 Name: Eve
ID: 104 Name: Charlie
ID: 105 Name: Samar
ID: 106 Name: Rass
ID: 108 Name: Kali
```


Accessing Elements on a Map

Maps, unlike arrays or vectors, cannot be accessed using index numbers since data is stored as key-value pairs rather than sequentially. Instead, we must access items using their keys. The square brackets [] are used with the key to get a value from a map.

C++ Accessing Elements Example

```
#include <iostream>
#include <map>
using namespace std;
int main()
{
    map<string, int> emp{"Sam", 2}, {"Kai", 4}, {"Mike", 1}, {"Dev", 3};
    cout << "Sam has " << emp["Sam"] << " years experiences.\n";
    cout << "Dev has " << emp["Dev"] << " years experiences.\n";
    cout << "Mike has " << emp.at("Mike") << " years experiences.\n";
    try
    {
        cout << "Samar has " << emp.at("Samar") << " years experiences.\n";
    }
    catch (const out_of_range& e)
    {
        cout << "Samar is not in the list of emp.\n";
    }
    return 0;
}
```

Output: ./map_accessing

Sam has 2 years experiences.

Dev has 3 years experiences.

Mike has 1 years experiences.

Samar has Samar is not in the list of emp.

Changing Values in a Map

In C++, we can alter the value of a specific key in a map by assigning a new value to that key. It is useful when we need to edit existing records, such as changing a salary, age, or score.

C++ Example to Change Values in a Map

```
#include <iostream>
#include <map>
using namespace std;
int main()
{
    map<string, int> emp_sal{"Kai", 38000}, {"Sam", 22000}, {"Raj", 50000};
    emp_sal["Raj"] = 90000;
    emp_sal.insert({"Dev", 150000});
    for(const auto& [name, sal] : emp_sal)
        cout << "Name:- " << name << " , Salary:- " << sal << endl;
    return 0;
}
```

Output: ./map Updating

Name:- Dev , Salary:- 150000

Name:- Kai , Salary:- 38000

Name:- Raj , Salary:- 90000

Name:- Sam , Salary:- 22000

Adding elements to a Map

The square brackets [] or the .insert() function is used to add elements to a map.

- **Square Brackets []:** It allows us to include new components to the map or change the value of an existing key. If the key does not exist, it generates a new

entry with the supplied value. If the key already exists, it will replace the current value.

- **.insert() Function:** This method creates new key-value pairs for the map. It does not overwrite existing keys. If the key already exists in the map, the insertion is ignored. It's useful when we want to make sure that only new keys are inserted without changing existing ones.

C++ Example to Add Elements to a Map

```
#include <iostream>
#include <map>
using namespace std;
int main()
{
    map<string, int> emp_sal{{"Kai", 38000}, {"Sam", 22000}, {"Raj", 50000}};
    emp_sal["Raja"] = 90000;
    emp_sal.insert({"Dev", 150000});
    for(const auto& [name, sal] : emp_sal)
        cout << "Name:- " << name << " , Salary:- " << sal << endl;
    return 0;
}
```

Output: ./map_adding

Name:- Dev , Salary:- 150000

Name:- Kai , Salary:- 38000

Name:- Raj , Salary:- 50000

Name:- Raja , Salary:- 90000

Name:- Sam , Salary:- 22000

Elements with Equal Keys

In C++, a map is not allowed to have duplicate keys. Each key in a map must be unique, which means that using .insert() function to insert an element with an existing key will result in the action being disregarded. Values might be the same, but keys must be separate.

C++ Elements with Equal Keys Example

```
#include <iostream>
#include <map>
using namespace std;
int main()
{
    map<string, int> emp_sal{{"Kai", 38000}, {"Sam", 22000}, {"Raj", 50000}};
    emp_sal["Raja"] = 90000;
    emp_sal.insert({"Dev", 150000});
    auto ans = emp_sal.insert({"Kai", 55600});
    if(!ans.second)
        cout << "Insertion Failed:- " << ans.first->first << " already present with
salary:- " << ans.first->second << endl;
    for(const auto& [name, sal] : emp_sal)
        cout << "Name:- " << name << " , Salary:- " << sal << endl;
    return 0;
}
```

Output: ./map_equal_key

Insertion Failed:- Kai already present with salary:- 38000

Name:- Dev , Salary:- 150000

Name:- Kai , Salary:- 38000

Name:- Raj , Salary:- 50000

Name:- Raja , Salary:- 90000

Name:- Sam , Salary:- 22000

Removing Elements from a Map

In C++, the `.erase(key)` function is used to remove a specific element. After that, call the `.clear()` method to remove all of the elements. These methods make it easier to manage and update your dataset in real-world applications.

C++ Example to Remove Elements from a Map

```
#include <iostream>
#include <map>
using namespace std;
int main()
{
    map<string, int> emp_sal{{"Kai", 38000}, {"Sam", 22000}, {"Raj", 50000}};
    emp_sal["Raja"] = 90000;
    emp_sal.insert({"Dev", 150000});
    auto ans = emp_sal.insert({"Kaii", 55600});
    if(!ans.second)
        cout << "Insertion Failed:- " << ans.first->first << " already present with
salary:- " << ans.first->second << endl;
    else
        cout << "Insertion Success:- " << ans.first->first << " already present with
salary:- " << ans.first->second << endl;
    cout << "Original Record:- \n";
    for(const auto& [name, sal] : emp_sal)
        cout << "Name:- " << name << " , Salary:- " << sal << endl;
    emp_sal.erase("Raja");
    cout << "Record After Removing \"Raja\":- \n";
    for(const auto& [name, sal] : emp_sal)
        cout << "Name:- " << name << " , Salary:- " << sal << endl;
    emp_sal.clear();
    cout << "Record After Clear:- \n";
    for(const auto& [name, sal] : emp_sal)
        cout << "Name:- " << name << " , Salary:- " << sal << endl;
    return 0;
}
```

Output: `./map_removing`

Insertion Success:- Kaii already present with salary:- 55600

Original Record:-

Name:- Dev , Salary:- 150000

Name:- Kai , Salary:- 38000

Name:- Kaii , Salary:- 55600

Name:- Raj , Salary:- 50000

Name:- Raja , Salary:- 90000

Name:- Sam , Salary:- 22000

Record After Removing "Raja":-

Name:- Dev , Salary:- 150000

Name:- Kai , Salary:- 38000

Name:- Kaii , Salary:- 55600

Name:- Raj , Salary:- 50000

Name:- Sam , Salary:- 22000

Record After Clear:-

Finding the Size of a Map

In C++, the `.size()` function of a map returns the total number of key-value pairs that are currently stored in it. In real-time applications, it is frequently used to count the amount of records, such as users, products, and employees.

C++ Example to find the size of a Map

```
#include <iostream>
#include <map>
using namespace std;
```

```

int main()
{
    map<string, int> emp_sal{{"Kai", 38000}, {"Sam", 22000}, {"Raj", 50000}};
    emp_sal["Raja"] = 90000;
    emp_sal.insert({"Dev", 150000});
    auto ans = emp_sal.insert({"Kaii", 55600});
    if(!ans.second)
        cout << "Insertion Failed:- " << ans.first->first << " already present with
salary:- " << ans.first->second << endl;
    else
        cout << "Insertion Success:- " << ans.first->first << " already present with
salary:- " << ans.first->second << endl;
        cout << "Before remove (Raja), Size of the map (emp_sal):- " << emp_sal.size()
<< endl;
        emp_sal.erase("Raja");
        cout << "After remove (Raja), Size of the map (emp_sal):- " << emp_sal.size() <<
endl;
        return 0;
}

```

Output: ./map_size

Insertion Success:- Kaii already present with salary:- 55600

Before remove (Raja), Size of the map (emp_sal):- 6

After remove (Raja), Size of the map (emp_sal):- 5

Checking if a Map is Empty

In C++, the .empty() method determines whether a map includes any key-value pairs. If the map is empty, it returns true (1); otherwise, it returns false(0). Similarly, the .count(key) function determines whether a given key exists in the map, which returns 1 if it does and 0 otherwise.

C++ Example to Check a Map is Empty

```

#include <iostream>
#include <map>
using namespace std;
int main()
{
    map<string, int> emp_sal{{"Kai", 38000}, {"Sam", 22000}, {"Raj", 50000}};
    emp_sal["Raja"] = 90000;
    emp_sal.insert({"Dev", 150000});
    auto ans = emp_sal.insert({"Kaii", 55600});
    if(!ans.second)
        cout << "Insertion Failed:- " << ans.first->first << " already present with
salary:- " << ans.first->second << endl;
    else
        cout << "Insertion Success:- " << ans.first->first << " already present with
salary:- " << ans.first->second << endl;
        cout << "Before clear(), Checking empty():- " << (!emp_sal.empty() ? "Not
Empty." : "Empty") << endl;
        emp_sal.clear();
        cout << "After clear(), Checking empty():- " << (emp_sal.empty() ? "Empty." :
"Not Empty." )<< endl;
        return 0;
}

```

Output: ./map_empty

Insertion Success:- Kaii already present with salary:- 55600

Before clear(), Checking empty():- Not Empty.

After clear(), Checking empty():- Empty.

Loop Through a Map

In C++, we can iterate through a map using a range-based for loop and the `auto` keyword. Each map element is a key-value pair that can be retrieved with the `.first()` function for the key and `.second` for the value. Maps sort keys in ascending order unless otherwise provided, such as `greater` for descending order.

C++ Example to Loop through a Map

```
#include <iostream>
#include <map>
#include <string>
#include <functional>
using namespace std;
int main()
{
    map< string, int, greater< string>> check_in;
    check_in["Akshay"] = 25;
    check_in["Sanjay"] = 27;
    check_in["Prasad"] = 30;
    for (auto q : check_in)
    {
        cout << q.first << " attended: " << q.second << " days" << endl;
    }
    return 0;
}
```

Output: ./map_loop

Sanjay attended: 27 days

Prasad attended: 30 days

Akshay attended: 25 days

C++ Files and Streams

In C++ programming we are using the iostream standard library, it provides cin and cout methods for reading from input and writing to output respectively.

To read and write from a file we are using the standard C++ library called **fstream**.

Data Type	Description
fstream	It is used to create files, write information to files, and read information from files.
ifstream	It is used to read information from files.
ofstream	It is used to create files and write information to the files.

C++ FileStream example: writing to a file

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ofstream filestream("write_to_file.txt");
    if(filestream.is_open())
    {
        filestream << "This is the test file to learn filestream.\n";
        filestream << "And I'm trying to learn this concept well.\n";
        filestream << "Thankd You.";
        filestream.close();
    }
    else
        cout << "File opening is Fail.\n";
    return 0;
}
```

Output: cat write_to_file.txt

This is the test file to learn filestream.

And I'm trying to learn this concept well.

Thankd You.

C++ FileStream example: reading from a file

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ifstream filestream("write_to_file.txt");
    string line;
    char ch;
    if(filestream.is_open())
    {
        // To read Line by Line
        /*while(getline(filestream, line))
        {
            cout << line << "\n";
        }*/

        // To read word by word
        /*while(filestream >> line)
        {
            cout << line << "\n";
        }*/

        // To read char by char
        while(filestream.get(ch))
        {
```

```

        cout << ch << " ";
    }
    filestream.close();
}
else
    cout << "Fail to open file for reading.\n";
return 0;
}

```

Output: // line by line

./read_to_file

This is the test file to learn filestream.

And I'm trying to learn this concept well.

Thankd You.

Output: //word by word

./read_to_file

This

is

the

test

file

to

learn

filestream.

And

I'm

trying

to

learn

this

concept

well.

Thankd

You.

Output: //char by char

./read_to_file

This is the test file to learn filestream.

And I'm trying to learn this concept well.

Thankd You.

C++ Read and Write Example

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    // Writing to file
```

```
    ofstream fin("write-read_to_file.txt");
```

```
    if(fin.is_open())
```

```
    {
```

```
        fin << "This is the code for learning.\n";
```

```
        fin << "Thank You.\n";
```

```
        fin.close();
```

```
    }
```

```
    else
```

```
        cout << "Fail to open file for writing.\n";
```

```
    // Reading from file
```

```

ifstream fou("write-read_to_file.txt");
string line;
if(fou.is_open())
{
    while(getline(fou, line))
    {
        cout << line << endl;
    }
    fou.close();
}
else
    cout << "Fail to open file for reading.\n";
return 0;
}

```

Output: ./read-write_to_file
This is the code for learning.
Thank You.

C++ **getline()**

In C++, the `getline()` is a built-in function defined in the `<string>` header file that allows reading a full line of text from an input stream. The `cin` is an object which is used to take input from the user but does not allow it to take the input in multiple lines.

We use the `getline()` function to accept the multiple lines. It enables reading a complete line, including whitespace characters, which makes it a powerful tool for handling more complex input scenarios in C++.

Syntax of getline() function:

1. The first way of declaring is to pass three parameters.

```
istream& getline( istream& is, string& str, char delim );
```

Where,

is: It is an object of the `istream` class that defines from where to read the input stream.

str: It is a string object in which string is stored.

delim: It is the delimiting character.

Return value

This function returns the input stream object, which is passed as a parameter to the function.

2. The second way of declaring is to pass two parameters.

```
istream& getline( istream& is, string& str );
```

Where,

is: It is an object of the `istream` class that defines from where to read the input stream.

str: It is a string object in which string is stored.

Return value

This function also returns the input stream, which is passed as a parameter to the function.

C++ `getline()` Function Example:

```

#include <iostream>
using namespace std;
int main()
{
    string name;
    cout << "Enter name:- ";
    cin >> name;
    cout << "Hello, " << name << endl;
    return 0;
}

```

Output: ./getline_2_para
Enter name:- Samaranjan Manjhi

Hello, Samaranjan

Now using getline to resolve the problem of the incomplete name in output.

Example

```
#include <iostream>
using namespace std;
int main()
{
    string name;
    cout << "Enter name:- ";
    getline(cin, name);
    cout << "Hello, " << name << endl;
    return 0;
}
```

Output: ./getline_2_para_1

Enter name:- Samaranjan Manjhi

Hello, Samaranjan Manjhi

Using the getline() with a Delimiter to Stop Input at a Space in C++

Example

```
#include <iostream>
using namespace std;
int main()
{
    string name;
    cout << "Enter name:- ";
    getline(cin, name, 'z');
    cout << "Hello, " << name << endl;
    return 0;
}
```

Output: ./getline_2_para_delimiter

Enter name:- Samaranjan Manjhi Son of Pankaj Manjhi z talk

Hello, Samaranjan Manjhi Son of Pankaj Manjhi

Getline Character Array:

We can also define the getline() function for the character array, but its syntax is different from the previous one.

Syntax:

```
istream& getline(char* , int size);
```

Where,

- **char*:** It is a character pointer that points to the array.
- **Size:** It acts as a delimiter that defines the size of the array means input cannot cross this size.

C++ Getline Character Array Example:

```
#include <iostream>
using namespace std;
int main()
{
    char name[15] = "";
    cout << "Enter name:- ";
    cin.getline(name, 15);
    cout << "Hello, " << name << endl;
    return 0;
}
```

Output: ./getline_char_array

Enter name:- Samaranjan Manjhi

Hello, Samaranjan Man

Note: Last byte for null terminator '\0', so output will have always size (n-1) char.

C++ Signal Handling

In C++, signals are the interrupts, which are delivered to a process by the operating system to stop its ongoing task and attend to the task for which the interrupt has been generated. Interrupts are used to notify the program of specific events or conditions, such as an error, a termination request, or an external input. When a signal is received, the operating system stops the program's usual execution to handle it.

In C++, signals can also be generated by the operating system on the basis of system or error conditions. We can generate interrupts by pressing Ctrl+ C on Linux, UNIX, Mac OS X, or Windows system.

When a signal is given, the system can:

- Terminate the program.
- Pause it
- Just ignore it.
- Alternatively, invoke a programmer-defined custom handler function.

Catching and Handling Signals

In C++, not all signals can be caught or handled. Some signals are non-catchable, such as SIGKILL or SIGSTOP, and the operating system strictly enforces them to ensure system stability. Many other signals, such as SIGINT, SIGFPE, SIGSEGV, and SIGTERM, can be intercepted and handled within a program.

If we want to work with signals in C++, utilize the <csignal> header, which provides signal names and functions, such as signal() to associate a handler with a signal.

C++ Simple Signal Handling Example:

```
#include <iostream>
#include <unistd.h>
#include <signal.h>
using namespace std;
void sig_handler(int signum)
{
    cout << "\nInterrupt signal (" << signum << ") received. Program will now
terminate.\n";
    exit(signum);
}
int main()
{
    signal(SIGINT, sig_handler);
    string str = "Ram";
    string temp = "";
    cout << "Binary is running, To stop press Ctrl + C.\n";
    while(1)
    {
        if(temp.empty())
            temp = str;
        else
            temp += " " + str;
        cout << temp << endl;
        sleep(3);
    }
    return 0;
}
```

Output: ./signal_example

Binary is running, To stop press Ctrl + C.

Ram

Ram Ram

Ram Ram Ram

^C

Interrupt signal (2) received. Program will now terminate.

Exception Handling in C++

In C++, exception handling is a technique to handle runtime errors. It is a technique that is used to deal with unwanted and unexpected events that occur during the program execution. The main purpose of exception handling is to handle the normal flow of the application, even after runtime errors.

In C++, the exception is an event that is thrown at runtime to show an error. Most exceptions are derived from the `std::exception` class, which is a part of the C++ Standard Library. If an exception is not maintained effectively, the program will print an error message and terminate exceptionally.

For Instance:

```
divide_by_zero = 5 / 0;
```

Syntax:

```
try {  
    // Code that cause an exception  
    throw argument;  
}  
catch (exception) {  
    // code to handle the exception  
}
```

In this syntax, the code can generate an exception inside the try block, and the catch block follows every try block. If an exception occurs in the code, the throw statement throws an exception that is found by the catch block.

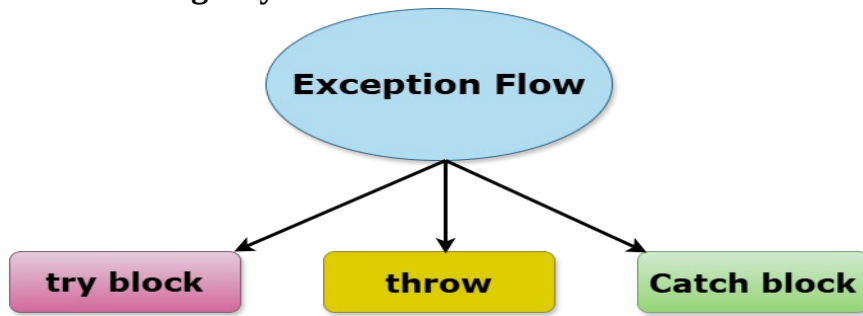
C++ Simple Exception Handling Example:

```
#include <iostream>  
using namespace std;  
int main(int argc, char** argv)  
{  
    int numo, deno;  
    if(argc < 2)  
    {  
        numo = 10;  
        deno = 2;  
    }  
    else  
    {  
        numo = atoi(argv[1]);  
        deno = atoi(argv[2]);  
    }  
    try  
    {  
        if(deno == 0)  
            throw "deno is 0.";  
        cout << "Result:- " << numo/deno << endl;  
    }  
    catch(const char* msg)  
    {  
        cout << "Execption Caught:- " << msg << endl;  
    }  
    return 0;  
}
```

Output:

```
./exception_example 20 0  
Execption Caught:- deno is 0.  
./exception_example 20 5  
Result:- 4  
./exception_example  
Result:- 5
```

C++ Exception Handling Keywords



In C++, we mainly used 3 keywords to perform exception handling:

- try Block
- throw Statement
- catch Block

1. try block

The try block is utilized to identify the code block that can cause an error. If an exception occurs inside the try block, the control flow moves to the catch block.

Syntax of try Block:

```
try {  
    // Code that may cause an error  
}
```

2. throw Statement

If an error happens, we can use throw to send an error message or value. It is performed using a throw keyword.

Syntax:

```
throw value; // value can be a number, string, or object
```

3. catch Block

The catch block gets the error thrown by throw. It helps to fix or show the error message. It only runs if there is a problem in the try block. The catch keyword shows the catch exception in the program.

Syntax:

```
catch(type variable) {  
    // Code to handle the exception  
}
```

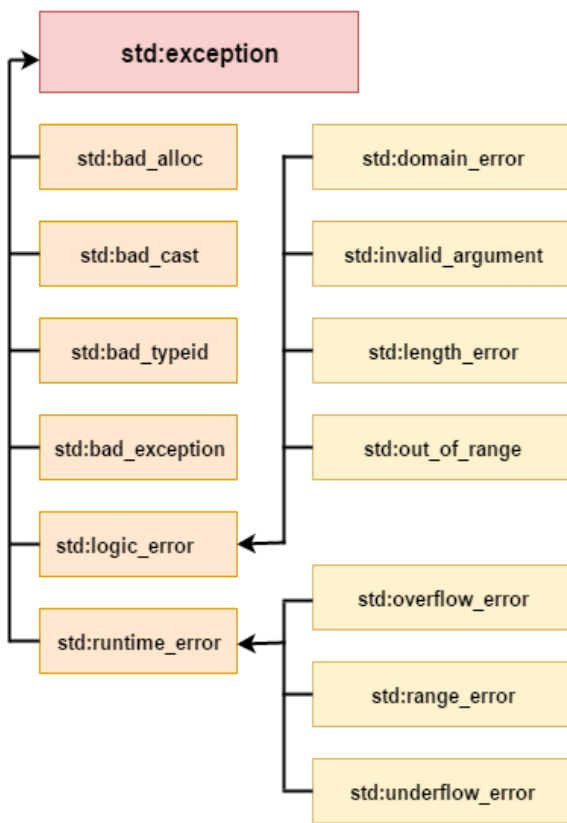
Why Exception Handling is used in C++?

In C++, exception handling is used to handle and respond to unexpected runtime errors in the program. In the program execution, several errors can occur, such as division by zero, file access failure, invalid input, and many more. If we don't use exception handling, these errors will cause the program to terminate and unexpected behavior.

If we use exception handling, we can catch these errors using try, catch, and throw statements. It helps to improve the code readability by separating normal logic from error-handling logic, which makes the program more robust, user-friendly, and easy to debug.

C++ Exception Classes

In C++, standard exceptions are specified in <exception> class that we can utilize inside our programs. The arrangement of the parent-child class hierarchy is shown in the given diagram:



Exception

Description

<code>std::exception</code>	It is an exception and parent class of all standard C++ exceptions.
<code>std::logic_error</code>	It is an exception that can be detected by reading a code.
<code>std::runtime_error</code>	It is an exception that cannot be detected by reading a code.
<code>std::bad_exception</code>	It is used to handle the unexpected exceptions in a c++ program.
<code>std::bad_cast</code>	This exception is generally thrown by dynamic_cast .
<code>std::bad_typeid</code>	This exception is generally thrown by typeid .
<code>std::bad_alloc</code>	This exception is generally thrown by new .

Throwing Exceptions:

In C++, exceptions are thrown using the **throw** keyword. When an exception is thrown in the program, the normal execution of the program stops, and the control is moved to the nearest catch block that may handle the exception. If no catch block is found in the code, the program terminates.

An exception can be any valid C++ object, but typically, we throw objects of standard exception classes or user-defined exceptions.

1) Throwing Exception Example:

```

#include <iostream>
int main()
{
    try
    {
        throw std::exception();
    }
    catch (const std::exception& e)
    {
        std::cout << "Caught std::execption: " << e.what() << std::endl;
    }
    return 0;
}

```

Output: ./exception_std

Caught std::exception: std::exception

2) Bad Alloc Exception Example:

```

#include <iostream>

```

```

using namespace std;
int main()
{
    try
    {
        int* p = new int[9999999999999999LL];
        (void)p;
    }
    catch (const bad_alloc& e)
    {
        cout << "Caught:- " << e.what() << endl;
    }
    return 0;
}

```

Output: ./exception_bad_alloc

Caught:- std::bad_alloc

3) Bad Cast Exception Example:

```

#include <iostream>
using namespace std;
class Animal {
public:
    virtual ~Animal() {}
};
class Dog : public Animal {};
class Cat : public Animal {};
int main()
{
    Dog dog;
    Animal& animalRef = dog;
    try
    {
        Cat& cat = dynamic_cast<Cat&>(animalRef);
        (void)cat;
    }
    catch(const bad_cast& e)
    {
        cout << "Caught:- " << e.what() << endl;
    }
    return 0;
}

```

Output: ./exception_bad_cast

Caught:- std::bad_cast

4) Bad Typeid Exception Example:

```

#include <iostream>
using namespace std;
class Shape {
public:
    virtual ~Shape() {}
};
int main()
{
    Shape* ptr = NULL;
    try
    {
        cout << typeid(*ptr).name() << endl;
    }
    catch (const bad_typeid& e)
    {

```

```

        cout << "Caught:- " << e.what() << endl;
    }
    return 0;
}

```

Output: ./exception_bad_typeid

Caught:- std::bad_typeid

5) Bad Exception Exception Example:

```

#include <iostream>
#include <exception>
int main() {
    try {
        throw std::bad_exception();
    } catch (const std::bad_exception& e) {
        std::cout << "Caught std::bad_exception: " << e.what() << std::endl;
    }

    // Real-world context: std::current_exception + std::rethrow_exception
    std::exception_ptr eptr;
    try {
        throw std::runtime_error("original error");
    } catch (...) {
        eptr = std::current_exception(); // capture any exception
    }

    try {
        if (eptr) std::rethrow_exception(eptr);
    } catch (const std::exception& e) {
        std::cout << "Re-caught via exception_ptr: " << e.what() << std::endl;
    }

    return 0;
}

```

Output: ./exception_bad_exception

Caught std::bad_exception: std::bad_exception

Re-caught via exception_ptr: original error

6) Bad Runtime Exception Example:

```

#include <iostream>
#include <fstream>
#include <string>
void loadConfig(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open())
        throw std::runtime_error("Failed to open config file: " + filename);
    std::string line;
    while (std::getline(file, line)) {
        std::cout << line << std::endl;
    }
}
int main() {
    // Example 1: File not found — only discoverable at runtime
    try {
        loadConfig("config.txt");
    } catch (const std::runtime_error& e) {
        std::cout << "Caught std::runtime_error: " << e.what() << std::endl;
    }
    // Example 2: Manually throwing runtime_error
    try {
        bool networkAvailable = false;

```

```

        if (!networkAvailable)
            throw std::runtime_error("Network connection unavailable");
    } catch (const std::runtime_error& e) {
        std::cout << "Caught std::runtime_error: " << e.what() << std::endl;
    }
    return 0;
}

```

Output: ./exception_runtime

Caught std::runtime_error: Failed to open config file: config.txt

Caught std::runtime_error: Network connection unavailable

C++ User-Defined Exceptions

The new exception can be defined by overriding and inheriting **exception** class functionality.

C++ user-defined exception example

```

#include <iostream>
#include <exception>
using namespace std;
class MyException: public exception {
public:
    const char* what() const throw()
    {
        return "Attempted to divide by 0.";
    }
};
int main()
{
    try
    {
        int x, y;
        cout << "Enter 2 nos. ";
        cin >> x >> y;
        if(y == 0)
        {
            MyException z;
            throw z;
        }
        else
            cout << "x/y = " << (float)x/y << endl;
    }
    catch (exception& e)
    {
        cout << e.what() << endl;
    }
    return 0;
}

```

Output:

./exception_user_define

Enter 2 nos. 15

5

x/y = 3

./exception_user_define

Enter 2 nos. 10

0

Attempted to divide by 0.

./exception_user_define

Enter 2 nos. 10

100

x/y = 0.1